

User-Mode Linux

A Comprehensive Technical Report

History, Architecture, Implementation, and the v2 Redesign

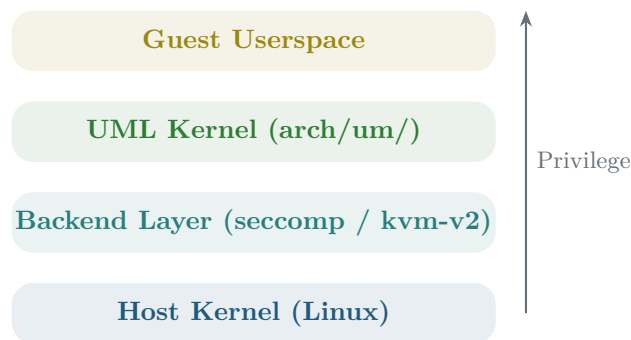
Historical reporting artifact

This report is archived at its May 2026 reporting cutoff. Current UML v2 status lives in `Documentation/virt/uml/redesign/STATUS.md` and the live sequencing inventory, not in this report.

Michael J. Bommarito II

`michael.bommarito@gmail.com`

May 2026



Based on the `mjbommar/linux` kernel tree, branch `next`
7 series-level commits on `torvalds/master` (v7.1-rc5+)

Prepared with assistance from Claude (Anthropic)

Contents

1	Introduction	1
1.1	What is User-Mode Linux?	1
1.2	Why UML Matters	2
1.2.1	The real kernel, not a reimplementaion	2
1.2.2	Full user/kernel separation	2
1.2.3	No hardware virtualization requirements	2
1.2.4	Application domains	3
1.3	The Current Moment	3
1.4	Report Structure	4
1.5	Contributions of This Work	5
2	History of User-Mode Linux	7
2.1	The Linux Kernel Before UML (1991–1999)	7
2.1.1	The arch/ Abstraction	7
2.1.2	The Debugging Problem	8
2.2	Jeff Dike and the Creation of UML (1999–2001)	8
2.2.1	The Initial Announcement	9
2.2.2	The Tracing Thread (TT) Mode	9
2.2.3	Key Insight: UML Is Not an Emulator	10
2.3	The SKAS Evolution (2002–2006)	10
2.3.1	The Problem with Shared Address Spaces	10
2.3.2	SKAS0: No Host Kernel Patches Required	10
2.3.3	SKAS3: The /proc/mm Approach	10
2.3.4	SKAS4 and the OLS Talks	11
2.3.5	The Prentice Hall Book	11
2.4	Mainline Merge and the Golden Era (2002–2010)	11
2.4.1	Kernel Development and Debugging	11
2.4.2	Netkit and Network Simulation	12
2.4.3	Security Research and Honeypots	12
2.4.4	Virtual Hosting	12
2.5	Decline and Maintenance Mode (2010–2020)	12
2.5.1	Hardware Virtualization Changes the Landscape	13
2.5.2	KVM: Virtualization in the Kernel	13
2.5.3	Containers Capture the Lightweight Market	13
2.5.4	Code Rot and Declining Maintenance	14
2.6	The Linux Virtualization Timeline	14
2.6.1	Xen and Paravirtualization (2003)	14
2.6.2	Intel VT-x and AMD-V (2005–2006)	14
2.6.3	KVM Enters Mainline (2007)	15
2.6.4	Cgroups and Namespaces Mature (2008–2013)	15
2.6.5	gVisor: A Different Approach (2018)	15
2.6.6	Firecracker: MicroVMs for Serverless (2020)	15

2.7	The Revival (2020–2026)	16
2.7.1	Benjamin Berg and the Seccomp Backend	16
2.7.2	Tiwei Bie and SMP Support	16
2.7.3	Anton Ivanov and Virtio-UML	16
2.7.4	Hajime Tazaki and the LKL-into-UML RFC	17
2.7.5	The Coding Assistants Policy	17
2.7.6	The v2 Architecture	17
2.8	Timeline	17
2.9	Lessons from History	17
3	Technical Foundations	21
3.1	Virtualization Taxonomy	21
3.1.1	The Popek–Goldberg Requirements	21
3.1.2	Taxonomy of Hypervisors	21
3.1.3	Where User-Mode Kernels Fit	22
3.2	The x86 Privilege Model	22
3.2.1	Rings, CPL, and DPL	22
3.2.2	Implications for UML	23
3.3	Hardware Virtualization Extensions	23
3.3.1	Intel VT-x (VMX)	23
3.3.2	AMD-V (SVM)	24
3.3.3	KVM and the ioctl Interface	24
3.3.4	Why This Matters for UML	24
3.4	The Linux Process Model	25
3.4.1	ptrace: The Original Trap Mechanism	25
3.4.2	seccomp: The Modern Trap Mechanism	25
3.4.3	futex: Stub Synchronization	25
3.4.4	mmap / munmap / mprotect: Guest Memory	25
3.4.5	clone / fork: Guest Processes	25
3.4.6	KVM ioctl Interface	26
3.5	Address Space Layout	26
3.5.1	Seccomp Backend Layout	26
3.5.2	KVM Backend: Guest Page Tables	27
3.6	Trap Mechanisms Compared	27
3.6.1	ptrace: PTRACE_SYSEMU	27
3.6.2	seccomp: SIGSYS + futex	28
3.6.3	KVM: vmentry/vmexit + LSTAR Gadget	28
3.6.4	Comparison Table	29
3.7	Memory Management Under UML	29
3.7.1	Guest Physical Memory	29
3.7.2	Guest Page Tables	30
3.7.3	The TLB Synchronization Problem	30
3.8	Signal Handling	31
3.8.1	Signal Delivery Across Backends	31
3.8.2	Signal Masking and Reentrancy	32
4	UML Architecture and Design	33
4.1	The arch/um/ Directory	33
4.1.1	Directory Structure	33
4.2	The Three-Layer Architecture	34
4.2.1	Layer 1: The Backend Ops Table	34
4.2.2	Layer 2: Static-Key Hot-Path Gates	36

4.2.3	Layer 3: Compile-Time Wraps	37
4.2.4	Cost Model	37
4.3	Backend Selection Mechanism	38
4.3.1	Build-Time: Kconfig	38
4.3.2	Boot-Time: <code>init_backend()</code>	38
4.4	The Stub Mechanism	39
4.4.1	Stub Lifecycle	39
4.4.2	The <code>stub_data</code> Structure	39
4.4.3	Futex-Based Synchronization	40
4.5	Guest Memory Management	41
4.5.1	The TLB Synchronization Layer	41
4.5.2	SMP TLB Generation Tracking	41
4.5.3	Deferred Free: The <code>mmu_gather</code> Integration	42
4.6	SMP Implementation	42
4.6.1	Thread Model	42
4.6.2	Context Switching	42
4.6.3	IPI Delivery	43
4.6.4	The <code>migrate_disable</code> Fix (SMP-T13)	43
4.7	Device Model	44
4.8	Build System	44
4.8.1	Basic Build	44
4.8.2	The <code>0=</code> Discipline	44
4.8.3	Kconfig Structure	45
4.8.4	SUBARCH and Host Architecture	45
4.9	Chapter Summary	45
5	UML Implementation Details	49
5.1	Process Lifecycle	49
5.1.1	Creation: <code>fork</code> through <code>copy_thread</code>	49
5.1.2	The <code>thread_struct</code> and Per-Task State	50
5.1.3	Context Switch: <code>__switch_to</code>	50
5.1.4	Destruction	51
5.2	The Seccomp Backend: Trap Path Walkthrough	51
5.2.1	Step 1: Guest Issues a System Call	51
5.2.2	Step 2: <code>stub_signal_interrupt</code> — The Stub Handler	51
5.2.3	The Futex Protocol	52
5.2.4	The <code>stub_data</code> Structure	53
5.3	Memory Management Implementation	54
5.3.1	Physical Memory: The Backing File	54
5.3.2	The TLB Sync Path: <code>um_tlb_sync</code>	55
5.3.3	The <code>mm_context_t</code> and Generation Tracking	55
5.3.4	The <code>um_memory_region</code> Descriptor	56
5.4	Signal Handling	56
5.4.1	Signal Installation and Masking	57
5.4.2	Signals as Interrupts: The Enable/Disable Discipline	57
5.4.3	The Timer Interrupt	58
5.5	The <code>hostfs</code> Filesystem	58
5.5.1	Architecture	58
5.5.2	<code>io_uring</code> Integration for Async Writeback	58
5.5.3	Security: <code>openat2</code> and <code>RESOLVE_BENEATH</code>	59
5.6	Boot Sequence	59

5.6.1	Stage 1: Host Process Setup (<code>os-Linux/main.c</code>)	59
5.6.2	Stage 2: Kernel Pre-Boot (<code>linux_main</code>)	59
5.6.3	Stage 3: Standard Linux Boot (<code>start_kernel</code>)	59
5.7	The Backend Ops Table	60
5.7.1	Structure Definition	60
5.7.2	Dispatch Mechanism	61
5.7.3	Backend Selection at Boot	61
5.7.4	Capability Flags	62
5.8	Key Code Listings: Annotated Excerpts	62
5.8.1	The Complete Futex Protocol in <code>stub_signal_interrupt</code>	62
5.8.2	The <code>syscall_handler</code> Batch Processor	63
6	The KVM v2 Backend	65
6.1	Motivation	65
6.2	Architecture Overview	66
6.3	The Trap Path	67
6.3.1	Step-by-step walk-through	67
6.3.2	The SYSCALL mask	69
6.4	The LSTAR Gadget	69
6.4.1	Handled system calls	69
6.4.2	Per-vCPU state page mechanism	69
6.4.3	Dispatch tree layout	70
6.4.4	Performance impact	70
6.4.5	Record/replay bypass	70
6.5	Exception Handling	70
6.5.1	IDT, GDT, and TSS layout	71
6.5.2	Handler stubs	71
6.5.3	IST stacks and triple-fault prevention	71
6.5.4	The <code>#BP</code> handler and debugger support	71
6.6	Per-CPU vCPU Pool	72
6.6.1	Why per-CPU, not per-task	73
6.6.2	The <code>migrate_disable</code> discipline (SMP-T13)	73
6.7	SMP Challenges and Solutions	73
6.7.1	SMP-T13: <code>migrate_disable</code> (Section 6.6)	74
6.7.2	SMP-T26/T27: FPU cross-task leak	74
6.7.3	SMP-T29: fork-snapshot clobber	74
6.7.4	SMP-T33/T41: TDP <code>prev_roots</code> cache contamination	74
6.7.5	SMP-T55: lazy FPU epoch flag	75
6.7.6	SMP-T57: AVX/XSAVE enablement	75
6.7.7	SMP-T73: <code>KVM_GET_FPU</code> to <code>KVM_GET_XSAVE</code>	75
6.8	TLB Synchronization	75
6.9	Snapshot and Record/Replay	76
6.9.1	Snapshot	76
6.9.2	Record/replay	77
6.10	Performance Results	77
6.10.1	Python startup	78
6.10.2	Correctness: cpython parity	78
6.10.3	Overhead breakdown	78
6.11	Summary	78
7	Applications and Use Cases	81
7.1	Kernel Development and Debugging	81

7.1.1	The debugging workflow	81
7.1.2	kprobes and ftrace integration	82
7.1.3	Sanitizer support	82
7.2	Kernel Fuzzing	82
7.2.1	The current landscape: QEMU-KVM	83
7.2.2	The LKL precedent	83
7.2.3	UML as a fuzzing target	83
7.2.4	The syzkaller vm/uml backend	84
7.2.5	KCOV integration	84
7.3	Network Simulation	84
7.3.1	Netkit	84
7.3.2	Marionnet	85
7.3.3	Wireless network simulation	85
7.4	Security Research	85
7.4.1	Kernel exploit development and analysis	85
7.4.2	Advantage over containers	86
7.4.3	Honeypot deployment	86
7.4.4	Malware analysis	86
7.5	CI/CD and Automated Testing	86
7.5.1	The /dev/kvm problem	87
7.5.2	kselftest integration	87
7.5.3	Linux Test Project (LTP)	87
7.5.4	Bisection workflows	88
7.6	Education	88
7.6.1	UML as a teaching tool	88
7.6.2	Concrete course applications	88
7.6.3	Comparison with teaching alternatives	89
7.7	Record/Replay for Bug Reproduction	89
7.7.1	Existing approaches	89
7.7.2	UML record/replay	90
7.7.3	Time-travel debugging	90
7.8	Snapshot/Restore for Fast Iteration	90
7.8.1	Fuzzing reset	90
7.8.2	Debugging iteration	91
7.8.3	Test parallelization	91
7.9	Summary	91
8	Related Work	93
8.1	gVisor	93
8.1.1	Architecture	93
8.1.2	Key difference: reimplementation vs. the real kernel	94
8.1.3	Performance comparison	94
8.2	Firecracker	94
8.2.1	Architecture	94
8.2.2	Key difference: full VM vs. process	95
8.2.3	Performance	96
8.3	Linux Kernel Library (LKL)	96
8.3.1	Architecture	96
8.3.2	Performance	96
8.3.3	Limitations	96
8.3.4	The LKL–UML unification effort	97

8.4	QEMU and KVM	97
8.4.1	Architecture	97
8.4.2	Comparison with UML	98
8.4.3	Complementary roles	98
8.5	Containers	99
8.5.1	Architecture	99
8.5.2	The shared-kernel problem	99
8.5.3	Comparison with UML	99
8.6	Unikernels	99
8.6.1	Architecture	99
8.6.2	Comparison with UML	100
8.7	Dune	100
8.7.1	Architecture	100
8.7.2	Relationship to UML’s KVM backend	101
8.8	Library OS: Graphene / Gramine	101
8.8.1	Architecture	101
8.8.2	Comparison with UML	101
8.9	WSL1	102
8.9.1	Architecture	102
8.9.2	The reimplementation problem	102
8.9.3	Lessons for UML	102
8.10	Comprehensive Comparison	103
8.11	Architectural Taxonomy	103
8.11.1	Execution substrate	103
8.11.2	Kernel model	104
8.12	Summary	105
9	The v2 Redesign Project	107
9.1	Project Overview	107
9.2	Design Goals	107
9.3	What We Are Not Building	108
9.4	The Series Breakdown	109
9.4.1	Series 1: BPF JIT Stub and x86 Hygiene	109
9.4.2	Series 2: KMSAN Early-Shadow Callback	109
9.4.3	Series 3: notrace on Signal-Adjacent Thread Functions	110
9.4.4	Series 4: Backend Ops Abstraction (Tentpole RFC)	110
9.4.5	Series 5: KVM v2 Backend	110
9.4.6	Series 6: umlctl and Kselftests	111
9.4.7	Series 7: Documentation	112
9.5	Validation Methodology	112
9.5.1	The Validation Ladder	112
9.5.2	The umlctl mission Gate	112
9.5.3	Third-Party and Real-World Tests	113
9.6	The CPython Bug Discovery	113
9.7	Upstream Submission Plan	113
9.7.1	Phase 1: Independent Subsystem Patches (Series 1–3)	114
9.7.2	Phase 2: The Tentpole RFC (Series 4)	114
9.7.3	Phase 3: KVM Backend and Tooling (Series 5–7)	114
9.7.4	The LKML Strategy	114
9.8	Lessons Learned	114
10	Conclusion and Future Work	117

10.1	Summary of Contributions	117
10.2	Current State (May 2026)	117
10.3	What Remains	118
10.3.1	Phase J: 24-Hour Continuous Soak	118
10.3.2	Upstream Acceptance	118
10.3.3	Architecture Ports	118
10.3.4	syzkaller Integration	119
10.3.5	Record/Replay Host-Side Wiring	119
10.3.6	Additional Validation	119
10.4	The Broader Impact	119
10.4.1	Full Kernel, Full Debugging, No Hardware	119
10.4.2	The Cloud-Native and eBPF Context	120
10.4.3	Security and Reproducibility	120
10.5	Closing Thoughts	120
	Bibliography	121
	Source Code Map	125
.1	Overall Statistics	125
.2	Architecture Port: <code>arch/um/</code>	125
.3	Host-Side Tooling: <code>tools/uml/</code>	127
.4	Kselftest Suite: <code>tools/testing/selftests/um/</code>	128
.5	File Type Distribution	130
	Benchmark Data	131
.6	Methodology	131
.7	Per-Syscall Latency	131
.8	Cross-Host Benchmark Results	132
.8.1	Micro Tier	134
.8.2	Application Tier (Python Startup)	134
.8.3	Stress Tier	134
.9	Snapshot Performance	134
.10	CPython Substrate Parity	135
.11	Validation Gates	135
.12	Profile Cost Model	135
.13	Summary of Performance Claims	137

List of Figures

1.1	UML architectural layers	2
2.1	Timeline of User-Mode Linux development in the context of Linux kernel and virtualization history (1991–2026). Upper events relate to UML and the Linux kernel; lower events show competing virtualization technologies that shaped UML’s trajectory.	18
3.1	The x86 privilege ring model. Traditional kernels occupy Ring 0; UML runs entirely at Ring 3 (CPL=3), sharing the same privilege level as ordinary user processes.	23
3.2	Address space layout for the <code>seccomp</code> backend. The KVM backend additionally constructs guest-side PML4 page tables that map the same physical memory through EPT/NPT.	27
3.3	System call interception paths for the three UML trap mechanisms. Dashed arrows show the return-to-guest path.	29
4.1	The three-layer architecture of UML’s v2 redesign. Each layer composes orthogonally with the others.	35
4.2	Backend selection flow: build-time Kconfig choice followed by boot-time parameter resolution.	39
4.3	Futex-based synchronization between the UML kernel and the <code>seccomp</code> stub child. The shared <code>stub_data.futex</code> field alternates between <code>FUTEX_IN_CHILD</code> and <code>FUTEX_IN_KERN</code>	40
4.4	UML SMP thread model: one host pthread per virtual CPU, each pinned to a host CPU via <code>pthread_setaffinity_np</code> . Guest tasks are scheduled within each pthread by UML’s copy of the Linux scheduler.	43
5.1	The <code>seccomp</code> trap path: complete lifecycle of a guest system call. Solid arrows show the control flow; dashed arrows show the futex-mediated handoff between the stub child and the UML kernel process.	53
5.2	UML boot sequence from <code>./linux</code> invocation to running <code>init</code>	60
6.1	KVM v2 architecture overview	67
6.2	KVM v2 trap path	68
6.3	PML4 kernel-half layout	72
6.4	Per-CPU vCPU pool	74
6.5	Per-syscall getpid latency	78
7.1	Kernel debugging ecosystem under UML	82
7.2	Netkit-style network topology	85
7.3	Snapshot-based test parallelization	91
8.1	gVisor vs. UML architecture	95
8.2	Speed–fidelity spectrum	97

8.3	Architectural taxonomy of isolation systems	105
1	Per-syscall round-trip cost comparison (log scale, cycles). The LSTAR gadget achieves $\sim 1,200\times$ improvement over the <code>seccomp</code> backend for trivial syscalls. LKL is shown for reference but sacrifices ring-split fidelity.	132
2	Micro-tier (<code>getpid-loop</code>) speedup. The variation across hosts reflects differences in <code>seccomp</code> notification overhead rather than KVM performance: the <code>kvm-v2</code> gadget cost is consistently 87–153 cycles, while <code>seccomp</code> costs vary from $\sim 15,000$ to $\sim 140,000$ cycles by host.	133
3	<code>kvm-v2</code> Python startup speedup over <code>seccomp</code> . The consistent $3\text{--}4\times$ band across diverse hardware confirms an architectural (backend mechanism) improvement, not a microarchitecture-specific one.	133
4	Stress tier (<code>mt-mini SMP T=8, ncpus=4</code>) speedup. The wider variance ($3.71\text{--}8.72\times$) reflects the <code>mmap/page-fault-heavy</code> workload where TLB shootdown cost and host scheduler behavior create per-host variation. Zero strict memory failures on any host.	134
5	Snapshot capture and restore latency. The $13\mu\text{s}$ median restore enables checkpoint-at-every-syscall workflows without budget pressure.	135
6	Profile cost model. The value of the profile system is the ability to span from direct-call library mode (6 ns) through heavily instrumented research and time-travel builds ($1.2\mu\text{s}$) without forking the tree. The <code>native</code> bar is a bare-metal host reference, not a UML profile.	136

List of Tables

3.1	Trap mechanism comparison. Latencies are representative measurements for a null system call (e.g., <code>getpid()</code>) on contemporary hardware (Intel Alder Lake / AMD Zen 4, 2024–2026).	30
4.1	Top-level directories under <code>arch/um/</code>	34
4.2	Per-syscall-roundtrip cost model across layers and profiles. Figures are representative; actual measurements vary with host microarchitecture and workload. . . .	38
4.3	UML device drivers and their host-side backing mechanisms.	47
6.1	LSTAR gadget handled system calls	69
6.2	Kernel-half page layout in <code>PML4[448]</code> / <code>PUD[0]</code> / <code>PMD[0]</code>	71
6.3	IO-port trap class encoding	72
6.4	SMP state-audit tracker summary (selected items)	76
6.5	Backend performance comparison	77
6.6	Overhead breakdown for a single guest syscall (KVM v2, exit path)	79
7.1	Fuzzing target comparison	83
7.2	Comparison of OS teaching environments	89
7.3	Application domains and key UML advantages	91
8.1	Comprehensive comparison of related systems	103
8.2	Qualitative property comparison	104

9.1	Profile matrix: nine named configurations from one tree.	108
9.2	Seven series-level commits on <code>torvalds/master</code>	109
1	Source tree summary statistics.	125
2	Directory structure of <code>arch/um/</code>	125
3	Directory structure of <code>tools/uml/</code> (excluding build artifacts).	127
4	Kselftest suites in <code>tools/testing/selftests/um/</code>	128
5	File type distribution across the v2 source tree.	130
6	Per-syscall (<code>getpid</code>) round-trip latency by backend.	131
7	Post-gadget <code>kvm-v2</code> speedup over <code>seccomp</code> across 8 hosts.	132
8	Snapshot benchmark (kernel <code>9ccbf5e7bb58</code> , AMD Ryzen 7 7840HS, $N = 64$). . . .	134
9	CPython substrate parity results.	135
10	Key validation gates (May 2026, kernel HEAD on <code>next</code>).	136
11	Per-syscall cost breakdown by profile and layer.	136
12	Summary of validated performance claims.	137

Chapter 1

Introduction

The best way to understand a kernel is to run it where you can poke at it.

Jeff Dike, 2000

1.1 What is User-Mode Linux?

User-Mode Linux (UML) is an architecture port of the Linux kernel in which the kernel itself executes as an ordinary userspace process on a host Linux system. Where a conventional kernel runs on bare metal—managing page tables, servicing interrupts, and scheduling threads with the full authority of CPU ring 0—UML performs all of these functions from ring 3, mediating access to host resources through the host kernel’s system call interface.

In the Linux source tree, UML lives alongside x86, ARM, RISC-V, and the other architecture ports under `arch/um/`. It shares the same scheduler, the same VFS layer, the same memory management subsystem, and the same networking stack as every other port. The code that differs is precisely the code that must differ: the low-level trap handling, context switching, timer management, and memory mapping routines that, on hardware architectures, speak directly to the CPU and chipset. In UML, these routines instead call `clone()`, `mmap()`, `sigaction()`, `ioctl()`, and a handful of other host system calls.

Jeff Dike created UML in 1999–2000 and presented it at the USENIX Annual Linux Showcase in Atlanta [1]. The original implementation used the `ptrace` system call to intercept guest system calls: the UML kernel ran as a tracing process that caught every `SIGTRAP` delivered when a guest userspace process invoked a system call. This mechanism, while conceptually elegant, imposed a heavy performance penalty—each guest syscall required multiple context switches between the tracing kernel and the traced guest—and the subsequent two decades of UML development can be read as a sustained effort to reduce that overhead.

The architecture is best understood through the Popek–Goldberg framework [2]. A *virtual machine monitor* (VMM) must faithfully reproduce the behavior of the underlying machine for guest software while retaining control over resource allocation. Traditional VMMs achieve this by exploiting hardware privilege rings: the guest kernel believes it runs in ring 0, but the VMM intercepts sensitive instructions from a more privileged mode. UML inverts this arrangement. There is no privilege separation *at the hardware level* between the UML kernel and its guests; instead, the host kernel enforces separation through the ordinary process isolation guarantees of Linux itself. The UML kernel and its guest processes are all host-level processes. What distinguishes them is that the UML kernel interposes on guest system calls before they can reach the host—first via `ptrace`, later via `seccomp` filters, and now, in the v2 redesign, optionally via KVM.

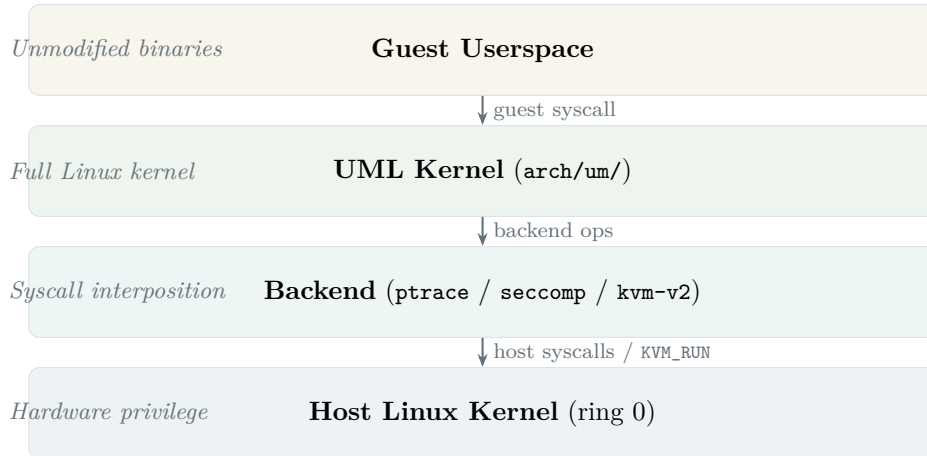


Figure 1.1: The four-layer UML architecture. Guest userspace programs issue system calls that are trapped by the UML kernel. The UML kernel services these using standard Linux kernel subsystems, performing I/O through the backend and host kernel layers.

1.2 Why UML Matters

In a landscape crowded with virtualization and sandboxing technologies—KVM, Xen [3], Firecracker [4], gVisor [5], containers, unikernels—it is reasonable to ask whether UML still has a role. The answer is emphatically yes, precisely because UML occupies a unique and underserved niche in the design space.

1.2.1 The real kernel, not a reimplementation

gVisor’s Sentry [5] reimplements the Linux system call interface in Go. It is an impressive feat of engineering, but it is not the Linux kernel: subtle semantic differences arise when the Sentry’s interpretation of a system call diverges from the kernel’s. The Linux Kernel Library (LKL) [6] compiles the kernel as a linkable library, but strips away process scheduling and user/kernel separation.

UML, by contrast, *is* the Linux kernel. Every syscall handler, every scheduler decision, every lock acquisition follows the same code paths as a kernel running on x86 hardware. A bug found in UML is a bug in mainline Linux, and a fix validated in UML is a fix for every architecture. This fidelity is not accidental—it is a consequence of UML being an `arch/` port, sharing all architecture-independent code with the rest of the tree.

1.2.2 Full user/kernel separation

Unlike LKL, which runs kernel code in the same address space and privilege level as the application, UML maintains genuine user/kernel separation. Guest userspace processes run in separate host processes with restricted memory mappings. A bug in a guest program cannot corrupt UML kernel data structures any more than a bug in a userspace program can corrupt a bare-metal kernel. This separation is essential for security research, kernel fuzzing, and any application where the guest is untrusted.

1.2.3 No hardware virtualization requirements

KVM [7] and Firecracker [8] require hardware virtualization extensions (Intel VT-x or AMD-V) and access to `/dev/kvm`. In many environments—containerized CI/CD runners, shared cloud instances, locked-down workstations—`/dev/kvm` is unavailable. UML with the `ptrace` or `seccomp`

backends has no such requirement: it runs wherever a Linux userspace process can run. This makes it uniquely suited for environments where hardware-assisted virtualization is either absent or administratively prohibited.

1.2.4 Application domains

The combination of kernel fidelity, process isolation, and minimal host requirements makes UML valuable across a range of application domains:

Kernel development and debugging.

A developer can compile the kernel, run it under `gdb`, set breakpoints in the scheduler, single-step through the page fault handler, and inspect kernel data structures—all from an ordinary terminal. No reboot, no serial console, no JTAG probe.

Kernel fuzzing.

Coverage-guided fuzzers like `syzkaller` [9] and `kAFL` [10] benefit from fast boot times and deterministic execution. UML boots in under a second, can be snapshot/restored without hardware support, and when combined with LKL-style in-process execution for the fast path, can achieve millions of syscalls per second.

Network simulation.

Projects like `Netkit` [11] and `Marionnet` [12] use UML to run dozens or hundreds of virtual Linux machines, each with its own networking stack, connected by virtual switches and routers. Students and researchers can build and tear down complex network topologies entirely in userspace.

Security research.

UML provides a complete kernel attack surface without requiring trust in hardware or a hypervisor. Researchers can study kernel exploits, test sandboxing mechanisms, and develop mitigations in a fully controlled environment.

CI/CD and automated testing.

Kernel selftests, `kselftest` suites, and distribution build processes benefit from a lightweight kernel environment that can be instantiated in seconds inside a container or a GitHub Actions runner.

Education.

UML turns the kernel into something a student can compile, run, modify, and observe without risk to the host system—lowering the barrier to kernel development from “dedicated test machine” to “laptop terminal.”

1.3 The Current Moment

This report is written at a moment of unusual convergence for UML. Five developments, none individually decisive but collectively transformative, have created a window of opportunity for the most significant redesign of UML since its creation.

1. The `seccomp` backend reaches mainline. The `seccomp` backend, which replaces the decades-old `ptrace` mechanism with `SECCOMP_IOCTL_NOTIF_SEND`-based syscall interception, was merged for Linux 6.16. This single change reduced guest syscall overhead from approximately 3 μ s (`ptrace` round-trip) to roughly 300 ns, making UML competitive with lightweight hypervisors for I/O-intensive workloads.

2. SMP support lands. Historically, UML was a uniprocessor architecture. The addition of SMP support—mapping UML virtual CPUs to host threads with proper inter-processor interrupt (IPI) emulation and TLB synchronization—allows UML to exercise the kernel’s concurrency paths, which is precisely where the most dangerous bugs hide.

3. gVisor’s systrap validates swappable backends. Google’s gVisor project demonstrated with its systrap platform [13] that a syscall-interposition system can support multiple backend mechanisms behind a common interface. The gVisor Sentry originally used `ptrace`, then KVM, then systrap—each time improving performance while maintaining the same guest-facing semantics. This architectural precedent directly informed the backend-ops abstraction in the UML v2 redesign.

4. LKL shows what speed is possible. The Linux Kernel Library [6, 14] compiles Linux as a single-threaded library with no MMU or process separation. It achieves extraordinary fuzzing throughput—millions of operations per second—at the cost of fidelity: there is no guest userspace, no scheduling, no realistic attack surface. LKL represents one extreme of the speed–fidelity tradeoff. The UML v2 redesign aims to approach LKL’s speed while retaining UML’s complete kernel semantics.

5. The kernel community accepts AI-assisted development. The addition of the `coding-assistants.html` document to the kernel’s `Documentation/process/` directory [15] signals an institutional willingness to engage with AI-assisted patch development. The v2 redesign demonstrates what becomes possible when AI tools are used not to generate mechanical patches, but to navigate architectural decisions across a codebase of 30,000,000+ lines of code.

1.4 Report Structure

This report is organized as follows:

Chapter 1: Introduction

(This chapter.) Motivation, context, and scope.

Chapter 2: History

The evolution of UML from Dike’s original `ptrace`-based prototype through the TT/SKAS transition, the years of maintenance mode, and the recent resurgence of interest.

Chapter 3: Foundations

Background material on virtualization theory (Popek–Goldberg), Linux process primitives (`ptrace`, `seccomp`, `clone`, `mmap`), and the KVM API—the building blocks from which UML is constructed.

Chapter 4: UML Architecture

The high-level architecture of UML: how it maps kernel abstractions (virtual memory, interrupts, timers, I/O) onto host-level mechanisms. The backend-ops abstraction layer and its role in enabling pluggable backends.

Chapter 5: UML Implementation

A detailed walk through the UML source tree, covering boot sequence, system call handling, memory management, signal delivery, SMP, and device I/O.

Chapter 6: The `kvm-v2` Backend

Design and implementation of the new KVM-based backend. How `KVM_CREATE_VM`, `KVM_CREATE_VCPU`,

and `KVM_RUN` are used to achieve approximately 100 ns syscall latency for guest-to-kernel transitions.

Chapter 7: Applications

Detailed case studies: kernel debugging under `gdb`, `syzkaller` integration, network simulation, CI/CD pipelines, and educational use.

Chapter 8: Related Work

Comparison with KVM/QEMU, Xen, Firecracker, `gVisor`, `LKL`, `Dune` [16], containers, and unikernels [17].

Chapter 9: The v2 Redesign

The complete contribution of this work: backend-ops abstraction, `kvm-v2` backend, SMP, snapshot/restore, record/replay, comprehensive selftests, and the Rust `umlctl` launcher tool. Design rationale, trade-offs, and lessons learned.

Chapter 10: Conclusion

Summary of contributions, open problems, and future directions.

Appendix A: Source Map

Annotated directory listing of the UML source tree with the v2 additions.

Appendix B: Benchmarks

Raw benchmark data and methodology for the performance measurements cited throughout the report.

1.5 Contributions of This Work

The v2 redesign of User-Mode Linux, developed by Michael Bommarito with AI-assisted tooling, contributes the following:

1. **Backend operations abstraction.** A `struct um_backend_ops` interface (defined in `arch/um/include/shared`) that decouples the UML kernel from any single syscall-interposition mechanism. The abstraction defines a contract for backend lifecycle management, guest process creation, syscall interception, context switching, and memory mapping. New backends can be added by implementing this interface without modifying core UML code.
2. **The `kvm-v2` backend.** A new backend that uses the KVM API to run UML guest code in hardware-accelerated virtual CPUs. When the guest executes a system call, the resulting VM exit is handled directly by the UML kernel—no `ptrace` round-trip, no `seccomp` notification. Measured syscall latency is approximately 100 ns, a 3× improvement over `seccomp` and a 30× improvement over `ptrace`.
3. **SMP support.** Multi-processor support for UML, mapping virtual CPUs to host threads with proper IPI emulation, TLB shutdown, and memory barrier semantics. This enables UML to exercise—and expose bugs in—the kernel’s concurrent code paths.
4. **Snapshot and restore.** Mechanisms for capturing and restoring the complete state of a running UML instance, enabling fast restart for fuzzing workloads and reproducible bug investigation.
5. **Record and replay.** Deterministic record/replay infrastructure that captures non-deterministic events (timer interrupts, I/O completions) during execution and replays them faithfully, enabling time-travel debugging of kernel code.

6. **Comprehensive selftests.** A test suite under `tools/testing/selftests/um/` covering back-end lifecycle, syscall interception, SMP behavior, memory management, and performance regression detection.
7. **The `umlctl` launcher.** A Rust-based command-line tool for building, configuring, launching, and managing UML instances. It handles kernel compilation, root filesystem preparation, backend selection, and instance lifecycle, reducing the barrier to entry from a page of shell commands to a single invocation.

These contributions are organized as seven series-level commits on the `mjbommar/linux` tree (branch `next`), based on `torvalds/master` at `v7.1-rc5`. Each series is independently reviewable and bisectable.

The remainder of this report proceeds from history and theory (Chapters 2–3), through architecture and implementation (Chapters 4–6), to applications and evaluation (Chapters 7–8), and concludes with the complete v2 redesign narrative (Chapter 9) and future directions (Chapter 10).

Chapter 2

History of User-Mode Linux

“I’m doing a (free) operating system (just a hobby, won’t be big and professional like gnu) for 386(486) AT clones.”

—Linus Torvalds, 1991 [18]

User-Mode Linux did not emerge in a vacuum. Its existence is a consequence of design decisions made years before its conception—most notably, the Linux kernel’s clean separation of architecture-dependent and architecture-independent code. This chapter traces the full arc: from the early Linux kernel and the debugging problems that motivated UML, through its creation and golden era, into a decade of decline, and finally to the revival that produced the modern multi-backend architecture described in this report.

2.1 The Linux Kernel Before UML (1991–1999)

On October 5, 1991, Linus Torvalds posted the famous announcement to `comp.os.minix` describing his “free minix-like kernel sources for 386-AT” [18]. Linux 0.01 was a monolithic kernel that ran exclusively on the Intel 80386 processor. It supported a single address space layout, a single disk controller, and a handful of system calls. But it compiled, it booted, and it was free.

Over the next eight years, Linux grew at an extraordinary rate. The 1.0 release (March 1994) was the first declared stable; 2.0 (June 1996) added symmetric multiprocessing (SMP) support and multiple architecture ports—Alpha, SPARC, MIPS, m68k, PowerPC. The 2.2 release (January 1999) brought improved SMP scalability, better networking, and the beginnings of enterprise-grade reliability. By the end of the decade, Linux was running on everything from embedded routers to IBM mainframes, and kernel development involved hundreds of contributors worldwide.

2.1.1 The `arch/` Abstraction

The most architecturally consequential decision of this era was the directory structure that emerged to support multiple hardware platforms. Rather than scattering `#ifdef` conditionals throughout the code, Linux adopted a clean split: the `arch/` directory contained all architecture-specific code—trap handling, context switching, page table manipulation, signal delivery, system call entry—while the rest of the tree (`kernel/`, `mm/`, `fs/`, `net/`, `drivers/`) remained architecture-independent.

Each architecture port implemented a well-defined interface: functions like `switch_to()`, `copy_thread()`, `start_thread()`, and page table accessors (`pgd_val()`, `pte_val()`, etc.). The interface was never formally specified—it evolved organically as new architectures were added—but it was remarkably clean. A new port meant implementing perhaps 50–100 key functions and data structures, after which the entire kernel (scheduler, VFS, networking stack, memory manager) would work without modification.

This abstraction was conceived to support hardware architectures: x86, Alpha, SPARC, ARM. No one anticipated that someone would write an “architecture” where the hardware was

another copy of Linux itself.

2.1.2 The Debugging Problem

Kernel debugging in the 1990s was, to put it charitably, primitive. The primary tools were:

printk() The kernel's `printf` equivalent. Developers added `printk()` calls, rebuilt the kernel, rebooted, reproduced the bug, read the console output. This cycle could take minutes to hours depending on the bug.

Serial console A step up: connect two machines with a serial cable, use one to observe the other's console output. Essential for debugging panics where the screen was already dead.

kgdb An external patch (never merged into mainline during this era) that allowed GDB to connect to a running kernel via serial line. It worked, but it required a second machine, serial hardware, and careful setup. Breakpoints in interrupt handlers were hazardous.

kdb SGI's kernel debugger, also out-of-tree. More capable than `kgdb` for some workflows, but equally dependent on specialized hardware setups.

Crash analysis When the kernel panicked, you got a register dump and a call trace. Interpreting these required intimate knowledge of the architecture's calling conventions and the ability to cross-reference addresses against `System.map`.

What you could *not* do was the thing that every application programmer took for granted: run the code under a debugger. You could not set a breakpoint in the scheduler and single-step through a context switch. You could not attach `valgrind` to the memory manager. You could not run the kernel under `strace` to see what system calls it made. The kernel was the bottom of the stack—there was nothing underneath it to provide these services.

This asymmetry between kernel and userspace development productivity was a constant source of frustration. It also meant that certain classes of bugs—race conditions, memory corruption, subtle locking errors—were disproportionately expensive to diagnose and fix. The kernel needed a way to be a *process*.

2.2 Jeff Dike and the Creation of UML (1999–2001)

Jeff Dike was a kernel developer at VA Linux Systems in the late 1990s. VA Linux was one of the first companies to build a business around Linux, and its engineers were deeply involved in kernel development. Dike had been thinking about the debugging problem and, more broadly, about how to make the kernel more accessible for development and experimentation.

His key insight was deceptively simple: *the Linux kernel's `arch/` abstraction is clean enough that you can write an “architecture” where the “hardware” is the host kernel's system call interface.*

Consider what an architecture port must provide: a way to create and switch between address spaces, a way to handle interrupts and traps, a way to start and stop the CPU, a way to manage memory pages. All of these have direct analogs in the POSIX API:

- Address spaces → `mmap()`, `mprotect()`, `munmap()`
- Interrupts → POSIX signals (SIGALRM for the timer tick, SIGIO for I/O)
- Context switching → `setjmp()/longjmp()` or signal stack manipulation
- Trap handling → `ptrace()` to intercept guest system calls
- Physical memory → a large `mmap()`'d file (the “physical memory” file)
- Block devices → host files accessed via `read()/write()`
- Network devices → host TAP/TUN interfaces or Unix domain sockets

The resulting system would not be an emulator. It would not translate instructions or simulate hardware. It *would be* the Linux kernel—the same source code, compiled for a different “architecture.” The kernel’s scheduler would schedule processes; its VFS would manage file systems; its TCP/IP stack would route packets. The only difference was that instead of running on bare metal (or in a hypervisor), it ran as a collection of ordinary Linux processes.

2.2.1 The Initial Announcement

Dike announced User-Mode Linux on the Linux Kernel Mailing List (LKML) in 1999. The initial response was a mixture of fascination and skepticism. Some developers immediately grasped the implications for debugging and testing. Others questioned whether the performance overhead would make it practical. A few doubted whether the `arch/` interface was really clean enough to support such an unconventional “architecture.”

Dike persisted. By 2000, he had a working implementation that could boot a full Linux system, run a shell, and execute userspace programs. He presented the work at the 4th Annual Linux Showcase and Conference (ALS) in Atlanta [1].

2.2.2 The Tracing Thread (TT) Mode

The first UML execution engine used a model called “tracing thread” (tt) mode. The architecture was as follows:

1. The UML kernel ran as a single host process (the “tracing thread”).
2. Each guest userspace process was a separate host process.
3. The tracing thread used `ptrace()` to attach to every guest process as a debugger.
4. When a guest process made a system call, the host kernel delivered a `SIGTRAP` to the tracing thread (via the `ptrace` notification mechanism).
5. The tracing thread intercepted the system call, extracted its arguments from the guest’s registers, dispatched it to the UML kernel’s system call handler, wrote the result back into the guest’s registers, and resumed the guest.

This worked, and it was conceptually elegant: `ptrace` was designed exactly for this kind of process inspection and control. But it had significant drawbacks:

Host visibility Every guest process appeared as a real process on the host. If the guest ran 500 processes, the host’s process table showed 500+ entries. This was confusing for administrators and made it impossible to cleanly distinguish guest and host processes.

Security Because guest processes were real host processes, they had real PIDs, real signal delivery, and real access to `/proc/self/`. A sufficiently determined attacker inside the guest could potentially escape—for example, by exploiting `ptrace` race conditions or by directly invoking host system calls that the tracing thread failed to intercept.

Performance Every guest system call required *at minimum* four context switches: guest → host kernel (for the syscall trap) → tracing thread (`SIGTRAP` delivery) → host kernel (to resume guest) → guest. In practice, the overhead was worse because the tracing thread also needed to read and write guest registers via `PTRACE_PEEKUSR` and `PTRACE_POKEUSR`. Each such operation was itself a system call.

Signal handling Delivering a signal to a guest process required elaborate coordination between the tracing thread, the host kernel’s signal machinery, and the guest’s signal handling code. Getting this right was one of the most complex parts of the implementation.

Despite these limitations, tt mode was a remarkable achievement. For the first time, a developer could run the Linux kernel under GDB, set breakpoints in the scheduler, single-step through context switches, and use `valgrind` to detect memory errors. The productivity impact was immediate and substantial [19].

2.2.3 Key Insight: UML Is Not an Emulator

It is worth emphasizing a point that was frequently misunderstood (and still is): UML is not an emulator. Emulators like Bochs or (later) QEMU translate machine instructions from one ISA to another, or simulate hardware devices. UML does neither. The guest kernel code is *compiled natively* for the host architecture. When a function like `schedule()` runs inside UML, it is executing the same compiled x86-64 instructions that it would on bare metal. The architecture layer merely provides a different implementation of the hardware abstraction—one that talks to the host kernel instead of to physical hardware.

This distinction has profound implications for fidelity. Code that works in UML will work on bare metal (modulo hardware-specific drivers), and bugs found in UML are real kernel bugs. UML is the kernel, running in a different environment.

2.3 The SKAS Evolution (2002–2006)

The limitations of tt mode—particularly the security and host-visibility problems—motivated a fundamental redesign of the execution engine. The result was SKAS: **S**eparate **K**ernel **A**ddress **S**pace mode.

2.3.1 The Problem with Shared Address Spaces

In tt mode, the UML kernel code ran in the same address space as the guest process it was currently serving. This meant that a guest process could, in principle, read or corrupt kernel data structures. The tracing thread did its best to prevent this, but the fundamental architecture made strong isolation impossible.

The goal of SKAS was to give the UML kernel its own address space, separate from all guest processes. When a guest process made a system call, execution would switch to the kernel address space, handle the call, and switch back—just as a real kernel switches between kernel and user mode via privilege ring transitions.

2.3.2 SKAS0: No Host Kernel Patches Required

The first SKAS implementation, known as SKAS0, achieved address space separation using only standard host kernel facilities. The technique was ingenious:

1. The UML kernel ran in a dedicated host process with its own address space.
2. Guest processes each had their own host process (as in tt mode), but the tracing thread could now switch the guest process's memory mappings to point at the kernel's address space when needed.
3. Address space switching was accomplished via careful use of `PTRACE_SYSCALL` to intercept `mmap()` calls and remap the process's memory.

SKAS0 was a major improvement over tt mode. Guest processes could no longer directly access kernel memory, and the security model was substantially stronger. The performance cost of the address-space switching was nontrivial, but the security benefits were considered worth it.

2.3.3 SKAS3: The `/proc/mm` Approach

SKAS0's address-space manipulation was clever but slow, requiring multiple system calls per switch. Dike proposed a host kernel extension—a `/proc/mm` interface—that would allow a tracing process to directly manipulate another process's address space with a single system call. This variant was called SKAS3 (the number referring to the host kernel patch revision that introduced the necessary support).

With `/proc/mm`, the tracing thread could:

- Create a new memory management context via `/proc/mm`
- Map and unmap pages in that context with a single `ioctl()`
- Switch a traced process between contexts efficiently

The performance improvement over SKAS0 was significant—roughly 2x for system-call-intensive workloads. However, the approach required running a patched host kernel, which limited adoption. Most users preferred the slightly slower SKAS0 over the burden of maintaining a custom host kernel.

2.3.4 SKAS4 and the OLS Talks

Dike continued refining the approach. SKAS4 proposed further optimizations to the host kernel interface, aiming to minimize the number of context switches required for guest system call handling. He presented this work at the Ottawa Linux Symposium in 2006 [20], building on his earlier OLS 2003 talk [21] that had focused on making UML “ready for the mainstream.”

The SKAS evolution demonstrated a recurring tension in UML’s history: achieving good performance often required host kernel modifications, but requiring host kernel modifications limited adoption. This tension would recur with every subsequent execution engine, including the seccomp backend and the `kvm-v2` backend described in this report.

2.3.5 The Prentice Hall Book

In 2006, Dike published *User Mode Linux* with Prentice Hall [22]. The book provided the first comprehensive treatment of UML’s architecture, covering the `tt` and SKAS execution models, the memory system, device drivers, networking, and debugging techniques. It also included practical guidance on using UML for development, testing, and virtual hosting.

The book remains the most detailed single source on UML’s original architecture, although much of the implementation detail it describes has since been superseded by the seccomp and `kvm-v2` backends.

2.4 Mainline Merge and the Golden Era (2002–2010)

UML was merged into the Linux 2.5 development tree in 2002 and was included in the 2.6.0 release (December 2003). The merge was a significant event: UML was the first virtualization technology to be included in the mainline Linux kernel. Its presence in `arch/um/` gave it legitimacy and ensured that kernel API changes would (in theory) be propagated to the UML port along with every other architecture.

The years from 2002 to roughly 2010 represented UML’s golden era. During this period, it was actively developed, widely used, and considered a key part of the kernel developer’s toolkit.

2.4.1 Kernel Development and Debugging

The primary use case was exactly what Dike had envisioned: making kernel development easier. With UML, a developer could:

- Boot a full Linux system in seconds, without hardware or a heavy VM
- Run the kernel under GDB with full source-level debugging
- Use `valgrind` to detect memory errors in kernel code
- Test file system changes without risking data on real disks
- Reproduce and debug race conditions in the scheduler
- Develop and test networking code with virtual networks
- Run kernel unit tests without dedicated hardware

The fast boot time was particularly valuable. A UML instance could go from cold start to a shell prompt in under two seconds on contemporary hardware, compared to minutes for a full hardware boot. For the edit-compile-test cycle, this speed difference was transformative.

2.4.2 Netkit and Network Simulation

One of UML’s most successful applications was Netkit, a network emulation tool developed by Maurizio Pizzonia and Massimo Rimondini at Roma Tre University [11]. Netkit used UML instances as virtual network nodes, connected by virtual Ethernet links implemented via Unix domain sockets.

Netkit enabled educators and researchers to create complex network topologies on a single machine. A student could configure a multi-router network with BGP, OSPF, DNS, and DHCP—all running real Linux networking code—on a laptop. Netkit was widely adopted in networking courses at universities across Europe and became one of UML’s most visible applications.

The related project Marionnet [12] provided a graphical interface for constructing virtual network topologies, further lowering the barrier to entry for network education and experimentation.

2.4.3 Security Research and Honeypots

UML found a natural audience in the security research community. Its lightweight isolation made it useful for:

Honeypots Running deliberately vulnerable systems to attract and study attackers. UML’s low overhead made it practical to run many instances on a single host, and its isolation (especially in SKAS mode) provided reasonable confidence that a compromised guest could not easily escape to the host.

Malware analysis Executing suspicious binaries in a controlled environment where every system call could be observed.

Intrusion detection research Running experimental IDS systems against realistic kernel and network stacks.

2.4.4 Virtual Hosting

Before containers and lightweight VMs became mainstream, UML was used by some hosting providers to offer virtual private servers (VPS). Each customer received a UML instance with its own root filesystem, network configuration, and process space. The overhead was lower than full hardware emulation (which at the time meant something like VMware or Bochs), and the isolation model was stronger than chroot-based approaches.

Companies like Linode (in its early days, before switching to Xen) and various European hosting providers offered UML-based VPS products. While the performance overhead of ptrace-based system call interception made UML slower than native execution, it was adequate for many web hosting and light-computation workloads.

2.5 Decline and Maintenance Mode (2010–2020)

UML’s decline was not the result of any single event. It was the cumulative effect of multiple technological shifts that eroded its competitive advantages while UML itself stagnated.

2.5.1 Hardware Virtualization Changes the Landscape

The most consequential shift was the arrival of hardware virtualization support in commodity x86 processors. Intel released VT-x (originally codenamed “Vanderpool”) in late 2005 on the Pentium 4 662 and 672 [23]. AMD followed in 2006 with AMD-V (SVM, originally “Pacifica”) [24]. These extensions added a new privilege level—VMX root and non-root modes (Intel) or host and guest modes (AMD)—that allowed a hypervisor to run unmodified guest operating systems with near-native performance.

The impact on the virtualization landscape was seismic. Before VT-x/AMD-V, running an unmodified guest OS on x86 required either full emulation (slow), binary translation (complex, patented by VMware), or paravirtualization (required guest OS modifications, as with Xen). Hardware virtualization made it possible to run unmodified guests with overhead measured in single-digit percentages rather than factors of 2–10x.

For UML, this was an existential threat. UML’s ptrace-based system call interception imposed an overhead of roughly 1–5 μ s per guest system call. For a workload making 100,000 system calls per second, this translated to 0.1–0.5 seconds of overhead per second of execution—a 10–50% performance penalty on a system-call-intensive workload. Hardware virtualization reduced the equivalent overhead to tens of nanoseconds per VM exit, roughly two orders of magnitude faster.

2.5.2 KVM: Virtualization in the Kernel

The decisive blow came in February 2007, when Avi Kivity’s KVM (Kernel-based Virtual Machine) was merged into Linux 2.6.20 [7]. KVM leveraged VT-x/AMD-V to turn the Linux kernel itself into a hypervisor. It was elegant, minimal (the initial patch was under 10,000 lines), and *fast*.

KVM’s architecture was the mirror image of UML’s. Where UML ran a kernel as a userspace process, KVM ran guest kernels in hardware-supported VM mode, with the host kernel as the hypervisor. Where UML used ptrace to intercept system calls, KVM used VT-x/AMD-V to trap only privileged operations. Where UML shared the host’s physical memory via `mmap()`, KVM used hardware page tables (EPT/NPT) to provide isolated guest physical address spaces.

Combined with QEMU [25] for device emulation, KVM quickly became the default virtualization solution for Linux. It offered near-native performance for CPU-bound workloads and excellent I/O performance via virtio paravirtualized drivers. For the use cases where UML had been the best available option—kernel development, testing, lightweight VMs—KVM/QEMU was now competitive and often superior.

2.5.3 Containers Capture the Lightweight Market

The other major technological shift was the rise of OS-level virtualization (containers). The Linux kernel’s cgroup and namespace infrastructure matured between 2006 and 2013 [26, 27], providing the building blocks for lightweight isolation: PID namespaces (2008), network namespaces (2009), user namespaces (2012), and cgroup v1 for resource accounting and limits.

Docker, released in 2013, packaged these primitives into an accessible developer tool and ignited the container revolution. For the use case of “run an isolated Linux environment with low overhead,” containers offered effectively zero additional syscall overhead (the guest ran natively, sharing the host kernel) and millisecond startup times.

Containers did not provide the kernel-level isolation that UML offered—a container shared the host kernel, so kernel bugs affected all containers—but for the vast majority of development and deployment workloads, the trade-off was acceptable. The combination of near-zero overhead and excellent tooling made containers the dominant lightweight isolation mechanism.

2.5.4 Code Rot and Declining Maintenance

As users migrated to KVM and containers, developer attention followed. UML’s maintenance situation deteriorated:

- The `tt` mode was eventually removed, simplifying the codebase but also removing a working (if slow) execution engine.
- SMP support was never completed; UML remained UP-only for over a decade.
- Various kernel API changes were not properly propagated to `arch/um/`, causing build breakages that sometimes persisted for entire release cycles.
- The SKAS0 mode accumulated technical debt, with workarounds and hacks layered on top of the original ptrace-based design.
- Test coverage was minimal—there were no automated selftests, and manual testing was sporadic at best.
- Documentation drifted out of date with the actual implementation.

Richard Weinberger took over as nominal maintainer, and his efforts kept UML from being removed entirely. But the subsystem was in maintenance mode: patches were mostly limited to fixing build breakages caused by core kernel API changes, with little new feature development.

The perception within the kernel community reflected this state. UML was widely viewed as a legacy subsystem—useful in its time, but superseded by better alternatives. Proposals to remove it from the tree surfaced periodically, though they never gained enough traction to reach consensus.

2.6 The Linux Virtualization Timeline

To understand UML’s trajectory, it helps to place it in the broader context of Linux virtualization history. The following milestones shaped the landscape in which UML operated:

2.6.1 Xen and Paravirtualization (2003)

Xen, developed at the University of Cambridge, introduced efficient paravirtualization for x86 [3]. By modifying the guest OS to make “hypercalls” instead of privileged instructions, Xen achieved virtualization overhead as low as 2–5% for many workloads. Xen was widely adopted in cloud computing (notably by Amazon Web Services in its early infrastructure) and demonstrated that high-performance virtualization was possible even without hardware support.

Xen’s success illustrated a design philosophy opposite to UML’s: rather than making the kernel run as a userspace process, Xen made the hardware (or a thin software layer mimicking hardware) available to multiple kernel instances. Both approaches achieved isolation, but Xen’s approach was more natural for production deployment.

2.6.2 Intel VT-x and AMD-V (2005–2006)

Intel’s VT-x [23] and AMD’s SVM [24] extensions added hardware support for virtualization to commodity x86 processors. The key features included:

- A new CPU mode (VMX non-root / guest mode) for running guest code
- Automatic trapping of privileged operations to the hypervisor (VM exits)
- Hardware-managed guest/host state save/restore (VMCS/VMCB)
- Extended Page Tables (EPT, Intel, 2008) and Nested Page Tables (NPT, AMD, 2007) for efficient guest address translation

These extensions eliminated the need for binary translation or paravirtualization, enabling hypervisors to run unmodified guest operating systems with near-native performance. They also provided a hardware-enforced security boundary between guest and host that was far stronger than any software-based approach, including UML’s ptrace-based isolation.

2.6.3 KVM Enters Mainline (2007)

KVM’s merge into Linux 2.6.20 [7] was a watershed moment. It established that the Linux kernel itself would be the primary hypervisor for Linux-based virtualization, rather than an external project like Xen or a proprietary product like VMware. KVM’s tight integration with the Linux kernel meant that it benefited automatically from every kernel improvement: scheduler optimizations improved guest scheduling, memory management improvements improved guest memory performance, and so on.

2.6.4 Cgroups and Namespaces Mature (2008–2013)

The cgroup and namespace infrastructure that would eventually underpin containers was developed incrementally between 2006 and 2013 [27]. Key milestones included:

- 2006: cgroup v1 infrastructure merged
- 2008: PID namespaces
- 2009: Network namespaces
- 2012: User namespaces (allowing unprivileged container creation)
- 2013: Docker 0.1 released, combining cgroups, namespaces, and union filesystems into an accessible tool

Containers [26] did not provide the same isolation guarantees as virtual machines (a kernel exploit in a container compromises the host), but they offered effectively zero overhead and subsecond startup. For development workflows and many production deployments, this trade-off was overwhelmingly attractive.

2.6.5 gVisor: A Different Approach (2018)

Google’s gVisor [5], released in 2018, represented a fascinating convergence with some of UML’s original ideas. gVisor implements a user-space kernel (called Sentry) that intercepts guest system calls and reimplements them in Go. Like UML, gVisor runs guest applications as processes on the host. Unlike UML, gVisor does not use the Linux kernel source code—it is a clean-room reimplementation of the Linux system call interface.

gVisor’s original platform used `ptrace` for system call interception (just as UML’s `tt` mode did), and later moved to a KVM-based platform for better performance. In 2023, gVisor introduced Systrap, a SIGSYS-based platform that uses `seccomp-bpf` for system call interception—conceptually identical to the approach that Benjamin Berg would bring to UML.

The parallel evolution is striking: two projects, decades apart, converging on the same insight that `seccomp-bpf` provides a faster mechanism for system call interception than `ptrace`.

2.6.6 Firecracker: MicroVMs for Serverless (2020)

Amazon Web Services released Firecracker in 2020 [8], a purpose-built virtual machine monitor (VMM) for serverless computing. Firecracker uses KVM as its hypervisor but strips away nearly all of QEMU’s device emulation complexity, providing only a minimal set of paravirtualized devices (`virtio-net`, `virtio-block`, a serial console). The result is a VMM that can boot a guest in approximately 125 ms and consumes less than 5 MB of memory per VM.

Firecracker’s existence demonstrated that the performance gap between VMs and containers could be narrowed dramatically with careful engineering. It also showed that for production workloads requiring strong isolation, the kernel community had converged on KVM as the foundation.

2.7 The Revival (2020–2026)

Against this backdrop, several independent threads of development converged in the early 2020s to revive UML. The motivations were varied, but they shared a common recognition that UML’s unique position—a real kernel running as a userspace process—provided capabilities that no other virtualization technology could match.

2.7.1 Benjamin Berg and the Seccomp Backend

The most consequential development was Benjamin Berg’s work on a seccomp-based execution backend. The idea, building on the seccomp-bpf framework [28] that Will Drewry had added to the kernel in 2012, was to replace ptrace-based system call interception with SIGSYS-based notification.

The mechanism works as follows:

1. A seccomp-bpf filter is installed that matches all system calls from guest processes.
2. When a guest process makes a system call, the kernel generates a SIGSYS signal instead of executing the system call.
3. The signal handler (running in the UML kernel context) examines the system call, dispatches it to the appropriate UML kernel handler, and manipulates the guest’s register state to reflect the result.
4. Execution resumes in the guest process with the system call appearing to have completed normally.

The performance advantage over ptrace was substantial. Ptrace-based interception required a minimum of two context switches per system call (guest → tracer → guest) plus additional system calls for register access. The seccomp approach handled everything within a single signal delivery, which the kernel could process largely in-place. Benchmarks showed roughly 3–4x improvement in system call throughput.

Berg also redesigned the UML process model around futex-based synchronization, replacing the signal-heavy IPC that the old ptrace backend had used. The result was a cleaner, faster, and more maintainable execution engine.

The seccomp backend was merged into Linux 6.16 (released 2025), marking the first major new feature to land in `arch/um/` in over a decade.

2.7.2 Tiwei Bie and SMP Support

Tiwei Bie (Intel) developed a series of patches to bring SMP support to UML. The v4 series implemented multi-CPU scheduling using host threads as virtual CPUs, with appropriate locking and synchronization to ensure correct concurrent execution.

SMP support was a long-standing gap in UML’s capabilities. Modern kernel code is thoroughly concurrent, with per-CPU data, RCU, and lock-free algorithms that behave differently on UP and SMP systems. Running UML in UP mode meant that an entire class of concurrency bugs—the class that is most difficult to diagnose and fix—was invisible.

2.7.3 Anton Ivanov and Virtio-UML

Anton Ivanov contributed `virtio-uml`, a transport layer that allows UML to use virtio devices via `vhost-user` or direct kernel virtio backends. This work was significant because it connected UML to the broader virtio ecosystem: instead of implementing custom device models for each device type (block, network, console), UML could use the same paravirtualized device stack that KVM/QEMU, Firecracker, and other VMMs used.

2.7.4 Hajime Tazaki and the LKL-into-UML RFC

Hajime Tazaki submitted an RFC series (reaching v8) proposing to unify the Linux Kernel Library (LKL) [6, 29] into UML. LKL compiled the Linux kernel as a library that could be linked into userspace applications, providing access to the kernel’s networking stack, file systems, and other subsystems without running a full VM.

The LKL-into-UML RFC stalled due to the complexity of reconciling LKL’s library model with UML’s process model, but it demonstrated continuing interest in the fundamental idea of running kernel code in userspace. It also highlighted the need for a cleaner backend abstraction—a need that the modern `backend_ops` architecture addresses.

2.7.5 The Coding Assistants Policy

In April 2026, the Linux kernel documentation was updated with `coding-assistants.rst` [15], establishing official guidelines for AI-assisted kernel development. While not specific to UML, this policy change facilitated the rapid development of infrastructure code (selftests, documentation, build tooling) that had been a persistent bottleneck.

2.7.6 The v2 Architecture

The convergence of these efforts—`seccomp` backend, SMP, `virtio`, and renewed community interest—created the conditions for a more fundamental redesign. The v2 architecture, detailed in Chapter 9 of this report, introduces a `backend_ops` abstraction layer that cleanly separates the UML kernel from its execution engine. This allows multiple backends (`seccomp`, `kvm-v2`, and potentially others) to coexist, selected at boot time, while sharing the rest of the kernel infrastructure.

The `kvm-v2` backend, described in Chapter 6, leverages the same KVM hardware virtualization that powers production cloud infrastructure, but uses it to accelerate UML rather than to replace it. This closes the performance gap with bare-metal execution while preserving UML’s defining advantage: the kernel runs as a debuggable, instrumentable userspace process.

2.8 Timeline

Figure 2.1 presents a visual timeline of the major events described in this chapter, placing UML’s development in the context of the broader Linux virtualization ecosystem.

2.9 Lessons from History

The arc of UML’s history offers several lessons relevant to the v2 redesign:

Architecture matters more than performance. UML’s original `arch/` insight—that a well-abstracted interface can be reimplemented on top of very different substrates—was correct in 1999, correct in 2007 when applied to KVM, and remains correct today. The v2 `backend_ops` abstraction applies the same principle within UML itself: abstracting the execution engine so that new backends (`seccomp`, KVM, and future possibilities) can be added without rewriting the rest of the architecture.

The `ptrace` tax was real. UML’s decade-long decline was not due to a fundamental flaw in the concept of running a kernel as a userspace process. It was due to a specific implementation bottleneck: `ptrace`-based system call interception was too slow. The `seccomp` backend demonstrates that with a better interception mechanism, the concept remains viable. The `kvm-v2` backend goes further, eliminating the interception overhead entirely for most operations.

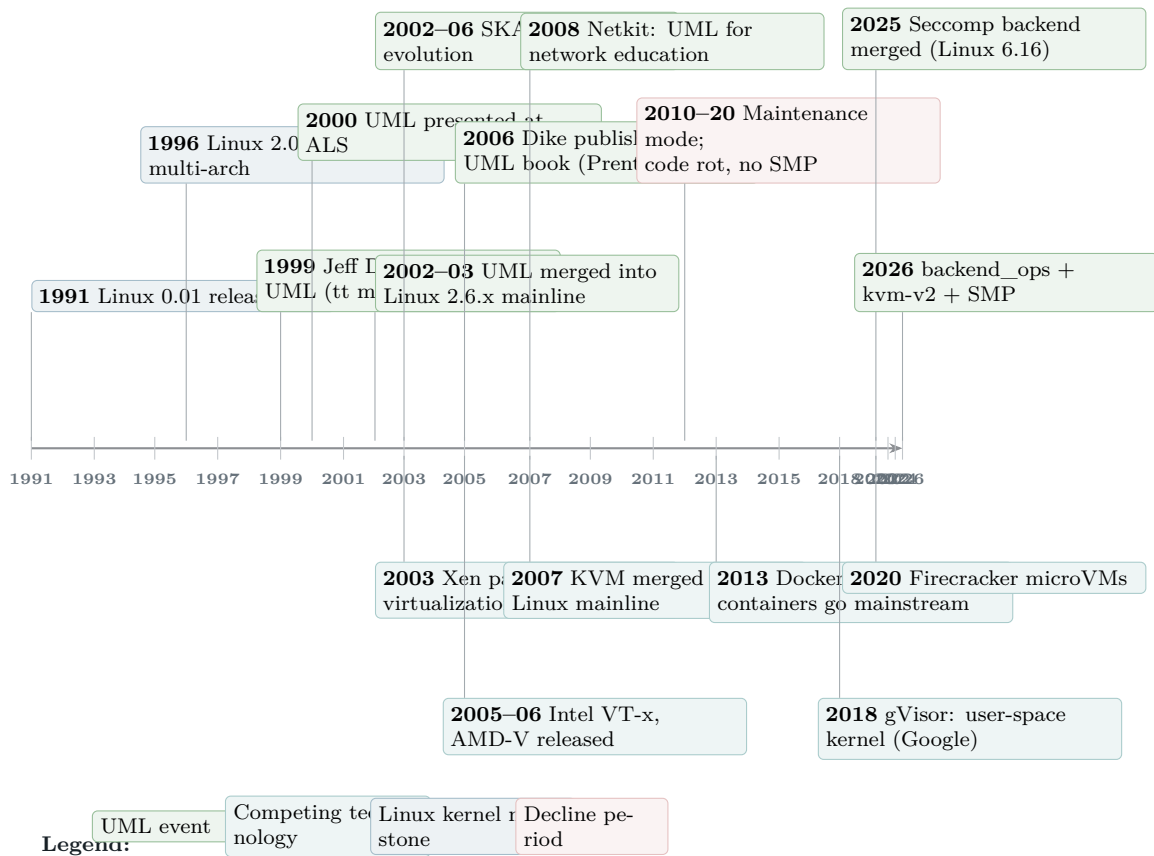


Figure 2.1: Timeline of User-Mode Linux development in the context of Linux kernel and virtualization history (1991–2026). Upper events relate to UML and the Linux kernel; lower events show competing virtualization technologies that shaped UML’s trajectory.

Isolation without hardware is still valuable. Containers have no kernel-level isolation. Hardware VMs provide strong isolation but require VT-x/AMD-V and cannot be run inside many restricted environments (CI/CD containers, nested VMs, some cloud instances). UML occupies a unique middle ground: real kernel isolation, implemented purely in software, runnable anywhere a Linux process can run. This niche has expanded rather than contracted as CI/CD pipelines and restricted environments have become ubiquitous.

Debugging the kernel will always matter. No amount of hardware virtualization performance eliminates the need to debug the kernel. KVM/QEMU provides a debuggable target via GDB's remote protocol, but the developer experience is qualitatively different from running the kernel as a native process. UML allows kernel code to be debugged with the same tools, workflows, and muscle memory that developers use for userspace applications. This advantage is permanent.

Maintenance requires a constituency. UML's decade of rot was a maintenance problem, not a technical one. The technology had no organized user community demanding features, no companies depending on it for production workloads, and no continuous integration preventing regressions. The v2 effort addresses this by integrating kselftests, providing comprehensive documentation, and connecting UML to use cases (kernel fuzzing, CI/CD, AI-assisted development) that create ongoing demand for maintenance.

With this historical context established, we turn in the next chapter to the technical foundations on which UML is built: the Linux process model, the ptrace and seccomp interfaces, and the KVM API.

Chapter 3

Technical Foundations

Any sufficiently complex virtual machine architecture is indistinguishable from a real one.

—with apologies to Arthur C. Clarke

This chapter establishes the technical concepts required to understand how a kernel can run as an ordinary userspace process. We begin with the classical virtualization taxonomy and show where user-mode kernels fit—and where they do not. We then trace the x86 privilege model from its original ring-based design through the hardware virtualization extensions that now underpin KVM. Finally, we examine the Linux process-model primitives that UML exploits—`ptrace()`, `seccomp`, `mmap()`, and the KVM `ioctl` interface—and compare the three trap mechanisms that have defined UML’s performance envelope over the past quarter century.

3.1 Virtualization Taxonomy

3.1.1 The Popek–Goldberg Requirements

The formal study of virtual machines begins with Goldberg’s 1973 taxonomy of *virtual machine monitors* (VMMs) [30] and the celebrated Popek–Goldberg theorem of 1974 [2]. Their framework identifies three properties a VMM must satisfy:

- P1. Equivalence (Fidelity).** A program running under the VMM exhibits behavior identical to that observed when running directly on the hardware, with at most timing differences.
- P2. Resource Control (Safety).** The VMM retains complete control of the virtualized resources. No guest operation may access a resource not explicitly allocated to that guest.
- P3. Efficiency (Performance).** A statistically dominant fraction of guest instructions execute directly on the hardware without VMM intervention.

The theorem proves that a VMM satisfying all three properties can be constructed for any *third-generation architecture*¹ if and only if the set of *sensitive instructions*² is a subset of the set of *privileged instructions* (those that trap when executed in user mode). When every sensitive instruction is privileged, the VMM can intercept all guest attempts to observe or modify machine state by running the guest at a lower privilege level and catching the resulting traps.

3.1.2 Taxonomy of Hypervisors

Smith and Nair [31] refine the taxonomy into two orthogonal dimensions:

¹Popek and Goldberg use “third generation” to mean a conventional von Neumann machine with a supervisor/user mode split, exemplified by IBM System/370.

²Instructions whose behavior depends on the current privilege level or that affect the machine’s resource-allocation state.

Type 1 (bare-metal) vs. Type 2 (hosted) hypervisors.

A Type 1 hypervisor runs directly on the hardware with no intervening operating system (e.g., VMware ESXi, Xen). A Type 2 hypervisor runs as a process atop a host OS (e.g., VMware Workstation, VirtualBox). KVM blurs the boundary: it is a loadable module that turns the Linux kernel itself into a Type 1 hypervisor while simultaneously serving as a host OS.

Full virtualization vs. paravirtualization vs. OS-level.

Full virtualization presents an unmodified hardware interface to the guest; the guest kernel need not know it is virtualized. Paravirtualization (exemplified by Xen [3]) modifies the guest kernel to replace sensitive instructions with explicit *hypercalls*. OS-level virtualization (containers) shares a single kernel instance and isolates guests via namespaces and cgroups [26].

3.1.3 Where User-Mode Kernels Fit

UML does not fit cleanly into any of these categories, and this taxonomic awkwardness is part of what makes it interesting.

- It is not a hypervisor in the Popek–Goldberg sense, because it does not satisfy the Efficiency property: in the original `ptrace()`-based design, *every* guest system call requires VMM intervention.
- It is not OS-level virtualization, because it runs an independent kernel instance with its own scheduler, memory management, and driver stack.
- It is not paravirtualization, because the guest userspace is unmodified—ordinary Linux binaries run without recompilation.
- With the KVM backend (Chapter 6), UML gains the Efficiency property: a dominant fraction of guest instructions execute directly on the CPU inside a KVM virtual machine, and the LSTAR gadget handles trivial system calls entirely in-guest without any VMM exit.

Perhaps the most accurate characterization is that UML is a *process-level system virtual machine* [31]: it implements a complete system interface (an entire Linux kernel) within the confines of a host process. The KVM backend promotes it to a *hybrid* system VM that uses hardware virtualization for the hot path while retaining the hosted-process model for I/O, lifecycle management, and debugging.

3.2 The x86 Privilege Model**3.2.1 Rings, CPL, and DPL**

The x86 architecture defines four privilege levels, conventionally called *rings* 0 through 3. The **Current Privilege Level** (CPL) is the ring at which the processor is currently executing; it is encoded in the two least-significant bits of the CS and SS segment selectors. The **Descriptor Privilege Level** (DPL) field in segment descriptors, gate descriptors, and page-table entries gates access: a code segment can only be called from a CPL that is at least as privileged as the target DPL.

Traditional operating systems use a two-ring model: the kernel at Ring 0, user processes at Ring 3. Rings 1 and 2 exist in hardware but are almost universally unused.³

³OS/2 and a few microkernel-inspired designs briefly used Ring 1 or Ring 2 for device drivers, but the x86-64 long mode effectively reduces the model to two rings in practice.

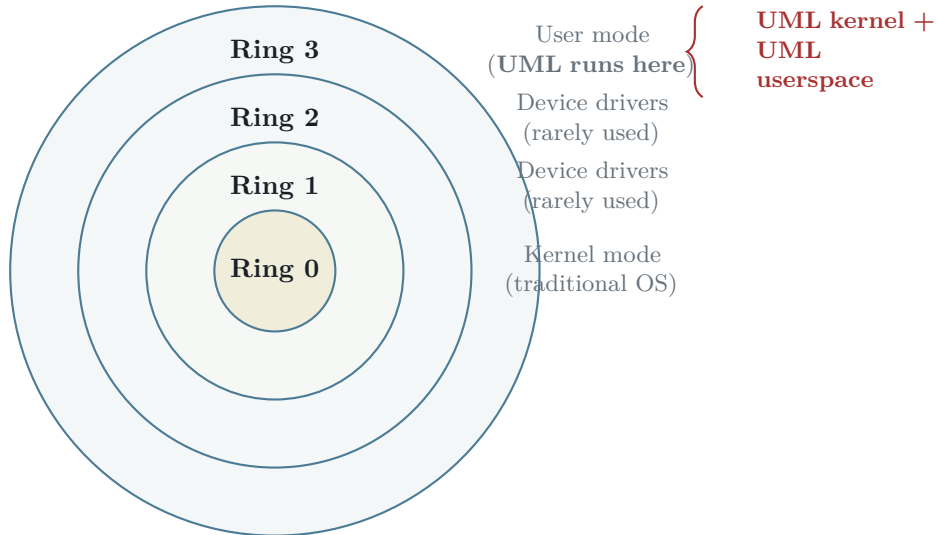


Figure 3.1: The x86 privilege ring model. Traditional kernels occupy Ring 0; UML runs entirely at Ring 3 (CPL=3), sharing the same privilege level as ordinary user processes.

3.2.2 Implications for UML

UML runs entirely at Ring 3. This has several consequences:

1. **No privileged instructions.** The UML kernel cannot execute `cli/sti` (interrupt control), `mov cr3` (page-table switching), `lgdt/lidt` (descriptor-table loading), or any other privileged instruction. Every such operation must be emulated via host system calls.
2. **No direct hardware access.** All I/O must pass through the host kernel's system-call interface. UML's block driver issues `read()/write()` on a host file descriptor; its network driver uses host sockets or TUN/TAP devices.
3. **Shared address space with the host.** The UML process occupies the same 48-bit (or 57-bit) virtual address space managed by the host kernel. The host kernel's own mappings (above the canonical hole) are inaccessible from CPL=3, but the host's ASLR layout, vDSO, and stack placement all constrain where UML can place its own data structures.
4. **System call interception required.** When guest userspace issues a `syscall` instruction, it would normally trap into the *host* kernel, not the UML kernel. UML must intercept this trap and redirect it to its own system-call handler. The mechanism for this interception—`ptrace()`, `seccomp`, or KVM—is the defining architectural choice of each UML backend.

3.3 Hardware Virtualization Extensions

The x86 architecture famously violates the Popek–Goldberg requirements: several sensitive instructions (notably `popf`, `sgdt`, and `pushf`) are not privileged and therefore do not trap when executed in user mode [32]. This made software-only full virtualization on x86 difficult, requiring either binary translation (VMware) or paravirtualization (Xen).

Hardware virtualization extensions—Intel VT-x (2005) and AMD-V (2006)—solve this by adding a new, orthogonal dimension of privilege.

3.3.1 Intel VT-x (VMX)

VT-x [33] introduces **VMX root mode** (the hypervisor) and **VMX non-root mode** (the guest). The transition between the two is mediated by a per-vCPU data structure called the

Virtual Machine Control Structure (VMCS):

VM Entry (VMLAUNCH / VMRESUME).

The hypervisor loads guest state from the VMCS and transfers control to the guest. The guest runs at whatever CPL the VMCS guest-state area specifies—typically Ring 0 for a full-system guest, or Ring 3 for a process-level guest like UML’s KVM backend.

VM Exit.

Certain events cause the processor to stop guest execution, save guest state into the VMCS, and transfer control back to VMX root mode. The exit reason is recorded in the VMCS. Events that cause exits include: execution of privileged instructions, I/O instructions (if the I/O bitmap so indicates), external interrupts, page faults (if configured), and the `CPUID` instruction.

Extended Page Tables (EPT).

EPT provides a second level of address translation: guest-physical addresses (GPAs) are translated to host-physical addresses (HPAs) by hardware, without any hypervisor intervention for non-faulting accesses. This eliminates the “shadow page table” overhead that made pre-EPT virtualization expensive.

3.3.2 AMD-V (SVM)

AMD’s Secure Virtual Machine [34] follows the same principles with different terminology: the per-vCPU structure is the **Virtual Machine Control Block** (VMCB); the hardware nested paging feature is called **Nested Page Tables** (NPT); and the entry/exit instructions are `VMRUN` and—implicitly—any intercepted event. The functional model is equivalent for the purposes of this report.

3.3.3 KVM and the `ioctl` Interface

KVM [7] exposes hardware virtualization to userspace through a layered `ioctl` interface on `/dev/kvm`:

1. `KVM_CREATE_VM` returns a VM file descriptor.
2. `KVM_SET_USER_MEMORY_REGION` on the VM fd maps host-virtual memory into the guest-physical address space (backing EPT/NPT).
3. `KVM_CREATE_VCPU` on the VM fd returns a vCPU file descriptor, with a shared `struct kvm_run` page accessible via `mmap()` on the vCPU fd.
4. `KVM_RUN` on the vCPU fd enters VMX/SVM non-root mode. When the guest causes an exit, `KVM_RUN` returns and the exit reason + auxiliary data are available in the shared `kvm_run` page.

3.3.4 Why This Matters for UML

UML’s KVM backend uses this interface to run guest code at Ring 3 *inside a virtual machine*, while the UML kernel process remains an ordinary userspace process on the host. The critical insight is that KVM’s VM entry/exit cycle is far cheaper than a `ptrace()` round-trip or even a `seccomp` SIGSYS delivery. A `KVM_RUN` → `KVM_EXIT_IO` round-trip costs approximately 100 ns to 150 ns; with the LSTAR gadget (Section 3.6), trivial system calls complete in approximately 90 cycles without exiting the VM at all.

3.4 The Linux Process Model

From the host kernel’s perspective, a running UML instance is an ordinary multi-threaded process (or set of processes, depending on the backend). UML exploits a specific set of Linux system calls to implement its virtualization layer.

3.4.1 ptrace: The Original Trap Mechanism

The `ptrace()` system call [35] allows one process (the *tracer*) to observe and control the execution of another (the *tracee*). UML’s original backend uses:

`PTRACE_SYSEMU`.

Resumes the tracee but intercepts the next system call *without executing it*. The tracee stops with a `SIGTRAP|0x80` signal. This is the key extension that Jeff Dike contributed to the Linux kernel specifically for UML: without it, the host kernel would execute the guest’s system call before UML could intercept it.

`PTRACE_GETREGS` / `PTRACE_SETREGS`.

Read and write the tracee’s general-purpose registers. UML reads the registers to determine which system call the guest requested (from `RAX`) and its arguments (from `RDI`, `RSI`, `RDX`, `R10`, `R8`, `R9` per the x86-64 ABI), then writes the return value back into `RAX`.

3.4.2 seccomp: The Modern Trap Mechanism

The `seccomp` BPF filter [28] was originally designed for sandboxing (Chrome, Firefox, container runtimes), but its `SECCOMP_RET_TRAP` return action proved ideal for UML. When a `seccomp` filter returns `SECCOMP_RET_TRAP`, the kernel delivers a `SIGSYS` signal to the thread that issued the system call—without executing it. This is functionally equivalent to `PTRACE_SYSEMU` but with dramatically lower overhead because the signal is delivered in-process rather than requiring a context switch to an external tracer.

3.4.3 futex: Stub Synchronization

The `futex()` system call provides efficient userspace-to-kernel synchronization [36]. In the `seccomp` backend, the stub process and the UML kernel use a shared `futex` word (in the `stub_data` page) to implement a lightweight turnstile: the stub writes its trap state, issues `FUTEX_WAKE`, and blocks on `FUTEX_WAIT`; the UML kernel wakes, processes the trap, writes the result, and wakes the stub.

3.4.4 mmap / munmap / mprotect: Guest Memory

UML implements guest “physical” memory as a host-side anonymous mapping or a file-backed mapping (typically on `tmpfs`). Guest page-table changes are realized by calling `mmap()`, `munmap()`, and `mprotect()` on the host to adjust the stub process’s address space. Each guest TLB flush translates to a batch of these host system calls.

3.4.5 clone / fork: Guest Processes

Each guest process in UML is backed by a host process (or thread, depending on the configuration). The `clone()` system call creates the host-side entity; `CLONE_VM` may or may not be used depending on whether the backend shares address spaces.

3.4.6 KVM ioctl Interface

The KVM backend [37] replaces the `ptrace()` and `seccomp` trap mechanisms with hardware-assisted virtualization. The key ioctls are:

```
1  /* VM creation (once per UML instance) */
2  vm_fd = ioctl(kvm_fd, KVM_CREATE_VM, 0);
3
4  /* Map guest physical memory (host tmpfs -> guest GPA) */
5  struct kvm_userspace_memory_region mem = {
6      .slot          = 0,
7      .guest_phys_addr = 0,
8      .memory_size    = physmem_size,
9      .userspace_addr  = (u64)uml_physmem,
10 };
11 ioctl(vm_fd, KVM_SET_USER_MEMORY_REGION, &mem);
12
13 /* Per-vCPU creation (one per host CPU in the pool) */
14 vcpu_fd = ioctl(vm_fd, KVM_CREATE_VCPU, cpu_id);
15 run = mmap(NULL, sizeof(struct kvm_run), PROT_RW,
16            MAP_SHARED, vcpu_fd, 0);
17
18 /* Hot loop: enter guest, handle exit, re-enter */
19 while (1) {
20     ioctl(vcpu_fd, KVM_RUN, 0);
21     switch (run->exit_reason) {
22     case KVM_EXIT_IO:
23         handle_io_trap(regs, run, vcpu);
24         break;
25     /* ... other exit reasons ... */
26     }
27 }
```

Listing 3.1: KVM ioctl sequence for UML’s v2 backend.

3.5 Address Space Layout

UML must carve a complete kernel and user address space out of the host process’s available virtual address range. The layout differs between the `seccomp` and KVM backends.

3.5.1 Seccomp Backend Layout

The `seccomp` backend’s stub process has the following regions in its virtual address space (Figure 3.2):

1. **Guest userspace** (0x0 to approximately `TASK_SIZE`). Mapped by the UML kernel via batched `mmap()/mprotect()` calls into the stub process.
2. **Stub pages** (`STUB_START` to `STUB_END`). Three pages: one page of stub executable code (the `SIGSYS` handler, the `futex()` wait/wake loop, the syscall-batch executor) and two pages of `stub_data` (shared state: registers, signal information, the `futex` word, and the syscall-batch buffer).
3. **UML physical memory** (`uml_physmem`). The contiguous region backing guest RAM, typically `mmap()`’d from a `tmpfs` file descriptor or as anonymous memory. Guest page tables reference offsets within this region.

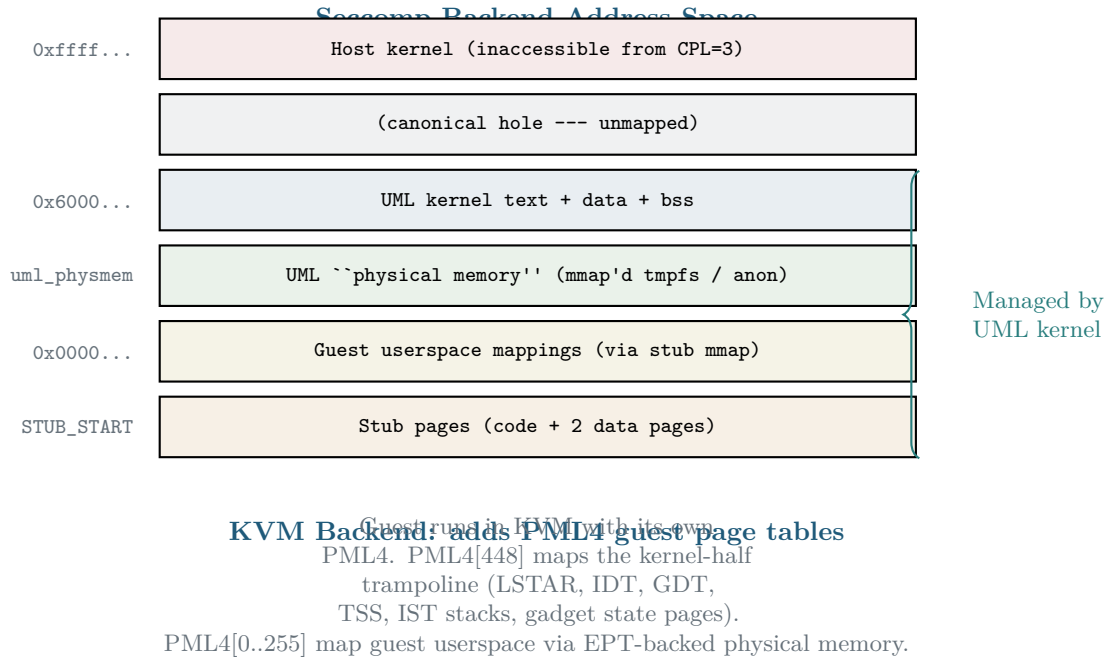


Figure 3.2: Address space layout for the `seccomp` backend. The KVM backend additionally constructs guest-side PML4 page tables that map the same physical memory through EPT/NPT.

4. **UML kernel text/data.** The UML binary’s own code and data segments, loaded by the host’s ELF loader.

3.5.2 KVM Backend: Guest Page Tables

The KVM backend constructs a full x86-64 PML4 page-table hierarchy for the guest. UML’s physical memory is registered with KVM via `KVM_SET_USER_MEMORY_REGION`, which populates EPT/NPT entries so that guest-physical addresses translate through host-virtual addresses to host-physical pages without hypervisor intervention on each access.

The guest PML4 contains a *kernel half* at PML4 slot 448 (guest virtual address `0xffffe000_00000000`) that maps the trampoline page (LSTAR entry point), the IDT, exception handler stubs, the GDT, per-vCPU TSS and IST stack pages, and per-vCPU gadget state pages. This kernel-half mapping is marked `US=0` (supervisor-only) so guest userspace at `CPL=3` cannot access it directly; the CPU transitions to `CPL=0` only transiently during the `SYSCALL` instruction’s microarchitectural handling before the LSTAR trampoline code runs.

3.6 Trap Mechanisms Compared

The performance of UML is dominated by the cost of intercepting guest system calls. Three mechanisms have been used historically and in the current implementation. Figure 3.3 illustrates the data flow for each.

3.6.1 ptrace: PTRACE_SYSEMU

The original UML trap mechanism uses `ptrace()` to intercept every guest system call:

1. Guest userspace executes the `syscall` instruction.
2. The host kernel intercepts it (via `PTRACE_SYSEMU`) and stops the tracee with signal `SIGTRAP|0x80`.

3. The UML kernel (the tracer) wakes from `waitpid()`.
4. `PTRACE_GETREGS` reads the tracee's register state.
5. The UML kernel dispatches the appropriate handler.
6. `PTRACE_SETREGS` writes the return value.
7. `PTRACE_SYSEMU` resumes the tracee.

Each round-trip requires two context switches between the tracer and the tracee plus multiple kernel transitions. Measured latency: **1 μ s to 5 μ s** per system call.

3.6.2 seccomp: SIGSYS + futex

The `seccomp` backend (contributed by Johannes Berg, circa 2022) replaces the external-tracer model with an in-process signal:

1. Guest userspace executes the `syscall` instruction.
2. The host kernel's `seccomp` filter returns `SECCOMP_RET_TRAP`, delivering `SIGSYS` to the stub thread—*without executing the system call*.
3. The stub's `SIGSYS` handler saves register state into the shared `stub_data` page.
4. The stub issues `FUTEX_WAKE` on the shared `futex` word, waking the UML kernel thread.
5. The UML kernel reads registers from `stub_data`, dispatches the handler, and writes the result back.
6. The UML kernel issues `FUTEX_WAKE`; the stub wakes from its `FUTEX_WAIT` and resumes guest execution from the signal return path.

Because no external tracer is involved, the context-switch overhead is eliminated. The signal delivery and `futex` round-trip are the dominant costs. Measured latency: **300 ns to 500 ns** per system call—roughly an order of magnitude faster than `ptrace()`.

3.6.3 KVM: vmentry/vmexit + LSTAR Gadget

The KVM backend replaces the signal-based mechanism with hardware VM entry/exit:

1. Guest userspace executes the `syscall` instruction.
2. The CPU loads `MSR_LSTAR` into `RIP` (standard x86-64 `SYSCALL` behavior). Inside the guest VM, `LSTAR` points to the UML trampoline page.
3. The `LSTAR` trampoline executes `out %a1, $0xf4` (an I/O instruction on port `0xf4`).
4. The I/O instruction causes a `KVM_EXIT_IO` VM exit.
5. The UML kernel (running in VMX root mode via the `KVM_RUN` return path) reads the exit reason, extracts registers from the shared `kvm_run` page, and dispatches the system-call handler.
6. The handler writes the return value into `kvm_run`, and the next `KVM_RUN` call re-enters the guest. The guest resumes via `sysretq` from the trampoline.

Measured latency for the full round-trip: **100 ns to 150 ns** per system call.

The LSTAR Gadget: Avoiding the Exit Entirely

For a curated set of trivial system calls (`getpid()`, `gettid()`, `getppid()`, `getuid()`, `geteuid()`, `getgid()`, `getegid()`, `getcpu()`, `time()`, and `clock_gettime(CLOCK_MONOTONIC)`), the LSTAR trampoline contains an expanded 343-byte *gadget* that handles the system call entirely within the guest, without any VM exit:

1. `swapgs` switches GS base to a per-vCPU state page populated by the host on each context switch.
2. A dispatch chain (`cmp/je` on the syscall number in `EAX`) routes to the appropriate handler.
3. The handler reads the pre-computed return value from the per-vCPU state page via `%gs:offset`.
4. `swapgs + sysretq` returns directly to guest userspace.

The entire sequence—including the `SYSCALL/SYSRET` pair—completes in approximately **90 cycles** (≈ 30 ns at 3 GHz). For workloads that call `getpid()` or `clock_gettime` in tight loops, this approaches bare-metal performance.

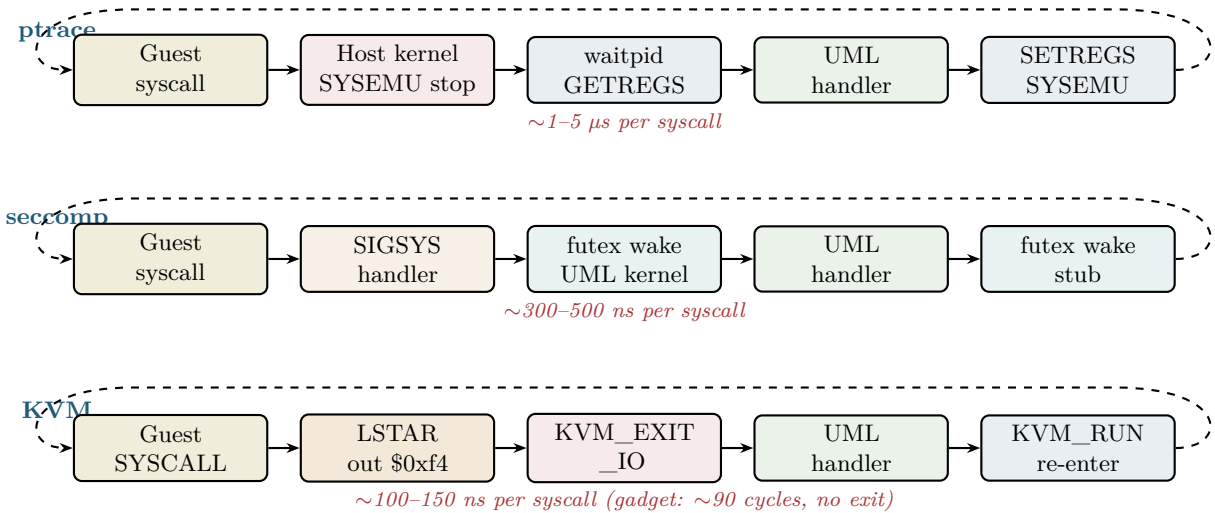


Figure 3.3: System call interception paths for the three UML trap mechanisms. Dashed arrows show the return-to-guest path.

3.6.4 Comparison Table

3.7 Memory Management Under UML

3.7.1 Guest Physical Memory

UML’s “physical memory” is a contiguous host-virtual region of size `physmem_size` (set by the `mem=` boot parameter, default 64 MiB). It is backed either by a file on `tmpfs` (the traditional approach) or by an anonymous `MAP_PRIVATE` mapping. The file-backed approach supports `MAP_SHARED` semantics, enabling host-side inspection of guest memory; the anonymous approach avoids filesystem overhead.

Guest physical addresses are simple offsets into this region:

$$\text{host_va}(\text{gpa}) = \text{uml_physmem} + \text{gpa}$$

Table 3.1: Trap mechanism comparison. Latencies are representative measurements for a null system call (e.g., `getpid()`) on contemporary hardware (Intel Alder Lake / AMD Zen 4, 2024–2026).

Property	<code>ptrace</code>	<code>seccomp</code>	KVM (v2)
Trap mechanism	<code>PTRACE_SYSEMU</code> → <code>SIGTRAP 0x80</code>	<code>SECCOMP_RET_TRAP</code> → <code>SIGSYS</code>	<code>SYSCALL</code> → <code>LSTAR</code> → <code>out</code> → <code>KVM_EXIT_IO</code>
Syscall latency	1 μ s to 5 μ s	300 ns to 500 ns	100 ns to 150 ns
Gadget fast-path	N/A	N/A	~90 cycles (~30 ns), no VM exit
Context switches	2 per syscall (tracer ↔ tracee)	0 (in-process signal)	1 VM exit + 1 VM entry
Host kernel requirement	<code>PTRACE_SYSEMU</code> support	<code>SECCOMP_RET_TRAP</code> (kernel ≥ 3.5)	/dev/kvm + VT-x/AMD-V
Guest memory model	<code>mmap()</code> into stub process	<code>mmap()</code> into stub process	<code>KVM_SET_USER_MEMORY_REGION</code> (EPT/NPT)
SMP model	One host process per guest CPU	One host thread per guest CPU	One vCPU per host CPU (pool)
Debugging support	Full <code>ptrace</code> attach	Limited (signal-based)	<code>KVM_GUESTDBG_*</code> (breakpoints, single-step)
Kernel version in-tree	v2.4.x – present (mainline)	v6.1 – present (Berg series)	v2 in development (this work)

3.7.2 Guest Page Tables

UML maintains full x86-64 four-level page tables (PML4 → PDP → PD → PT) in guest physical memory. These page tables serve different purposes depending on the backend:

Seccomp/ptrace backends.

Guest page tables exist only as a data structure internal to the UML kernel. They record the guest’s virtual-to-physical mappings, but the *actual* address translation is performed by the host kernel’s page tables (the stub process’s `mm_struct`). When the guest modifies a PTE, UML must call `mmap()/mprotect()` on the stub process to make the host-side mapping match.

KVM backend.

Guest page tables are loaded into the vCPU’s CR3 and walked by hardware during guest execution. EPT/NPT provides the second level of translation (guest-physical to host-physical). UML still maintains the guest page tables as data structures, but they are now *live*—the CPU reads them directly.

3.7.3 The TLB Synchronization Problem

When the UML kernel modifies a guest page-table entry—due to a page fault, an `mmap()`, an `mprotect()`, or a process exit—the change must be propagated to the execution substrate:

Seccomp backend:

Each PTE change is translated into a host `mmap()` or `mprotect()` call, batched into the

stub’s syscall buffer (`stub_data->syscall_data`), and flushed on the next stub round-trip. This is the `um_tlb_sync()` path in `arch/um/kernel/tlb.c`.

KVM backend:

PTE changes are written directly into the guest page-table pages (which live in UML’s physical memory region). The UML kernel then issues `INVLPG` or a full CR3 reload within the guest to invalidate stale TLB entries. On SMP configurations, `tlb_kick_others()` sends an IPI to force all other vCPUs to flush their TLBs as well—this is the KVM-specific backend op that the `seccomp` backend leaves `NULL` (the host kernel’s `mmu_notifier` mechanism handles cross-CPU invalidation transparently for the `seccomp` path).

The TLB sync path is one of the most performance-sensitive operations in UML. The `seccomp` backend batches multiple PTE changes into a single stub round-trip (amortizing the `futex` wake/wait overhead over dozens of mappings), while the KVM backend avoids host system calls entirely for the common case, paying only the cost of a guest-side `INVLPG` or CR3 write.

3.8 Signal Handling

Signals are the “interrupts” of the UML kernel. Where a bare-metal kernel receives timer ticks via a local APIC interrupt and I/O notifications via device IRQs, UML receives:

SIGALRM:

Timer interrupts. The UML kernel programs a host `timer_create/timer_settime` timer (or `setitimer` on older configurations) that delivers `SIGALRM` at the frequency required by the guest’s HZ setting. This drives the scheduler tick, watchdog timers, and all time-dependent kernel subsystems.

SIGIO:

I/O readiness notifications. UML’s virtual device drivers register host file descriptors with the UML IRQ subsystem, which uses `epoll` (or `poll`) and arranges for `SIGIO` delivery when any registered descriptor becomes ready. This is the functional equivalent of a hardware IRQ line going active.

SIGSEGV:

Page faults. When guest code accesses an address that is not mapped in the stub process’s host-side page tables (for the `seccomp` backend) or that causes an EPT violation (for the KVM backend), the resulting fault is delivered to the UML kernel, which resolves it by consulting the guest page tables, allocating physical pages if necessary, and updating the mapping.

SIGSYS:

System call interception (`seccomp` backend only). As described in Section 3.6, the `seccomp` filter’s `SECCOMP_RET_TRAP` action delivers this signal to indicate that the guest issued a system call.

3.8.1 Signal Delivery Across Backends

The signal model differs substantially between backends:

- **Seccomp backend:** `SIGALRM` and `SIGIO` are delivered asynchronously to the UML kernel thread (the “spawner” process or the per-mm worker process). The signal handler runs in the UML kernel’s context, interrupting whatever kernel code was executing. Signal masking ensures that only one signal is handled at a time.

- **KVM backend:** While a vCPU is inside `KVM_RUN`, host signals cannot be delivered to the thread—the thread is in kernel space executing the KVM entry/exit loop. Instead, pending signals cause `KVM_RUN` to return with `-EINTR` (or, with signal-window-based delivery, the signal is handled before the next `KVM_RUN` call). The UML kernel’s vCPU run loop checks for pending signals after each VM exit and dispatches them through the standard UML interrupt-handling path.

This means `SIGALRM` timer ticks are effectively “held” during guest execution and processed at the next VM exit boundary. For the `LSTAR` gadget fast path, which handles system calls without any VM exit, this creates a timer-delivery gap: a tight loop calling only gadget-handled system calls could theoretically starve the timer. The gadget’s *budget* mechanism addresses this: each gadget-handled `clock_gettime` call decrements a per-vCPU budget counter, and when the budget reaches zero, the gadget falls back to the full VM exit path, ensuring the host has an opportunity to deliver pending signals.

3.8.2 Signal Masking and Reentrancy

UML’s signal handlers must be carefully written to avoid reentrancy issues. The UML kernel uses `sigprocmask` to block signals during critical sections (analogous to `local_irq_disable()` on bare metal) and unblocks them at safe points (analogous to `local_irq_enable()`). The `block_signals_trace()` / `unblock_signals_trace()` wrappers in `arch/um/os-Linux/signal.c` implement this protocol, and all signal-handler dispatchers use them to ensure that a `SIGALRM` arriving during `SIGIO` handling (or vice versa) is deferred rather than creating a nested handler invocation that could corrupt kernel state.

With these foundations established—the virtualization taxonomy, the x86 privilege model, hardware virtualization extensions, the Linux process model, address-space layout, trap mechanisms, memory management, and signal handling—we are now equipped to examine the architecture of UML itself in Chapter 4.

Chapter 4

UML Architecture and Design

Every architecture port is a negotiation between the abstractions the kernel expects and the mechanisms the hardware provides. In UML, the “hardware” is another kernel.

Author’s observation

User-Mode Linux is, at its core, an architecture port. It implements the same interfaces that `arch/x86`, `arch/arm64`, or `arch/riscv` implement—trap dispatch, context switching, page table management, timer infrastructure, SMP synchronization—but the target “hardware” is not a CPU in ring 0. It is the host Linux kernel, accessed through system calls. This chapter describes the internal architecture of the `arch/um/` tree as it stands after the v2 redesign: the directory layout, the three-layer abstraction model, the backend selection mechanism, the stub process model, guest memory management, symmetric multiprocessing, the device model, and the build system.

4.1 The `arch/um/` Directory

In the Linux source tree, every supported architecture lives under a dedicated subdirectory of `arch/`. UML is no exception: `arch/um/` contains roughly 40,000 lines of C (excluding tests and the archived v1 KVM backend) spread across six major subdirectories and their associated headers. What makes UML special is that the “hardware abstraction layer” at the bottom of this code does not talk to PCI buses, interrupt controllers, or MMU hardware. Instead it calls `clone()`, `mmap()`, `mprotect()`, `futex()`, `ioctl()`, and `sigaction()` on the host kernel [1, 22].

4.1.1 Directory Structure

Table 4.1 summarizes the top-level layout. Two features of this structure deserve emphasis.

First, the split between `kernel/` and `os-Linux/` is an artifact of UML’s dual-world nature. Code in `kernel/` is compiled as kernel code: it has access to `<linux/types.h>`, `<linux/spinlock.h>`, the kernel’s `printk`, and all the usual kernel APIs. Code in `os-Linux/` is compiled as *host userspace* code: it can call `malloc`, `pthread_create`, and POSIX signal functions directly. The two worlds share data structures through the headers in `include/shared/`, but they cannot freely call each other’s functions without crossing the kernel/user boundary through well-defined interfaces (the `os_*` function family declared in `include/shared/os.h`).

Second, the `backend/` directory is new in the v2 redesign. Prior to the backend ops abstraction (Section 4.2), the seccomp and ptrace trap mechanisms were interleaved throughout `os-Linux/skas/` and gated by `if (using_seccomp)` conditionals. The v2 work extracted each backend into its own directory with a clean ops table, making it possible to add the KVM backend without touching the seccomp code paths.

Table 4.1: Top-level directories under `arch/um/`.

Directory	Purpose
<code>kernel/</code>	Core kernel-side logic: trap dispatch (<code>trap.c</code>), TLB synchronization (<code>tlb.c</code>), SMP (<code>smp.c</code>), timekeeping (<code>time.c</code>), backend arbiter (<code>backend.c</code>), Layer 2 static-key hooks (<code>hooks.c</code> , <code>hooks_bench.c</code>), template pause (<code>template_pause.c</code>), process management (<code>process.c</code>), and the linker script (<code>vmlinux.lds.S</code>).
<code>os-Linux/</code>	Host-side userspace code: signal handling (<code>signal.c</code>), memory operations (<code>mem.c</code>), process management (<code>process.c</code>), SMP pthread lifecycle (<code>smp.c</code>), startup and probe (<code>start_up.c</code>), and the <code>skas/</code> subdirectory containing the SKAS trap loop (<code>process.c</code>) and stub syscall batching (<code>mem.c</code>). These files are compiled with <code>USER_CFLAGS</code> —they link against <code>glibc</code> and the host kernel’s UAPI.
<code>backend/</code>	Pluggable backend implementations. Currently: <code>seccomp/</code> (the mainline seccomp-SIGSYS backend), <code>kvm-v2/</code> (the v2 KVM backend), <code>common/</code> (shared backend utilities), <code>contract/</code> (KUnit conformance tests), and <code>kvm-v1-archive/</code> (the archived v1 implementation, gated by <code>depends on BROKEN</code>).
<code>drivers/</code>	UML-specific device drivers: the <code>ubd</code> block device, vector and vector2 network drivers, console/TTY drivers, the management console (<code>mconsole</code>), <code>virtio-uml</code> , <code>vfio-uml</code> , and the host audio passthrough.
<code>include/</code>	Headers, organized into <code>asm/</code> (kernel-internal architecture headers), <code>shared/</code> (headers reachable from both kernel and USER translation units), and <code>uapi/</code> (userspace-visible ABI).
<code>configs/</code>	Default Kconfig files: <code>x86_64_defconfig</code> , <code>i386_defconfig</code> , <code>base_defconfig</code> , and a <code>profiles/</code> subdirectory with tuned configurations for production, fuzzing, research, and sandbox use cases.

4.2 The Three-Layer Architecture

The v2 redesign formalized UML’s runtime instrumentation and trap-mechanism abstraction into three cleanly separated layers. This decomposition is the central architectural contribution of the redesign effort. Each layer addresses a different concern, and the layers compose orthogonally: any combination of Layer 1 backend, Layer 2 gate configuration, and Layer 3 sanitizer set produces a valid kernel.

4.2.1 Layer 1: The Backend Ops Table

The foundation of the architecture is `struct um_backend_ops`, defined in `arch/um/include/shared/backend.h`. Every backend—seccomp, `kvm-v2`, and any future backend—populates a single global instance of this structure. The structure carries a contract version (`UM_BACKEND_CONTRACT_VERSION`, currently 2), a backend kind enumeration, four capability flags, and 18 function pointers organized into five categories:

```

1 struct um_backend_ops {
2     const char      *name;
3     enum um_backend_kind kind;
4     u32              contract_version;
5
6     /* Capability flags */
7     bool  uses_stub_reaper;

```



Figure 4.1: The three-layer architecture of UML’s v2 redesign. Each layer composes orthogonally with the others.

```

8  bool  has_syscall_stub_fd_map;
9  bool  stub_syscall_uses_futex;
10 bool  stub_child_runs_seccomp;
11
12  /* Lifecycle and trap (4) */
13  int  (*probe)(void);
14  int  (*init)(const struct um_backend_args *args);
15  void (*shutdown)(void);
16  void (*vcpu_run)(struct um_pt_regs *regs);          /* HOT */
17
18  /* Memory (5) */
19  int  (*mm_create)(struct mm_struct *mm);
20  void (*mm_destroy)(struct mm_struct *mm);
21  int  (*mm_region_added)(...);                      /* HOT */
22  int  (*mm_region_removed)(...);                    /* HOT */
23  int  (*mm_region_protected)(...);
24
25  /* Scheduling (4 + 1 optional) */
26  int  (*thread_create)(...);
27  int  (*thread_start_idle)(...);
28  void (*context_switch)(struct task_struct *prev,    /* HOT */
29                        struct task_struct *next);
30  int  (*ipi_send)(int cpu, int vector);
31  void (*tlb_kick_others)(struct mm_struct *mm);    /* may be NULL */
32
33  /* Time (3) */
34  u64  (*read_clock_ns)(void);                      /* HOT */
35  int  (*set_timer)(int cpu, u64 deadline_ns,
36                  enum um_timer_mode mode);
37  u64  (*read_persistent_clock_ns)(void);
38
39  /* Debug / introspection (3) */
40  void (*init_thread_regs)(...);
41  int  (*read_guest_regs)(...);
42  int  (*write_guest_regs)(...);
43  };

```

Listing 4.1: Abbreviated `struct um_backend_ops` (from `arch/um/include/shared/backend.h`).

The **HOT ops**—`vcpu_run`, `mm_region_added`, `mm_region_removed`, `context_switch`, and `read_clock_ns`—are the operations invoked on every guest system call, page fault, context switch, or timer read. These are validated non-NULL at boot time by `validate_hot_ops()` in `arch/um/kernel/backend.c`; a NULL hot op triggers an immediate kernel panic with a diagnostic message identifying the missing operation.

The dispatch macro provides the performance-critical abstraction:

```
1 #if defined(CONFIG_UM_BACKEND_SECCOMP_ONLY)
2 # define um_backend_dispatch(op, ...) seccomp_##op(__VA_ARGS__)
3 #else
4 # define um_backend_dispatch(op, ...) (um_backend->op(__VA_ARGS__))
5 #endif
```

Listing 4.2: The dispatch macro resolves at compile time in single-backend builds.

In a single-backend build (`CONFIG_UM_BACKEND_SECCOMP_ONLY`), the macro expands to a direct function call via token pasting—zero indirect dispatch overhead. In a dynamic build (`CONFIG_UM_BACKEND_DYNAMIC`), the macro becomes a function pointer dereference through the global `um_backend` pointer, costing a single well-predicted indirect call (~5–10 cycles on modern x86).

4.2.2 Layer 2: Static-Key Hot-Path Gates

Layer 2 addresses a design tension that plagues every kernel instrumentation system: the choice between *fast* and *observable*. Traditional approaches force this choice at compile time (e.g., enabling `CONFIG_FTRACE` imposes a permanent overhead) or at boot time (e.g., kernel command-line tracing parameters). UML’s static-key gates make observability a *runtime decision per event*.

The mechanism uses Linux’s jump-label infrastructure (`<linux/jump_label.h>`). Each gate is a `DEFINE_STATIC_KEY_FALSE` instance. When off, the call site contains either a 5-byte NOP (with `HAVE_ARCH_JUMP_LABEL`) or a load-compare branch that the CPU’s branch predictor resolves in under 2 ns. When on, the jump-label machinery patches the NOP to a jump into the slow-path handler.

Six gates are currently defined, each controlling a family of hook sites:

1. `um_hook_trace_syscalls` — system call entry/exit tracing, page fault tracing, context switch tracing, IRQ entry tracing, clock read tracing.
2. `um_hook_kcov_enabled` — coverage-guided fuzzing feedback at syscall boundaries.
3. `um_hook_time_travel_active` — deterministic time injection for reproducible execution.
4. `um_hook_kfence_sample` — KFENCE heap sampling at clock ticks.
5. `um_hook_record_replay` — record/replay event capture at syscall, page fault, context switch, IRQ, and clock boundaries.
6. `um_hook_perf_dispatch` — performance counter dispatch at syscall and context switch points.

The hook helpers are declared `__always_inline` so that the static-branch check expands *at the call site*, not inside a function prologue. A typical helper composes multiple gates:

```
1 static __always_inline void um_on_syscall_entry(struct pt_regs *regs)
2 {
```



```

3   if (static_branch_unlikely(&um_hook_trace_syscalls))
4       __um_trace_syscall_entry(regs);
5   if (static_branch_unlikely(&um_hook_kcov_enabled))
6       __um_kcov_record_syscall(regs);
7   if (static_branch_unlikely(&um_hook_record_replay))
8       __um_record_event_syscall_entry(regs);
9   if (static_branch_unlikely(&um_hook_perf_dispatch))
10      __um_perf_syscall(regs);
11 }

```

Listing 4.3: The `um_on_syscall_entry` hook helper composes four gates into a single inline expansion.

Gate state is toggled at runtime through debugfs: `/sys/kernel/debug/um/hooks/<name>` accepts writes of 1 (on) or 0 (off). Each gate maintains per-CPU hit counters, summed on read through `um_hook_stats_read()`, for observability without cross-CPU cacheline contention.

An in-kernel microbenchmark (`arch/um/kernel/hooks_bench.c`) measures per-gate cost at runtime via debugfs: 10 batches of 100,000 iterations each per hook site, median reported. This is the source of truth for the cost model.

4.2.3 Layer 3: Compile-Time Wraps

Layer 3 comprises the standard kernel sanitizer and coverage tools that UML supports by virtue of being a “normal” architecture port compiled by a standard compiler toolchain:

- **KASAN** (Kernel Address Sanitizer) [38] — detects out-of-bounds and use-after-free bugs in slab/page allocator objects.
- **KMSAN** (Kernel Memory Sanitizer) [39] — detects uses of uninitialized memory.
- **KCSAN** (Kernel Concurrency Sanitizer) [40] — detects data races via compile-time instrumentation of memory accesses.
- **KFENCE** — lightweight probabilistic heap-object guard-page detection, sampling at timer ticks (gated by the Layer 2 `kfence_sample` hook).
- **KCOV** — coverage-guided fuzzing feedback, collecting per-task coverage bitmaps for tools like syzkaller [9].

These wraps are completely orthogonal to Layers 1 and 2. A KASAN-instrumented UML running the seccomp backend with all Layer 2 gates off is a valid configuration; so is a clean (no-sanitizer) UML running the kvm-v2 backend with all gates on. The three layers multiply rather than interfere.

4.2.4 Cost Model

Table 4.2 shows the approximate per-event overhead for each layer and their composition into representative profiles. The baseline is a “bare” guest system call round-trip (guest → trap → kernel → return). All figures assume x86-64, seccomp backend, single-backend inline dispatch.

The key insight is that Layers 1 and 2 add *nanoseconds* of overhead per event, while Layer 3 (sanitizers) dominates total cost when enabled. The three-layer decomposition ensures that the fast path pays only for what it uses, and the decision to enable observability never requires a rebuild.

Table 4.2: Per-syscall-roundtrip cost model across layers and profiles. Figures are representative; actual measurements vary with host microarchitecture and workload.

Profile	Layer 1	Layer 2	Layer 3	Total overhead
prod-fast	0 (inline)	~4–8 ns (4 NOPs)	0	~4–8 ns
prod-with-hooks	0 (inline)	~50–100 ns (gates on)	0	~50–100 ns
research	~5–10 ns	~50–100 ns	0	~55–110 ns
fuzz	0 (inline)	~20–40 ns (kcov on)	KASAN: 2–3×	2–3×
fuzz-deep	0 (inline)	~50–100 ns	KASAN+KMSAN: 4–6×	4–6×
race	~5–10 ns	~50–100 ns	KCSAN: 3–5×	3–5×

4.3 Backend Selection Mechanism

Backend selection operates at two levels: *build-time* configuration through Kconfig, and *boot-time* resolution through the `backend=` kernel command-line parameter.

4.3.1 Build-Time: Kconfig

The Kconfig choice block in `arch/um/Kconfig` offers two dispatch modes:

CONFIG_UM_BACKEND_SECCOMP_ONLY

Single-backend inline dispatch. The `um_backend_dispatch()` macro expands to direct `seccomp_<op>()` calls via token pasting. Zero indirect-dispatch cost. Smallest binary. This is the recommended mode for production and sandbox deployments.

CONFIG_UM_BACKEND_DYNAMIC

Multi-backend indirect dispatch. Compiles every available backend (currently `seccomp`; `kvm-v2` joins this path once its Phase A integration is complete). The dispatch macro dereferences the global `um_backend` pointer, costing one well-predicted indirect call per site. This is the recommended mode for distribution kernels and CI matrices that need a single binary to support multiple backends.

Both modes `select UM_BACKEND_SECCOMP`; the `seccomp` backend is always available. The `kvm-v2` backend is gated by `CONFIG_UM_BACKEND_KVM_V2`, which in turn depends on `EXPERT` and `CONFIG_UM_BACKEND_DYNAMIC`.

4.3.2 Boot-Time: `init_backend()`

The function `init_backend()` in `arch/um/kernel/backend.c` is called once from `setup_arch()` during early boot. It resolves the backend selection, validates HOT ops, invokes the backend's `probe()` and `init()` lifecycle ops, and sets the immutable global `um_backend` pointer.

In `CONFIG_UM_BACKEND_DYNAMIC` builds, the resolution cascade in `pick_dynamic_backend()` is:

1. If `backend=force=<kind>` was specified and the requested backend is not compiled in, the kernel panics immediately.
2. If `backend=<kind>` was specified (non-force), the requested backend is used if available; otherwise the kernel falls back with a `pr_warn`.
3. If `backend=auto` (the default) or no parameter was given, the `seccomp` backend is selected.

The `ptrace` backend was removed entirely in the v2 refactoring. Requesting `backend=force=ptrace` triggers a panic with a diagnostic message directing the user to pin to v6.16 or earlier.

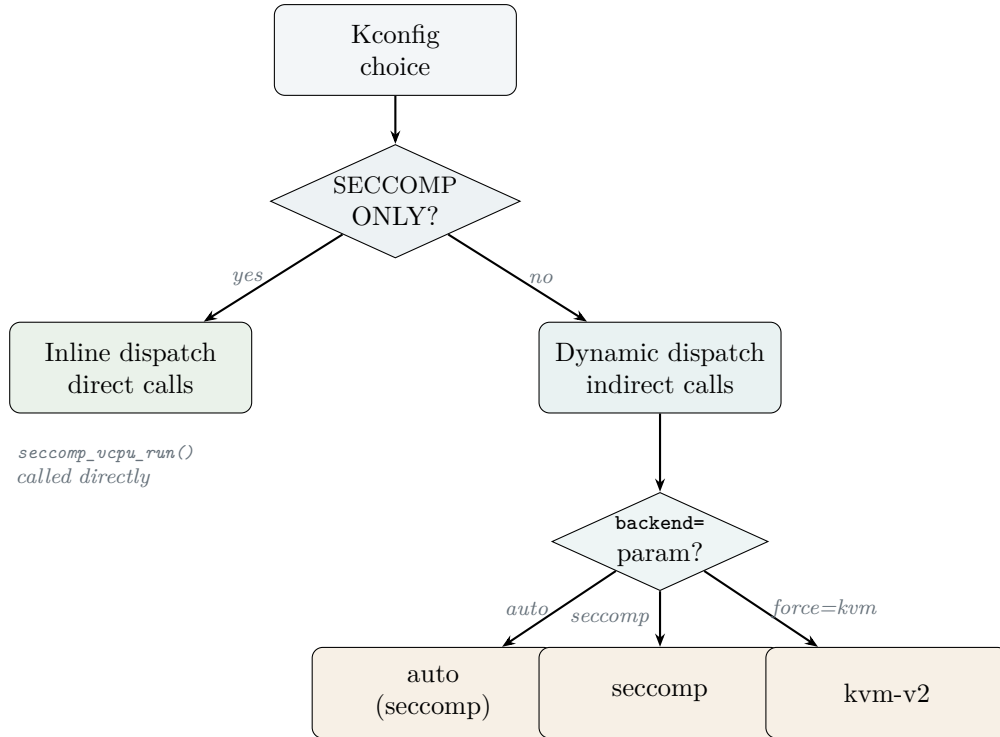


Figure 4.2: Backend selection flow: build-time Kconfig choice followed by boot-time parameter resolution.

4.4 The Stub Mechanism

The seccomp backend’s core abstraction is the *stub child*: a minimal host process that serves as the execution container for a single guest `mm_struct`. Each guest address space gets its own stub child, created by `mm_create()` and destroyed by `mm_destroy()`. The stub provides the host-side process context in which guest userspace code runs and guest system calls are intercepted.

4.4.1 Stub Lifecycle

When a new guest `mm_struct` is created (e.g., on `fork()` or `exec()`), the seccomp backend’s `mm_create` op clones a new host child process. This child—the stub—is not a conventional process. It executes a minimal trampoline (the “stub code”) that:

1. Installs a seccomp BPF filter [28] that traps all system calls, delivering `SIGSYS` to the stub’s signal handler instead of executing the syscall on the host.
2. Maps a shared communication page (the `stub_data` structure) at a fixed address in its address space.
3. Enters an infinite loop: wait on a futex in `stub_data`, process a batch of memory-management syscalls, and signal completion.

4.4.2 The `stub_data` Structure

The shared `stub_data` page (Listing 4.4) is the communication channel between the UML kernel (running in the host parent process) and the stub child. It occupies exactly one page (`UM_KERN_PAGE_SIZE`) and contains:

```

1 struct stub_data {
2     long err;
3     int syscall_data_len;
4     struct stub_syscall syscall_data[...] __aligned(16);
5
6     /* Shared with signal handler (seccomp mode) */
7     short restart_wait;
8     unsigned int futex; /* FUTEX_IN_CHILD or FUTEX_IN_KERN */
9     int signal;
10    unsigned short si_offset;
11    unsigned short mctx_offset;
12
13    struct stub_data_arch arch_data;
14
15    /* Signal handler stack */
16    unsigned char sigstack[UM_KERN_PAGE_SIZE]
17        __aligned(UM_KERN_PAGE_SIZE);
18 };

```

Listing 4.4: The `stub_data` structure (from `arch/um/include/shared/skas/stub-data.h`).

The `syscall_data` array holds a batch of `mmap/munmap` operations that the stub executes on behalf of the UML kernel’s TLB synchronization layer. Batching amortizes the futex wake/wait round-trip cost across multiple memory operations.

4.4.3 Futex-Based Synchronization

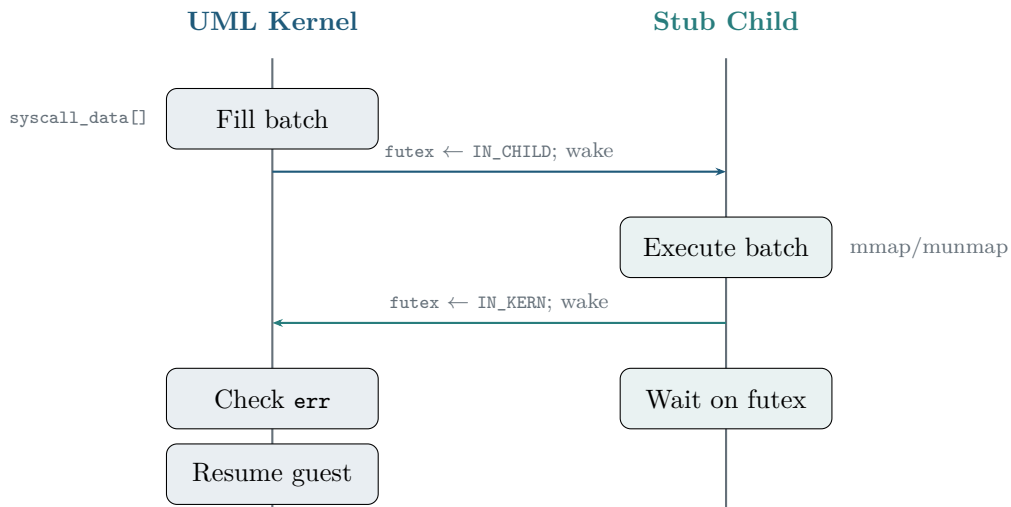


Figure 4.3: Futex-based synchronization between the UML kernel and the seccomp stub child. The shared `stub_data.futex` field alternates between `FUTEX_IN_CHILD` and `FUTEX_IN_KERN`.

Communication uses a single `unsigned int futex` field in `stub_data`. The protocol is simple (Figure 4.3):

1. The UML kernel fills the `syscall_data` array with the batch of memory operations to perform.
2. The kernel sets `futex = FUTEX_IN_CHILD` and issues a `FUTEX_WAKE` system call [36].
3. The stub child wakes, iterates through `syscall_data`, executes each `mmap/munmap`, and records any error in `err`.

4. The stub sets `futex = FUTEX_IN_KERN` and issues `FUTEX_WAKE`.
5. The kernel wakes, checks `err`, and proceeds.

This is substantially faster than the old `ptrace`-based mechanism, which required a `PTRACE_SETREGS` + `PTRACE_CONT` + wait round-trip per operation [35]. The `seccomp` path achieves roughly 3–4× lower syscall round-trip latency.

4.5 Guest Memory Management

UML’s guest physical memory is backed by a host-side memory object: typically an anonymous `mmap` or a `tmpfs` file. Guest page table changes are not self-effectuating—there is no hardware MMU to walk guest page tables. Instead, every guest PTE modification must be translated into a host system call (`mmap`, `munmap`, or `mprotect`) that adjusts the stub child’s host address space to match.

4.5.1 The TLB Synchronization Layer

The `mm-arbiter` in `arch/um/kernel/tlb.c` implements the `um_tlb_sync()` function, which is the single drain point for pending guest page table changes. The flow is:

1. Guest kernel code (e.g., `do_mmap`, `do_munmap`, page fault handler) modifies PTEs and calls `um_tlb_mark_sync()`, which records the affected address range in per-mm `sync_tlb_range_from/sync_tlb_range_to` fields.
2. Before the next guest userspace entry (`vcpu_run`), `um_tlb_sync()` walks the per-mm page table over the marked range, inspects `pte_needsync()` on each leaf, and emits a `struct um_memory_region` per dirty PTE.
3. Each region is dispatched through the backend’s `mm_region_added()` or `mm_region_removed()` op. For the `seccomp` backend, these ops translate into stub child `mmap/munmap` batches. For `kvm-v2`, they translate into KVM memslot updates.
4. After a successful drain, the `mm-arbiter` increments the per-mm `tlb_gen` counter and, if the backend provides a `tlb_kick_others` op, invokes it to notify other vCPUs.

4.5.2 SMP TLB Generation Tracking

Under SMP, multiple vCPUs may be executing code from the same `mm_struct` concurrently. When one vCPU modifies a page table and drains via `um_tlb_sync()`, the other vCPUs must be notified so they can flush their local view of the guest TLB.

The mechanism uses two counters per mm, defined in `arch/um/include/asm/mmu.h`:

```

1 typedef struct mm_context {
2     /* ... */
3     atomic64_t tlb_gen;
4     atomic64_t tlb_gen_seen_by[NR_CPUS];
5 } mm_context_t;
```

Listing 4.5: Per-mm TLB generation counters for SMP coherence.

- `tlb_gen`: a monotonically increasing counter, incremented after each successful `um_tlb_sync()` drain.

- `tlb_gen_seen_by[cpu]`: for each vCPU, the most recent `tlb_gen` value at which that vCPU flushed its local TLB state.

On each `vcpu_run` dispatch, the vCPU compares its `tlb_gen_seen_by[cpu]` against the current `tlb_gen`. On mismatch, it performs a guest TLB flush (for `kvm-v2`, a `CR4.PGE` toggle inside `KVM_RUN`) and updates its slot.

The cross-vCPU kick is targeted: `tlb_kick_others()` only sends IPIs to vCPUs whose `tlb_gen_seen_by[cpu]` is stale relative to the current mm's `tlb_gen`. This narrowing eliminates the IPI storm that a broadcast approach would cause under high mm-churn workloads.

An important subtlety: `tlb_gen_seen_by` is per-(mm, cpu), not per-vCPU globally. A previous design used a single per-vCPU `last_seen_tlb_gen` counter, but this suffered from cross-mm contamination: dispatching `mmB` (gen=5) after `mmA` (gen=3000) wrote 5 into the single counter, causing a return to `mmA` to see a spurious lag of 2995 and potentially misleading the kicker.

4.5.3 Deferred Free: The `mmu_gather` Integration

A subtlety of UML's deferred TLB-sync model is that it violates the standard `mmu_gather` contract. In a hardware architecture, `mmu_gather` collects pages during an unmap, issues a TLB flush, and then frees the pages. In UML, the TLB flush does not happen until the next `vcpu_run` dispatch. This creates a window where freed pages could be reused by the kernel while stale TLB entries still point to them.

The fix (rooted in `mm_context_t.deferred_free_pages`) intercepts the page-free path: instead of releasing pages immediately, they are queued on a per-mm deferred list and freed at the start of the next `vcpu_run`, after the guest TLB has been flushed.

4.6 SMP Implementation

UML implements symmetric multiprocessing by running one host pthread per virtual CPU. This is a cooperative model: the host kernel schedules the pthreads, and UML's scheduler runs within each pthread to multiplex guest tasks.

4.6.1 Thread Model

At boot time, CPU 0 runs on the main thread. For each additional CPU (1 through $N-1$), `os_start_cpu_thread()` in `arch/um/os-Linux/smp.c` calls `pthread_create()` to spawn a new host thread. Each thread is immediately pinned to the corresponding host CPU via `pthread_setaffinity_np()` (SMP-T37):

```
1  cpu_set_t  cpuset;
2  CPU_ZERO(&cpuset);
3  CPU_SET(cpu, &cpuset);
4  pthread_setaffinity_np(cpu_threads[cpu], sizeof(cpuset), &cpuset);
```

Listing 4.6: CPU affinity pinning for vCPU threads (from `arch/um/os-Linux/smp.c`).

This pinning ensures that each guest CPU's host thread always executes on the same physical CPU, which is critical for the `kvm-v2` backend's per-host-CPU vCPU pool: a given guest CPU always dispatches against the same KVM vCPU file descriptor.

4.6.2 Context Switching

A guest context switch is a call to the backend's `context_switch(prev, next)` op. For the seccomp backend, this involves saving and restoring the register state of the stub child process. For

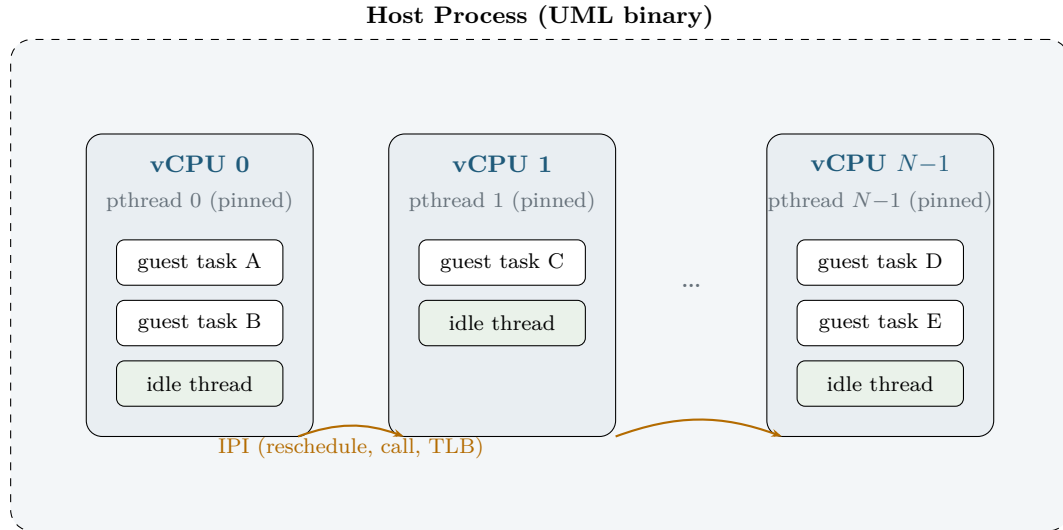


Figure 4.4: UML SMP thread model: one host pthread per virtual CPU, each pinned to a host CPU via `pthread_setaffinity_np`. Guest tasks are scheduled within each pthread by UML’s copy of the Linux scheduler.

kvm-v2, it involves saving FPU state from the per-CPU vCPU into `prev->thread` and loading `next->thread`’s saved state.

4.6.3 IPI Delivery

Inter-processor interrupts are sent through `um_backend_dispatch(ipi_send, cpu, vector)`. Four IPI vectors are defined:

`UML_IPI_RES` Reschedule request (triggers `scheduler_ipi()`).

`UML_IPI_CALL_SINGLE` Single-function call (`generic_smp_call_function_single_interrupt()`).

`UML_IPI_CALL` Broadcast function call (`generic_smp_call_function_interrupt()`).

`UML_IPI_STOP` CPU offline (enter infinite pause loop).

The template-pause mechanism (`arch/um/kernel/template_pause.c`) builds on top of the IPI infrastructure: it allows the UML kernel to quiesce all vCPUs to a named “ready point,” `SIGSTOP` the host process for an external supervisor to fork, and resume with per-child identity blobs. This is the foundation of the pool-based deployment model.

4.6.4 The `migrate_disable` Fix (SMP-T13)

A critical correctness bug in the kvm-v2 backend’s SMP support was discovered and fixed in the SMP-T13 work item (May 2026). The issue: UML builds with `CONFIG_PREEMPT_VOLUNTARY`, which makes `preempt_disable()` a no-op. The kvm-v2 backend’s `vcpu_run` looks up the per-host-CPU vCPU pointer using `smp_processor_id()` and then enters `KVM_RUN`. If the host scheduler migrates the pthread to a different host CPU between the lookup and the `ioctl`, the dispatch proceeds against the wrong vCPU—writing register state to the wrong `kvm_run` mmap, executing the wrong vCPU’s code.

The fix uses `migrate_disable()`, which pins the running task to its current host CPU regardless of the preempt-count semantics. Voluntary scheduling (e.g., inside `handle_syscall` via `sched_yield`) is still allowed—the key invariant is that when the task *resumes*, it resumes on the *same* CPU, so the per-CPU vCPU pointer remains valid.

4.7 Device Model

UML provides a set of paravirtualized devices that bridge the guest kernel’s standard driver interfaces to host resources. These are not emulations of physical hardware; they are purpose-built drivers that use host system calls for I/O.

The device model is deliberately thin. Because UML runs as a userspace process, it cannot intercept hardware interrupts or DMA transactions. Instead, UML devices use file descriptor-based I/O and signal-driven notification (typically `SIGIO`) to simulate device interrupts. The `arch/um/kernel/sigio.c` and `arch/um/os-Linux/sigio.c` files manage the translation of host fd readability events into UML interrupt delivery.

The `vector2` network driver deserves special mention: it replaces the legacy `vector` driver with a cleanly layered architecture separating the transport mechanism (how packets reach the host), the device model (how the kernel sees the NIC), and the queue management (how packets are buffered and batched). Each layer is independently testable via KUnit, and the transport layer is pluggable—adding a new transport (e.g., `AF_XDP`) requires implementing a small interface without touching the core driver.

4.8 Build System

Building UML requires specifying `ARCH=um` on the make command line. This selects the `arch/um/` directory as the architecture-specific source tree and uses the host’s native compiler (no cross-compilation toolchain needed, since the “target” is the host architecture itself running in userspace).

4.8.1 Basic Build

```
1 # Configure
2 make ARCH=um O=build/um x86_64_defconfig
3
4 # Optional: customize
5 make ARCH=um O=build/um menuconfig
6
7 # Build
8 make ARCH=um O=build/um -j$(nproc)
9
10 # Result: build/um/vmlinux (an ELF binary)
```

Listing 4.7: Standard UML build commands.

The output is a standard ELF binary named `vmlinux` that can be executed directly from the command line:

```
1 ./build/um/vmlinux mem=512M rootfstype=hostfs \
2   rw init=/bin/bash
```

4.8.2 The `O=` Discipline

An important operational requirement is to *always* use the `O=` out-of-tree build directory. A bare `make ARCH=um` (without `O=`) contaminates the source tree with generated files that opaquely break future `O=` builds. Recovery requires `make mrproper` (which removes all generated files, including `.config`). All examples in this report and in the project’s CI configurations use the `O=build/um` convention.

4.8.3 Kconfig Structure

UML’s Kconfig inherits from the host architecture (x86 on x86-64 hosts) but adds UML-specific options under the UML-prefixed namespace. Key configuration groups include:

```
CONFIG_UM_BACKEND_*
    Backend selection (Section 4.3).

CONFIG_UM_BACKEND_KVM_V2
    The v2 KVM backend (requires EXPERT).

CONFIG_UM_WORKER_PROCESS
    Per-mm host worker process model.

CONFIG_UM_TEMPLATE_PAUSE_FORK
    Fork-on-resume pool model.

CONFIG_UML_NET_*
    Network transport selection (TAP, raw, etc.).

CONFIG_BLK_DEV_UBD
    The ubd block device.

CONFIG_HOSTFS
    Host filesystem passthrough.
```

The `profiles/` subdirectory under `arch/um/configs/` provides ready-made configurations for common use cases: `prod-fast` (minimal overhead, single-backend inline dispatch, no sanitizers), `fuzz` (KASAN + KCOV + snapshot/forkserver), `research` (dynamic dispatch with all hooks available), and others.

4.8.4 SUBARCH and Host Architecture

UML’s build system uses a `SUBARCH` variable to determine the host architecture. On x86-64 systems, `SUBARCH=x86` is detected automatically. The host architecture’s headers (e.g., register layouts, calling conventions, signal frame structures) are pulled in from `arch/x86/` via the `HEADER_ARCH` mechanism in the top-level `arch/um/Makefile`:

```
1 HEADER_ARCH := $(SUBARCH)
2 ifneq ($(filter $(SUBARCH),x86 x86_64 i386),)
3     HEADER_ARCH := x86
4 endif
5 HOST_DIR := arch/$(HEADER_ARCH)
```

Listing 4.8: Host architecture header selection.

This dual-architecture nature—`arch/um/` for UML-specific code, `arch/x86/` for host-specific data structures—is what allows UML to run unmodified x86-64 binaries in the guest while itself being a fundamentally different “architecture” from the kernel’s perspective.

4.9 Chapter Summary

UML’s architecture is the product of a sustained negotiation between the abstractions the Linux kernel expects from a hardware platform and the mechanisms available through the host kernel’s system call interface. The v2 redesign formalized this negotiation into three orthogonal layers:

1. A **backend ops table** that encapsulates the trap mechanism behind a stable 18-operation interface, enabling both seccomp and KVM backends to coexist with zero code duplication in the shared kernel paths.
2. **Static-key gates** that make observability a per-event runtime decision, with nanosecond-scale overhead when disabled and no rebuild required to toggle.
3. **Compile-time sanitizer wraps** that compose with the first two layers without interference, enabling the full spectrum from bare-metal-speed production to deep sanitizer coverage.

The stub mechanism, TLB generation tracking, SMP thread pinning, and device model complete the picture of a virtual machine monitor that is simultaneously a kernel architecture port, a testing platform, and a research vehicle. Chapter 5 explores the implementation details of each subsystem, and Chapter 6 describes how the kvm-v2 backend leverages this architecture to add hardware-accelerated execution.

Table 4.3: UML device drivers and their host-side backing mechanisms.

Device	Source	Description
ubd	ubd_kern.c	Block device backed by host files (typically raw disk images or sparse files). Supports multiple partitions, copy-on-write layers, and synchronous or asynchronous I/O modes.
Vector network	vector_kern.c	Legacy network driver using vector (scatter-gather) transport. Supports TAP, raw sockets, and various encapsulation modes.
Vector2 network	vector2_core.c	Next-generation network driver with a modular transport/model/queue architecture. Supports TAP devices, host file descriptors, ethtool statistics, and pluggable queue management. Default in the v2 redesign.
Console/TTY	stdio_console.c	Serial-like consoles mapped to host stdio, PTYs, or named pipes. Multiple virtual consoles supported (<code>con0</code> , <code>con1</code> , ...), including the <code>stderr</code> console for early-boot diagnostics.
mconsole	mconsole_kern.c	UML-specific management console accessible via a Unix domain socket. Supports runtime commands: <code>halt</code> , <code>reboot</code> , <code>config</code> (hot-add devices), <code>sysrq</code> , <code>cad</code> (Ctrl-Alt-Del), and <code>proc</code> (read /proc entries from outside).
virtio-uml	virtio_uml.c	Standard virtio devices using vhost-user sockets. Allows UML to use the same virtio device backends (e.g., vhost-user-blk, vhost-user-net) that QEMU and other VMMs use.
hostfs	fs/hostfs/	Passthrough filesystem that maps a host directory tree directly into the guest's VFS namespace. Not a block device driver but a filesystem type (<code>mount -t hostfs</code>). Invaluable for development: edit files on the host, see them immediately inside the guest.
vfio-uml	vfio_kern.c	VFIO device passthrough for assigning host PCI devices to the UML guest (requires IOMMU support on the host).
Harddog	harddog_kern.c	Software watchdog timer backed by a host helper process.
RTC	rtc_kern.c	Real-time clock providing time-of-day to the guest, backed by host <code>clock_gettime</code> .
Random	random.c	Entropy source backed by host <code>/dev/urandom</code> .

Chapter 5

UML Implementation Details

Show me the data structures and I won't usually need your code; it will be obvious.

Fred Brooks, paraphrased

This chapter descends from the architectural overview of the previous chapters into the concrete data structures, code paths, and protocols that make User-Mode Linux work. Where earlier chapters described *what* UML does, this chapter describes *how*: the exact sequence of system calls, signal deliveries, futex operations, and page-table walks that execute when a guest process issues a system call, takes a page fault, or is context-switched. All code references are relative to the `arch/um/` subtree of the Linux 6.16+ source unless noted otherwise.

5.1 Process Lifecycle

A UML guest process is, from the host kernel's perspective, just another userspace thread. Its lifecycle maps onto the standard Linux process model but with a UML-specific twist at each stage: creation, scheduling, and destruction are all mediated through the active backend.

5.1.1 Creation: fork through `copy_thread`

When a guest process calls `fork()` or `clone()`, the standard kernel path reaches `copy_process()`, which eventually calls the architecture-specific `copy_thread()` in `arch/um/kernel/process.c`. This function is the first point where UML diverges from a hardware architecture port:

```
1  int copy_thread(struct task_struct *p,
2                  const struct kernel_clone_args *args)
3  {
4      void (*handler)(void);
5
6      p->thread = (struct thread_struct) INIT_THREAD;
7
8      if (!args->fn) {
9          /* Forking a user process: copy parent's register state */
10         memcpy(&p->thread.regs.regs, current_pt_regs(),
11              sizeof(p->thread.regs.regs));
12         PT_REGS_SET_SYSCALL_RETURN(&p->thread.regs, 0);
13         if (sp != 0)
14             REGS_SP(p->thread.regs.regs.gp) = sp;
15         handler = fork_handler;
16         arch_copy_thread(&current->thread.arch, &p->thread.arch);
17     } else {
18         /* Kernel thread: set up fresh register state */
19         um_backend_dispatch(init_thread_regs,
20                             p->thread.regs.regs.gp,
21                             p->thread.regs.regs.fp);
```

```

22     p->thread.request.thread.proc = args->fn;
23     p->thread.request.thread.arg  = args->fn_arg;
24     handler = new_thread_handler;
25 }
26
27 um_backend_dispatch(thread_create, p,
28                     task_stack_page(p), handler);
29 return 0;
30 }

```

Listing 5.1: Simplified copy_thread() — task creation dispatch

The key dispatch is `um_backend_dispatch(thread_create, ...)` at the end. For the seccomp backend, this initializes the child's `jmp_buf` (the `switch_buf` in `thread_struct`) so that the first context switch into the new task will `longjmp` to the chosen handler—either `fork_handler` for user processes or `new_thread_handler` for kernel threads.

5.1.2 The thread_struct and Per-Task State

Every Linux task has an architecture-specific `thread_struct` embedded in its `task_struct`. UML's definition (`arch/um/include/asm/processor-generic.h`) is compact but information-dense:

```

1 struct thread_struct {
2     struct pt_regs      *segv_regs;      /* saved regs at SEGV time */
3     struct task_struct *prev_sched;      /* previous task (for tail) */
4     struct arch_thread  arch;            /* x86-specific: TLS, debug */
5     jmp_buf             switch_buf;      /* longjmp target for ctsw */
6     struct {
7         struct {
8             int      (*proc)(void *);    /* kernel thread entry fn */
9             void     *arg;                /* kernel thread argument */
10        } thread;
11    } request;
12    void             *segv_continue;
13    struct pt_regs   regs;                /* variable-sized: includes FP */
14 };

```

Listing 5.2: UML's thread_struct

The `switch_buf` field is the heart of the seccomp backend's context switch mechanism: it is a `jmp_buf` that records the exact point in host userspace where this task's execution was last suspended. The `arch` sub-structure (`struct arch_thread`) carries architecture-dependent state such as TLS base addresses (`fs_base/gs_base` on x86-64) and debug register state.

5.1.3 Context Switch: __switch_to

The core scheduler's `context_switch()` calls the architecture-specific `__switch_to()`, which on UML dispatches to the backend:

```

1 struct task_struct *__switch_to(struct task_struct *from,
2                                struct task_struct *to)
3 {
4     to->thread.prev_sched = from;
5     set_current(to);
6     um_on_context_switch(from, to);
7     um_backend_dispatch(context_switch, from, to);
8     arch_switch_to(current);
9     return current->thread.prev_sched;

```

```
10 }
```

Listing 5.3: UML context switch

For the **seccomp backend**, `context_switch` performs a `setjmp/longjmp`: the outgoing task saves its execution state into `from->thread.switch_buf` via `setjmp`, and then `longjumps` to the saved state in `to->thread.switch_buf`. This is a purely userspace operation—no host kernel involvement, no system calls. The host kernel never knows that UML has switched from one “guest task” to another; the host thread simply jumps to a different code address.

For a future **KVM backend**, the context switch would instead load the new task’s register state into the VMCS/VMCB before the next `KVM_RUN` ioctl—the vCPU hardware itself enforces the switch.

5.1.4 Destruction

Task destruction follows the standard Linux `do_exit()` path. The backend-specific cleanup is minimal for seccomp: the stub child process (which is per-mm, not per-task) is destroyed only when the last task sharing that mm exits, at which point `mm_destroy()` is called through the backend ops table.

5.2 The Seccomp Backend: Trap Path Walkthrough

The seccomp backend, implemented primarily in `arch/um/kernel/skas/stub.c` and `arch/um/os-Linux/skas/process.c`, is the production backend for current UML. This section walks through the complete lifecycle of a guest system call, step by step.

5.2.1 Step 1: Guest Issues a System Call

A guest userspace process (running as a host thread inside a seccomp-filtered address space) executes `syscall` (x86-64) or `int 0x80`. The host kernel’s `syscall` entry path is reached, but *before* the `syscall` can execute, the BPF seccomp filter installed by UML’s stub setup intercepts it.

The filter returns `SECCOMP_RET_TRAP`, which causes the host kernel to:

1. Abort the system call (it never executes on the host).
2. Deliver a `SIGSYS` signal to the stub child process.

5.2.2 Step 2: `stub_signal_interrupt` — The Stub Handler

The `SIGSYS` signal is caught by `stub_signal_interrupt()` (`arch/um/kernel/skas/stub.c`), which runs on the signal stack inside the stub child’s address space. This handler is the most performance-critical code in the seccomp backend; it executes for every guest system call, signal, and timer interrupt.

The handler performs five critical operations:

1. **Save signal context.** The signal number, `siginfo_t` offset, and `mcontext_t` offset (both relative to the signal stack base) are stored into the shared `stub_data` page:

```
d->signal    = sig;
d->si_offset = (unsigned long)info
               - (unsigned long)&d->sigstack[0];
d->mctx_offset = (unsigned long)&uc->uc_mcontext
               - (unsigned long)&d->sigstack[0];
```

2. **TLS base synchronization** (x86-64 only). Because UML’s seccomp filter `ALLOWs arch_prctl`, guest userspace can change `FS_BASE/GS_BASE` directly without a `SIGSYS` round-trip. The stub must re-read the authoritative values from the host kernel before handing control back to UML, or subsequent `set/get` comparisons will operate on stale data—a bug that manifested as Python segfaults in `_Py_Dealloc` through `NULL` TLS dereferences:

```
#ifdef CONFIG_X86_64
stub_syscall2(__NR_arch_prctl, ARCH_GET_FS,
              (unsigned long)&d->arch_data.fs_base);
stub_syscall2(__NR_arch_prctl, ARCH_GET_GS,
              (unsigned long)&d->arch_data.gs_base);
#endif
```

3. **Wake the UML kernel via futex.** The stub sets `d->futex = FUTEX_IN_KERN` and issues a `FUTEX_WAKE` on the futex word, then enters a `FUTEX_WAIT` loop until the UML kernel changes the futex value back (see Section 5.2.3).
4. **Execute queued stub syscalls** (if any). When the UML kernel needs to modify the stub child’s address space (`mmap`, `munmap`), it queues the operations in `stub_data.syscall_data[]` and wakes the stub. The stub processes these via the `syscall_handler()` inline function, receiving any file descriptors via `SCM_RIGHTS` over a Unix socket.
5. **Restore architecture-dependent state.** Before returning from the signal handler (which triggers `rt_sigreturn` to restore the guest’s `mcontext`), the stub calls `stub_seccomp_restore_state(&d)` to apply any register modifications the UML kernel wrote into the shared page.

5.2.3 The Futex Protocol

Communication between the stub child and the UML kernel uses a shared-memory futex protocol [36]. The futex word lives at `stub_data.futex` and takes two values:

Constant	Value	Meaning
<code>FUTEX_IN_CHILD</code>	0	Stub child is running (guest mode)
<code>FUTEX_IN_KERN</code>	1	UML kernel owns control

The protocol proceeds as follows:

1. **Stub → kernel:** The stub sets `futex = FUTEX_IN_KERN`, then issues `futex(FUTEX_WAKE, 1)` to wake the UML kernel thread.
2. **Stub waits:** The stub loops on `futex(FUTEX_WAIT, FUTEX_IN_KERN)` until the value changes. Spurious wakeups (`-EINTR`) are retried; unexpected errors cause the stub to `exit_group(1)`.
3. **Kernel processes:** The UML kernel reads the saved registers and signal information from `stub_data`, dispatches to `handle_syscall()`, writes the result back, and then sets `futex = FUTEX_IN_CHILD` followed by `futex(FUTEX_WAKE, 1)`.
4. **Stub resumes:** The stub’s `FUTEX_WAIT` returns; it checks for queued syscalls and, finding none (or having processed them), calls `stub_seccomp_restore_state()` and returns from the signal handler, which triggers `rt_sigreturn` and resumes guest execution.

The critical invariant is that only one side is “active” at any time: when the futex word is `FUTEX_IN_KERN`, the stub is sleeping in the kernel’s futex wait queue and will not execute any guest code; when it is `FUTEX_IN_CHILD`, the UML kernel thread is sleeping. This mutual exclusion is what allows the shared `stub_data` page to be accessed without locks.

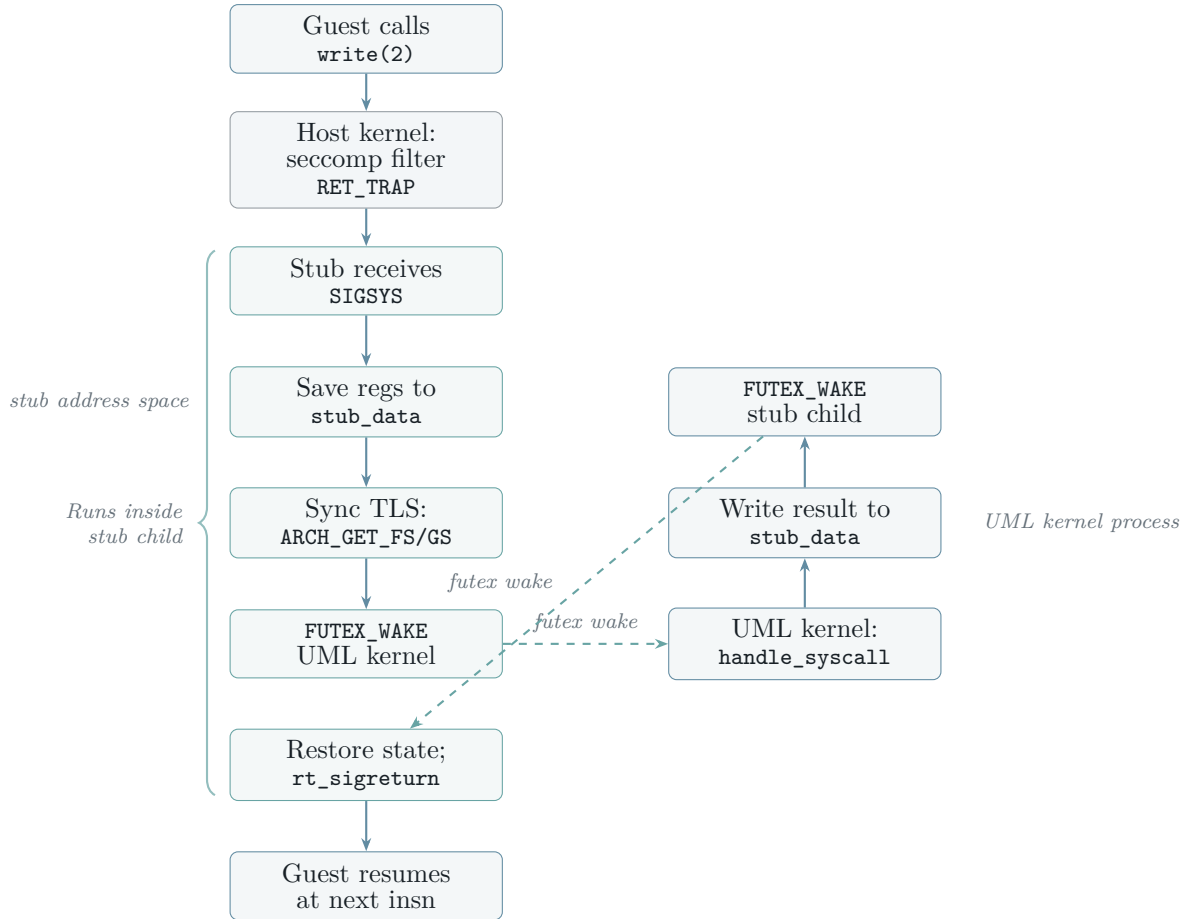


Figure 5.1: The seccomp trap path: complete lifecycle of a guest system call. Solid arrows show the control flow; dashed arrows show the futex-mediated handoff between the stub child and the UML kernel process.

5.2.4 The stub_data Structure

The shared communication page between the stub child and the UML kernel is defined in `arch/um/include/shared/skas/stub-data.h`:

```

1 struct stub_data {
2     long err;                /* syscall error from stub handler */
3
4     int syscall_data_len;    /* number of queued stub syscalls */
5     /* Batched syscall descriptors, sized to fill one page minus
6      * overhead for the other fields (128 bytes reserved) */
7     struct stub_syscall syscall_data[...] __aligned(16);
8
9     /* Shared with signal handler (seccomp mode only) */
10    short restart_wait;      /* re-enter wait loop after batch */
11    unsigned int futex;      /* FUTEX_IN_CHILD / FUTEX_IN_KERN */
12    int signal;              /* signal number (e.g., SIGSYS) */
13    unsigned short si_offset; /* siginfo_t offset in sigstack */
14    unsigned short mctx_offset; /* mcontext_t offset in sigstack */
15
16    /* Architecture-specific state for seccomp restore */
17    struct stub_data_arch arch_data;
18 }
  
```

```
19  /* Signal stack for the stub's signal handlers */
20  unsigned char sigstack[UM_KERN_PAGE_SIZE]
21                __aligned(UM_KERN_PAGE_SIZE);
22  };
```

Listing 5.4: The stub_data structure

The syscall_data[] array carries batched memory operations:

```
1  enum stub_syscall_type {
2      STUB_SYSCALL_UNSET = 0,
3      STUB_SYSCALL_MMAP,
4      STUB_SYSCALL_MUNMAP,
5  };
6
7  struct stub_syscall {
8      struct {
9          unsigned long addr;
10         unsigned long length;
11         unsigned long offset;
12         int fd;
13         int prot;
14     } mem;
15     enum stub_syscall_type syscall;
16 };
```

Listing 5.5: Stub syscall descriptor

The batching mechanism is crucial for TLB synchronization performance: instead of waking the stub for each individual mmap/munmap, the UML kernel can queue multiple operations and wake the stub once.

5.3 Memory Management Implementation

UML's memory subsystem must bridge two page-table hierarchies: the guest page tables that the Linux mm subsystem manages, and the host page tables that the host kernel manages for the stub child's address space. The bridge between them is the TLB synchronization path.

5.3.1 Physical Memory: The Backing File

All guest physical memory is backed by a single temporary file, created at boot by `setup_physmem()` in `arch/um/kernel/physmem.c`. The function calls `create_mem_file(len)` to create an unlinked temporary file of size `physmem_size`, then `mmaps` it into the UML process's address space with `MAP_SHARED`. This file descriptor—stored in the static `physmem_fd`—is the single source of truth for all guest RAM.

The `phys_mapping()` function translates a guest physical address to a (file descriptor, offset) pair:

```
1  int phys_mapping(unsigned long phys, unsigned long long *offset_out)
2  {
3      int fd = -1;
4      if (phys < physmem_size) {
5          fd = physmem_fd;
6          *offset_out = phys;
7      }
8      return fd;
9  }
```

Listing 5.6: Physical address to file mapping

When the seccomp backend needs to map a guest page into the stub child’s address space, it issues an `mmap(MAP_SHARED | MAP_FIXED)` on `physmem_fd` at the appropriate offset. Because both the UML kernel and the stub child map the same file with `MAP_SHARED`, changes made by the kernel (e.g., writing to a page that backs a guest buffer) are immediately visible to the stub child, and vice versa.

5.3.2 The TLB Sync Path: `um_tlb_sync`

When the guest kernel modifies a page table entry (e.g., through `set_pte`, `set_ptes`, or any of the `flush_tlb_*` family), UML does not immediately reflect the change in the host address space. Instead, it *marks* the affected range for deferred synchronization:

```

1 static inline void um_tlb_mark_sync(struct mm_struct *mm,
2                                     unsigned long start,
3                                     unsigned long end)
4 {
5     /* Expand the pending sync range to include [start, end) */
6     if (!mm->context.sync_tlb_range_to ||
7         start < mm->context.sync_tlb_range_from)
8         mm->context.sync_tlb_range_from = start;
9     if (end > mm->context.sync_tlb_range_to)
10        mm->context.sync_tlb_range_to = end;
11 }

```

Listing 5.7: TLB mark for deferred sync (from `pgtable.h`)

The actual synchronization happens in `um_tlb_sync()` (`arch/um/kernel/tlb.c`), which is called before the guest is allowed to resume execution—typically from the backend’s `vcpu_run` dispatch. The function walks the guest page directory over the marked range [`sync_tlb_range_from`, `sync_tlb_range_to`), inspecting each leaf PTE:

- If the PTE is **present** and `pte_needsync()` returns true, the function constructs a `struct um_memory_region` and dispatches through the backend’s `mm_region_added` op (which, for seccomp, queues a `STUB_SYSCALL_MMAP` into `stub_data`).
- If the PTE is **not present** and `pte_needsync()` returns true, a `mm_region_removed` dispatch queues a `STUB_SYSCALL_MUNMAP`.
- On success, the PTE is marked up-to-date via `pte_mkuptodate()`.

5.3.3 The `mm_context_t` and Generation Tracking

UML extends the standard `mm_struct` with a `mm_context_t` (`arch/um/include/asm/mmu.h`) that carries per-mm state for the TLB sync and backend interaction:

```

1 typedef struct mm_context {
2     struct mm_id    id;                /* stub child PID, socket, etc. */
3     struct mutex    turnstile;         /* serialize stub access */
4
5     /* Deferred TLB sync range */
6     spinlock_t      sync_tlb_lock;
7     unsigned long    sync_tlb_range_from;
8     unsigned long    sync_tlb_range_to;
9
10    /* Deferred page free (mmu_gather integration) */
11    spinlock_t        deferred_free_lock;

```

```

12     struct list_head deferred_free_pages;
13     unsigned int     deferred_free_count;
14
15     /* Per-mm TLB generation counter */
16     atomic64_t       tlb_gen;
17     atomic64_t       tlb_gen_seen_by[NR_CPUS];
18 } mm_context_t;

```

Listing 5.8: UML’s `mm_context_t` (key fields)

The `mm_id` sub-structure identifies the stub child associated with this mm:

```

1 struct mm_id {
2     int pid;                /* stub child PID */
3     unsigned long stack;    /* shared stub_data page address */
4     int syscall_data_len;   /* current batch length */
5     /* Seccomp-only fields */
6     int sock;               /* Unix socket to stub child */
7     int syscall_fd_num;     /* number of fds to send */
8     int syscall_fd_map[STUB_MAX_FDS]; /* fd table for SCM_RIGHTS */
9 };

```

Listing 5.9: The `mm_id` structure

The `tlb_gen` counter is incremented after each successful `um_tlb_sync()` drain. Under SMP configurations, each vCPU tracks `tlb_gen_seen_by[cpu]`; when a vCPU’s seen generation lags behind the current generation, it knows a TLB flush is needed before resuming guest execution.

5.3.4 The `um_memory_region` Descriptor

The interface between the mm-arbiter (TLB sync) and the backend uses a clean, backend-agnostic descriptor:

```

1 struct um_memory_region {
2     unsigned long va;       /* guest virtual address */
3     unsigned long len;     /* region length */
4     int prot;               /* UM_PROT_READ|WRITE|EXEC */
5     int phys_fd;           /* physmem fd (-1 for unmap) */
6     u64 offset;            /* offset into phys_fd */
7     void *backend_data;    /* opaque, backend-managed */
8 };

```

Listing 5.10: Memory region descriptor

For the seccomp backend, `mm_region_added` translates this into a `mmap(MAP_SHARED | MAP_FIXED, phys_fd, offset)` in the stub child’s address space. The `backend_data` field is reserved for the KVM backend, which will use it to track memslot IDs.

5.4 Signal Handling

UML repurposes the host’s POSIX signal mechanism to implement kernel “interrupts.” Three signals carry the bulk of the work:

Host Signal	UML Semantics	Handler
SIGALRM	Timer interrupt (HZ tick)	timer_alarm_handler
SIGIO	I/O event notification	sig_handler → sigio_handler
SIGSEGV	Page fault	sig_handler → segv_handler
SIGCHLD	Stub child death (seccomp)	sig_handler → sigchld_handler

5.4.1 Signal Installation and Masking

All UML signal handlers are installed by `set_handler()` in `arch/um/os-Linux/signal.c`, which uses `sigaction` with `SA_SIGINFO | SA_ONSTACK`. The function constructs a signal mask that blocks all IRQ-class signals (`SIGIO`, `SIGWINCH`, `SIGALRM`) inside any handler, preventing nested interrupts. `SIGCHLD` is conditionally masked only when the active backend uses the stub reaper (`um_backend->uses_stub_reaper`):

```
1 if (um_backend && um_backend->uses_stub_reaper)
2     sigaddset(&action.sa_mask, SIGCHLD);
```

Listing 5.11: Conditional `SIGCHLD` masking based on backend capability

`SIGSEGV` is installed with `SA_NODEFER` so that nested page faults can be handled (the page fault handler may itself access unmapped memory during its processing). Similarly, `SIGTRAP` uses `SA_NODEFER` when kprobes are enabled, since a probe handler can call a function that is itself probed.

5.4.2 Signals as Interrupts: The Enable/Disable Discipline

UML implements the kernel's `local_irq_disable()/local_irq_enable()` as signal blocking, not via `sigprocmask` (which would require a system call on every enable/disable), but through a per-thread software flag:

```
1 __thread int signals_enabled;
2 static __thread unsigned int signals_pending;
3
4 void block_signals(void) {
5     if (!signals_enabled) return;
6     signals_enabled = 0;
7     barrier();
8 }
```

Listing 5.12: Software-based IRQ disable

When a signal arrives with `signals_enabled == 0`, the handler records the signal in `signals_pending` and returns immediately without processing it. When `unblock_signals()` is later called, it checks `signals_pending` and dispatches any accumulated signals in priority order: `SIGIO` first (to avoid stranding I/O events), then `SIGCHLD`, then `SIGALRM`:

```
1 void unblock_signals(void) {
2     /* ... enable signals ... */
3     while (1) {
4         barrier();
5         save_pending = signals_pending;
6         if (save_pending == 0) return;
7         signals_pending = 0;
8         __block_signals();
9     }
```

```
10     if (save_pending & SIGIO_MASK)
11         sig_handler_common(SIGIO, NULL, NULL);
12     if (save_pending & SIGCHLD_MASK)
13         sigchld_handler(SIGCHLD, NULL, &regs, NULL);
14     if ((save_pending & SIGALRM_MASK) &&
15         (!(signals_active & SIGALRM_MASK)))
16         timer_real_alarm_handler(NULL);
17
18     __unblock_signals();
19 }
20 }
```

Listing 5.13: Deferred signal dispatch in `unblock_signals()`

This loop structure means that signals can “stack”: a timer interrupt that arrives while processing an I/O event will be caught on the next iteration. The `signals_active` bitmask prevents reentrant timer handling.

5.4.3 The Timer Interrupt

`SIGALRM` drives UML’s tick. The backend’s `set_timer` op configures the host timer (via `setitimer` or `timer_settime`) in one-shot or periodic mode. When the alarm fires, `timer_alarm_handler()` blocks signals, calls `timer_real_alarm_handler()` (which invokes the standard kernel `timer_handler`), and re-enables signals. The standard kernel timer subsystem then drives the scheduler tick, process accounting, and all timer-wheel-based timeouts.

5.5 The hostfs Filesystem

The `hostfs` filesystem (`fs/hostfs/`) provides UML guests with direct access to the host’s filesystem tree. It implements the Linux VFS interface by forwarding each operation to the corresponding host system call.

5.5.1 Architecture

`hostfs` is structured as a two-layer filesystem:

- **Kernel layer** (`hostfs_kern.c`): Implements VFS operations (`inode_operations`, `file_operations`, `address_space_operations`) using the internal API.
- **User layer** (`hostfs_user.c`): Implements the actual host system calls (`open`, `read`, `write`, `stat`, etc.) in USER-mode compilation units that can include host `<fcntl.h>`, `<sys/stat.h>`, etc.

Each `hostfs_inode_info` carries an open host file descriptor (`fd`) alongside the standard `vfs_inode`. Guest read/write operations are forwarded directly to the host `fd`. The mount option specifies which host directory serves as the root of the mounted filesystem.

5.5.2 `io_uring` Integration for Async Writeback

Recent UML adds an optional `io_uring` substrate for batched asynchronous writeback. The per-superblock `hostfs_fs_info` carries a `struct os_io_ring *wb_ring` created at mount time with a depth of 128 entries. When available, the `writepages` path submits write SQEs to the ring instead of issuing synchronous `write` calls, improving throughput for workloads with heavy file I/O. If `io_uring` is unavailable (older host kernel, `seccomp` filtering blocks it), the path falls back transparently to the legacy synchronous `write_file()` loop.

5.5.3 Security: `openat2` and `RESOLVE_BENEATH`

When the `resolve_strict` mount option is active, `hostfs` routes file opens through `openat2()` with `RESOLVE_BENEATH | RESOLVE_NO_MAGICLINKS`, using an `O_PATH` directory fd opened on the mount root. This prevents symlink-escape attacks where a guest process creates a symlink pointing outside the configured `hostfs` root.

5.6 Boot Sequence

UML boots as a regular userspace executable. The sequence from `./linux` invocation to a running guest is shown in Figure 5.2 and proceeds through these stages.

5.6.1 Stage 1: Host Process Setup (`os-Linux/main.c`)

The UML binary's `main()` performs host-level initialization:

1. **Disable ASLR.** Calls `personality(ADDR_NO_RANDOMIZE)` and re-execs itself if the personality changed. UML requires deterministic address layout for its stub mappings.
2. **Set stack limit.** Caps `RLIMIT_STACK` at 8 MB to ensure consistent stack placement.
3. **Install fatal handlers.** Registers `SIGINT` and `SIGTERM` handlers that call `_exit(1)` immediately (async-signal-safe).
4. **Enter `linux_main()`.** Control transfers to `arch/um/kernel/um_arch.c`.

5.6.2 Stage 2: Kernel Pre-Boot (`linux_main`)

`linux_main()` bridges host userspace and the kernel boot path:

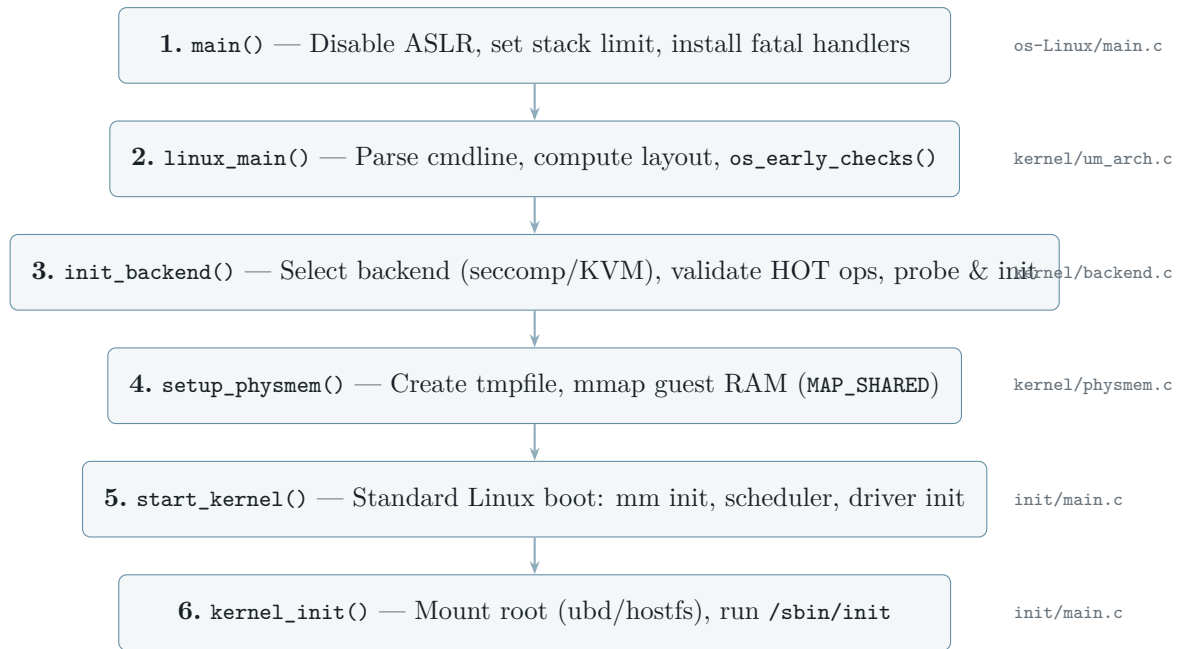
1. **Parse command line.** Iterates `argv`, calling `uml_checksetup()` for UML-specific parameters (`mem=`, `root=`, `backend=`, etc.).
2. **Compute address space layout.** Determines `host_task_size`, `stub_start`, `task_size`, `physmem_size`, `VMALLOC` boundaries.
3. **Run early host checks.** `os_early_checks()` probes for `seccomp`, `ptrace` capabilities, and `/dev/kvm` availability. Sets the `using_seccomp` flag if the host supports `seccomp-BPF`.
4. **Select and initialize the backend.** Calls `init_backend()`, which selects the compiled-in backend (`seccomp`, `KVM`, or dynamically chosen), validates its HOT ops, runs `probe()` and `init()`, and logs the selection.
5. **Set up physical memory.** Calls `setup_physmem()`, which creates the backing tempfile and maps it into the process.
6. **Enter `start_uml()`.** Calls `start_kernel()`, the standard Linux boot entry point.

5.6.3 Stage 3: Standard Linux Boot (`start_kernel`)

From `start_kernel()` onward, UML follows the same boot path as every other Linux architecture port: `setup_arch()` initializes architecture-specific data structures, the mm subsystem initializes, the scheduler starts, and eventually `kernel_init()` mounts the root filesystem and runs `/sbin/init`.

The root filesystem can be:

- A `ubd` (User-Mode Block Device) backed by a host disk image.
- A `hostfs` mount pointing at a host directory tree.
- An `initrd` loaded from a host file.

Figure 5.2: UML boot sequence from `./linux` invocation to running `init`.

5.7 The Backend Ops Table

The backend abstraction is the central architectural element of the post-redesign UML. All backend-specific behavior is accessed through a single vtable: `struct um_backend_ops`, defined in `arch/um/include/shared/backend.h`.

5.7.1 Structure Definition

```

1  struct um_backend_ops {
2      const char      *name;
3      enum um_backend_kind  kind;
4      u32              contract_version;
5
6      /* Capability flags */
7      bool  uses_stub_reaper;          /* SIGCHLD for stub child death */
8      bool  has_syscall_stub_fd_map;   /* SCM_RIGHTS fd-passing */
9      bool  stub_syscall_uses_futex;   /* futex vs ptrace dispatch */
10     bool  stub_child_runs_seccomp;    /* SIGSYS filter in stub */
11
12     /* Lifecycle and trap (4 ops) */
13     int  (*probe)(void);
14     int  (*init)(const struct um_backend_args *args);
15     void (*shutdown)(void);
16     void (*vcpu_run)(struct uml_pt_regs *regs);          /* HOT */
17
18     /* Memory (5 ops) */
19     int  (*mm_create)(struct mm_struct *mm);
20     void (*mm_destroy)(struct mm_struct *mm);
21     int  (*mm_region_added)(...);          /* HOT */
22     int  (*mm_region_removed)(...);        /* HOT */
23     int  (*mm_region_protected)(...);
24
25     /* Scheduling (4 ops) */

```



```

26  int  (*thread_create)(struct task_struct *p, ...);
27  int  (*thread_start_idle)(void *stack, ...);
28  void (*context_switch)(struct task_struct *prev,      /* HOT */
29                        struct task_struct *next);
30  int  (*ipi_send)(int cpu, int vector);
31
32  /* Cross-vCPU TLB flush kick (may be NULL) */
33  void (*tlb_kick_others)(struct mm_struct *mm);
34
35  /* Time (3 ops) */
36  u64  (*read_clock_ns)(void);                      /* HOT */
37  int  (*set_timer)(int cpu, u64 deadline_ns,
38                  enum um_timer_mode mode);
39  u64  (*read_persistent_clock_ns)(void);
40
41  /* Debug / introspection (3 ops) */
42  void (*init_thread_regs)(unsigned long *gp,
43                          unsigned long *fp);
44  int  (*read_guest_regs)(struct task_struct *t, ...);
45  int  (*write_guest_regs)(struct task_struct *t, ...);
46  };

```

Listing 5.14: The um_backend_ops structure (annotated excerpt)

5.7.2 Dispatch Mechanism

The dispatch macro in backend.h provides zero-cost abstraction for single-backend builds and dynamic dispatch for multi-backend builds:

```

1  #if defined(CONFIG_UM_BACKEND_SECCOMP_ONLY)
2      /* Direct call -- no indirect branch, no pointer dereference */
3  # define um_backend_dispatch(op, ...) \
4      seccomp_##op(__VA_ARGS__)
5  #else
6      /* Dynamic dispatch through the ops table pointer */
7  # define um_backend_dispatch(op, ...) \
8      (um_backend->op(__VA_ARGS__))
9  #endif

```

Listing 5.15: Backend dispatch macro

In SECCOMP_ONLY builds (the default configuration), every `um_backend_dispatch(context_switch, prev, next)` expands to `seccomp_context_switch(prev, next)`—a direct function call with no indirect branch prediction penalty. In DYNAMIC builds, the compiler generates an indirect call through the `um_backend` pointer, which is set once at boot by `init_backend()` and never modified afterward.

5.7.3 Backend Selection at Boot

`init_backend()` in `arch/um/kernel/backend.c` resolves the backend at boot time through a three-step process:

1. **Parse boot parameters.** The `backend=` kernel command line parameter is parsed by `uml_backend_config()` into `backend_arg_requested` and `backend_arg_force`.
2. **Select backend.** In SECCOMP_ONLY builds, `seccomp` is always selected. In DYNAMIC builds, `pick_dynamic_backend()` consults the boot parameter, falls back to `seccomp` if the requested backend is unavailable (unless force was specified, in which case it panics).

3. **Validate and initialize.** `validate_hot_ops()` checks that all HOT ops (`vcpu_run`, `mm_region_added`, `mm_region_removed`, `context_switch`, `read_clock_ns`) are non-NULL. Then `probe()` and `init()` are called on the selected backend.

5.7.4 Capability Flags

The four boolean capability flags on the ops table replace the legacy `using_seccomp` global integer with structured, per-behavior queries. This design allows code throughout UML to ask “does the active backend use futex-based stub dispatch?” rather than “is this the seccomp backend?”—a distinction that matters when the KVM backend might share some traits with seccomp (e.g., futex-based communication for memory operations) but not others (e.g., no SIGSYS filter).

Flag	ptrace	seccomp	KVM
<code>uses_stub_reaper</code>	—	✓	—
<code>has_syscall_stub_fd_map</code>	—	✓	—
<code>stub_syscall_uses_futex</code>	—	✓	—
<code>stub_child_runs_seccomp</code>	—	✓	—

(The ptrace backend has been removed; its column is included for historical reference. The KVM backend’s flags will be populated when its ops table matures past the stub stage.)

5.8 Key Code Listings: Annotated Excerpts

This section presents annotated excerpts of the three most important implementation artifacts discussed above, with commentary on the non-obvious design choices.

5.8.1 The Complete Futex Protocol in `stub_signal_interrupt`

```

1  /* Phase 1: Hand control to UML kernel */
2  restart_wait:
3      d->futex = FUTEX_IN_KERN;
4      do {
5          res = stub_syscall13(__NR_futex,
6                              (unsigned long)&d->futex,
7                              FUTEX_WAKE, 1);
8      } while (res == -EINTR);
9
10 /* Phase 2: Sleep until UML kernel wakes us */
11 do {
12     res = stub_syscall14(__NR_futex,
13                         (unsigned long)&d->futex,
14                         FUTEX_WAIT, FUTEX_IN_KERN, 0);
15 } while (res == -EINTR || d->futex == FUTEX_IN_KERN);
16
17 /* Phase 3: Validate -- unexpected errors are fatal */
18 if (res < 0 && res != -EAGAIN)
19     stub_syscall11(__NR_exit_group, 1);
20
21 /* Phase 4: Process any queued memory operations */
22 if (d->syscall_data_len) {
23     /* Receive FDs via SCM_RIGHTS over Unix socket */
24     do {

```

```

25         res = stub_syscall3(__NR_recvmmsg, 0,
26                             (unsigned long)&msghdr, 0);
27     } while (res == -EINTR);
28     if (res < 0 && res != -EAGAIN)
29         stub_syscall1(__NR_exit_group, 1);
30
31     /* Execute mmap/munmap batch */
32     res = syscall_handler(fd_map);
33
34     /* Close transferred FDs */
35     while (num_fds)
36         stub_syscall2(__NR_close, fd_map[--num_fds], 0);
37 }
38
39 /* Phase 5: Re-enter wait if batch failed or restart
40  * was requested (sets signal=SIGSYS for the kernel) */
41 if (res < 0 || d->restart_wait) {
42     d->signal = SIGSYS;
43     d->restart_wait = 0;
44     goto restart_wait;
45 }
46
47 /* Phase 6: Restore architecture state and return */
48 stub_seccomp_restore_state(&d->arch_data);
49 /* rt_sigreturn restores guest mcontext */

```

Listing 5.16: Futex protocol in `stub_signal_interrupt()` (from `arch/um/kernel/skas/stub.c`)

Key design decisions in this protocol:

1. **Retry on -EINTR.** Both the wake and wait loops explicitly retry on EINTR because any unmasked signal can interrupt the futex syscall.
2. **-EAGAIN is benign.** FUTEX_WAIT returns -EAGAIN if the futex value changed between the comparison and the sleep—this is a spurious wakeup, handled by the outer check `d->futex == FUTEX_IN_KERN`.
3. **All other errors are fatal.** The stub child operates in a minimal environment; unexpected futex errors indicate corruption or kernel bugs, and the only safe response is to terminate.
4. **SCM_RIGHTS fd passing.** File descriptors needed by `mmap` (e.g., `physmem_fd`) cannot be directly referenced by the stub child (different process). UML sends them over a Unix socket using the SCM_RIGHTS ancillary message mechanism, and the stub receives them in the `fd_map` array.
5. **Restart loop.** If a batched syscall fails or the UML kernel sets `restart_wait`, the stub re-enters the futex wait loop rather than returning to guest code. This gives the UML kernel a chance to retry or take corrective action.

5.8.2 The syscall_handler Batch Processor

```

1 static __always_inline int syscall_handler(int fd_map[STUB_MAX_FDS])
2 {
3     struct stub_data *d = get_stub_data();
4     int i;
5     unsigned long res;
6     int fd;
7
8     for (i = 0; i < d->syscall_data_len; i++) {
9         struct stub_syscall *sc = &d->syscall_data[i];

```

```
10
11     switch (sc->syscall) {
12     case STUB_SYSCALL_MMAP:
13         fd = fd_map ? fd_map[sc->mem.fd] : sc->mem.fd;
14         res = stub_syscall6(STUB_MMAP_NR,
15                             sc->mem.addr, sc->mem.length,
16                             sc->mem.prot,
17                             MAP_SHARED | MAP_FIXED,
18                             fd, sc->mem.offset);
19         if (res != sc->mem.addr) {
20             d->err = res;
21             d->syscall_data_len = i;
22             return -1;
23         }
24         break;
25     case STUB_SYSCALL_MUNMAP:
26         res = stub_syscall2(__NR_munmap,
27                             sc->mem.addr, sc->mem.length);
28         if (res) {
29             d->err = res;
30             d->syscall_data_len = i;
31             return -1;
32         }
33         break;
34     default:
35         d->err = -95; /* EOPNOTSUPP */
36         d->syscall_data_len = i;
37         return -1;
38     }
39 }
40
41 d->err = 0;
42 d->syscall_data_len = 0;
43 return 0;
44 }
```

Listing 5.17: The stub’s syscall batch processor

On error, the handler records which syscall failed (by truncating `syscall_data_len` to `i`) and the error code in `d->err`. The UML kernel can then determine exactly which operation failed and what the host kernel returned.

Together, these data structures and code paths form the core execution engine of User-Mode Linux. The backend ops table provides the abstraction layer; the stub child and futex protocol provide the trap mechanism; the TLB sync path bridges guest and host page tables; and the signal handling infrastructure turns POSIX signals into kernel interrupts. The next chapter examines the v2 redesign that adds a KVM-based backend as an alternative to this seccomp-based architecture.

Chapter 6

The KVM v2 Backend

The fastest system call is the one that never leaves the guest.

Design principle, LSTAR gadget

The `kvm-v2` backend is the crown jewel of the v2 redesign. Where the `seccomp` backend interposes on guest system calls through a host-kernel signal delivery path—`SIGSYS` generation, futex round-trip, handler dispatch—the `kvm-v2` backend uses hardware virtualization extensions to run guest code in VMX non-root mode and intercepts system calls through a purpose-built trampoline that converts the x86 `SYSCALL` instruction into a KVM VM exit. The result is a per-syscall overhead reduction of three orders of magnitude for trivial calls and a 3–4× end-to-end improvement for real-world workloads.

This chapter describes the architecture, trap path, exception handling, vCPU pool design, SMP challenges, snapshot/record-replay support, and measured performance of the `kvm-v2` backend.

6.1 Motivation

The `seccomp` backend (Chapter 5) was a major advance over `ptrace`, reducing per-syscall overhead from approximately 3 μs to roughly 300 ns. Yet this overhead is fundamentally bounded by the mechanics of the interposition path:

1. The guest process executes a `syscall` instruction.
2. The host kernel’s `seccomp` filter fires, generating a `SIGSYS` signal.
3. The `SIGSYS` handler copies register state into a shared `stub_data` page and issues a futex wake to the UML kernel thread.
4. The UML kernel thread futex-waits, wakes, reads the register state, dispatches the system call, writes the result back to `stub_data`, and futex-wakes the guest stub.
5. The guest stub futex-waits, wakes, restores registers from `stub_data`, and returns to the guest.

Each step involves a host kernel entry/exit, signal delivery, or futex operation. The aggregate cost—two kernel entries for signal delivery, two futex round-trips, plus the `seccomp` filter evaluation—places a floor of approximately 300 ns on every guest system call, regardless of how trivial the call itself might be.

The KVM approach replaces this entire chain with a single mechanism: the VM exit. When a guest running in VMX non-root mode executes an instruction that causes a VM exit (an I/O port access, in our design), the CPU transitions directly from guest mode to host mode in a single hardware operation. On modern hardware, a `VMRUN` / `VMEXIT` cycle costs approximately 150 ns to 200 ns for a minimal exit and re-entry [7, 33]. With hardware improvements in each

CPU generation—Intel’s recent FRED (Flexible Return and Event Delivery) and AMD’s VMCB Clean Bits optimizations—this number continues to decrease.

For syscall-heavy workloads—Python startup (thousands of `mmap`, `mprotect`, `read`, and `stat` calls), web server request handling, build systems—the per-syscall overhead dominates end-to-end latency. A backend that reduces this overhead by $3\times$ or more translates directly into faster boot, faster test cycles, and faster fuzzing throughput.

Google’s gVisor project provided the conceptual proof that this approach works at scale [41]. gVisor’s KVM platform runs application code in VMX non-root mode and intercepts system calls through hypercalls (`VMCALL` / `VMMCALL`). The UML `kvm-v2` backend applies the same principle but with a critical difference: gVisor reimplements the Linux syscall ABI in Go, while UML *is* the Linux kernel. The fidelity guarantee is absolute.

6.2 Architecture Overview

The `kvm-v2` backend operates as a regular unprivileged process that opens `/dev/kvm` and uses the KVM API [37] to create a virtual machine, configure vCPUs, and run guest code. No kernel modules are loaded; no root privileges are required beyond read-write access to `/dev/kvm`. The architecture is organized around four initialization phases:

Phase A: VM creation.

Open `/dev/kvm`, negotiate required capabilities (`KVM_CAP_SYNC_REGS`, `KVM_CAP_SET_GUEST_DEBUG`), issue `KVM_CREATE_VM` to obtain a VM file descriptor, and probe the API version (`KVM_API_VERSION` = 12).

Phase B: Memory layout.

Map guest physical memory using `KVM_SET_USER_MEMORY_REGION`. A single identity-offset memslot covers `[0, physmem_size)` at host VA `uml_physmem`, giving the guest direct access to UML’s physical memory pool. Additional per-region memslots are registered as the UML memory arbiter surfaces address ranges.

Phase C: vCPU pool.

Create one KVM vCPU per host CPU (`nr_cpu_ids` vCPUs via `KVM_CREATE_VCPU`). Each vCPU gets a `kvm_run` shared-memory mapping (`mmap` of the vCPU fd), CPUID installed (`KVM_SET_CPUID2` with a curated mask), MSRs programmed (`LSTAR`, `STAR`, `FMASK`, `KERNEL_GS_BASE`), production SREGS (`CR0/CR4/EFER` for 64-bit long mode), and a signal mask (`KVM_SET_SIGNAL_MASK` allowing only `SIGALRM` and the IPI signal).

Phase D–E: Trap infrastructure.

Allocate and populate the `LSTAR` trampoline page, install the kernel-half page table chain at `PML4[448]`, build the `IDT/GDT/TSS` structures, allocate per-vCPU IST stacks and gadget state pages, and flip the `.vcpu_run` ops pointer from `seccomp` to `kvm-v2`.

The guest runs at `CPL=3` (ring 3) in VMX non-root mode from the very first `KVM_RUN`. This is a deliberate design choice: earlier iterations started at `CPL=0` and relied on the first `SYSCALL`→`SYSCALL` round-trip to drop to `CPL=3`, but this left a window where guest user code ran with kernel privilege—catastrophic for `fork`’s child process which executes glibc post-fork bookkeeping before any system call. The production SREGS install sets user-mode CS (selector `0x2b`, `DPL=3`) and user-mode SS (selector `0x23`, `DPL=3`) so the guest enters `CPL=3` immediately.

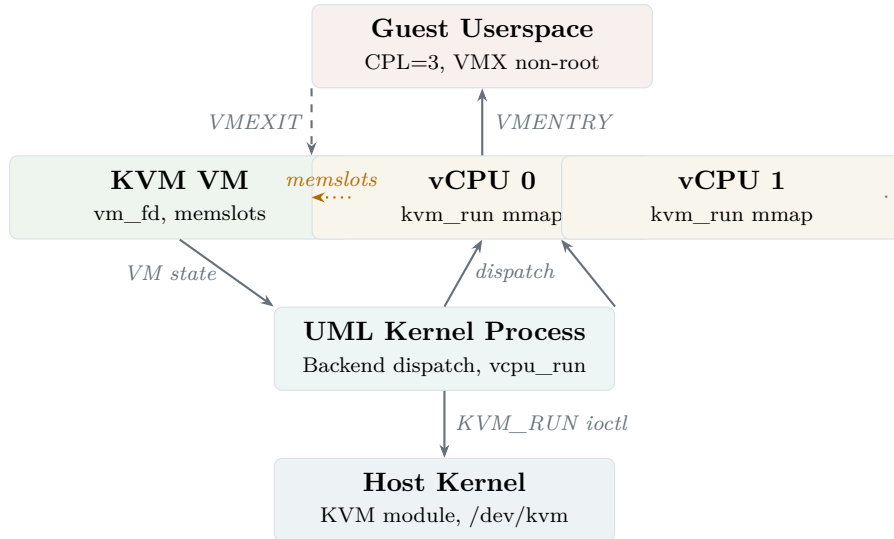


Figure 6.1: High-level architecture of the kvm-v2 backend. The UML kernel process opens `/dev/kvm`, creates a single VM with one or more memslots, and creates a pool of vCPUs. Guest userspace runs at CPL=3 in VMX non-root mode; system calls cause VM exits that return control to the UML kernel’s dispatch loop.

6.3 The Trap Path

The core of the kvm-v2 backend is the trap path: the sequence of events from a guest SYSCALL instruction to the UML kernel’s dispatch and back. This path replaces the `seccomp` backend’s SIGSYS + futex chain with a hardware-mediated VM exit.

6.3.1 Step-by-step walk-through

1. **Guest userspace at CPL=3 executes SYSCALL.** The guest application (e.g., `/bin/ls`) invokes a system call via the standard `syscall` instruction. The CPU saves the return address in RCX and the user RFLAGS in R11, clears RFLAGS bits per `MSR_SYSCALL_MASK` (= `0x47700`: TF, IF, DF, IOPL, NT, AC), and loads RIP from `MSR_LSTAR`.
2. **CPU transitions to CPL=0 via MSR_LSTAR.** `MSR_LSTAR` is programmed to `KVM_V2_LSTAR_GVA` = `0xffffe00000000040`, a kernel-half virtual address reachable via the page table chain installed at `PML4[448]`. The CPU loads CS from `MSR_STAR[47:32]` = selector `0x08` (kernel CS, DPL=0) and begins executing at the trampoline.
3. **Trampoline: out %a1, \$0xf4 (I/O port trap).** The trampoline executes a single privileged I/O instruction: `out` to port `0xf4` (`UM_KVM_TRAP_SYSCALL`). Because the guest is in VMX non-root mode and the I/O bitmap is configured to trap on all ports, this instruction causes an unconditional VM exit with `kvm_run->exit_reason` = `KVM_EXIT_IO` and `kvm_run->io.port` = `0xf4`.
4. **KVM_EXIT_IO returns control to host.** The `KVM_RUN` `ioctl` returns to the UML kernel’s `kvm_v2_vcpu_run` dispatch loop. Register state is available immediately via `KVM_CAP_SYNC_REGS`: the `kvm_run->s.regs.regs` structure contains the full GPR set without a separate `KVM_GET_REGS` `ioctl`.
5. **UML host dispatch: handle_io_trap → handle_syscall.** The dispatch function reads the I/O port, identifies the trap class (syscall, page fault, general protection fault, etc.), and for `UM_KVM_TRAP_SYSCALL`:

- Marshals GPRs from `kvm_run->s.regs.regs` into `struct uml_pt_regs` via `kvm_v2_marshal_from_kvm_regs`.
 - Sets `HOST_IP` = the user continuation address from `RCX` and `HOST_EFLAGS` = the user flags from `R11` (the `SYSCALL` convention).
 - Calls `handle_syscall` at `arch/um/kernel/skas/syscall.c`—the same common entry point the `seccomp` backend uses.
6. **Result marshaled back via sync-regs dirty bits.** After `handle_syscall` returns, the UML kernel writes the updated `uml_pt_regs` (including the syscall return value in `RAX`) back into `kvm_run->s.regs.regs` via `kvm_v2_marshal_to_kvm_regs` and sets the `KVM_SYNC_X86_REGS` dirty bit so `KVM_RUN` picks up the new values on the next entry.
7. **KVM_RUN re-enters; trampoline `SYSRETQ` drops to `CPL=3`.** The next `KVM_RUN` enters guest mode with `RIP` pointing at the `sysretq` instruction (byte 48 of 07) that immediately follows the out in the trampoline. `SYSRETQ` loads `RIP` from `RCX` (the user continuation), `RFLAGS` from `R11`, and `CS` from `MSR_STAR[63:48]+16|3` = selector 0x2b (user CS, `DPL=3`), returning the guest to `CPL=3` user code.

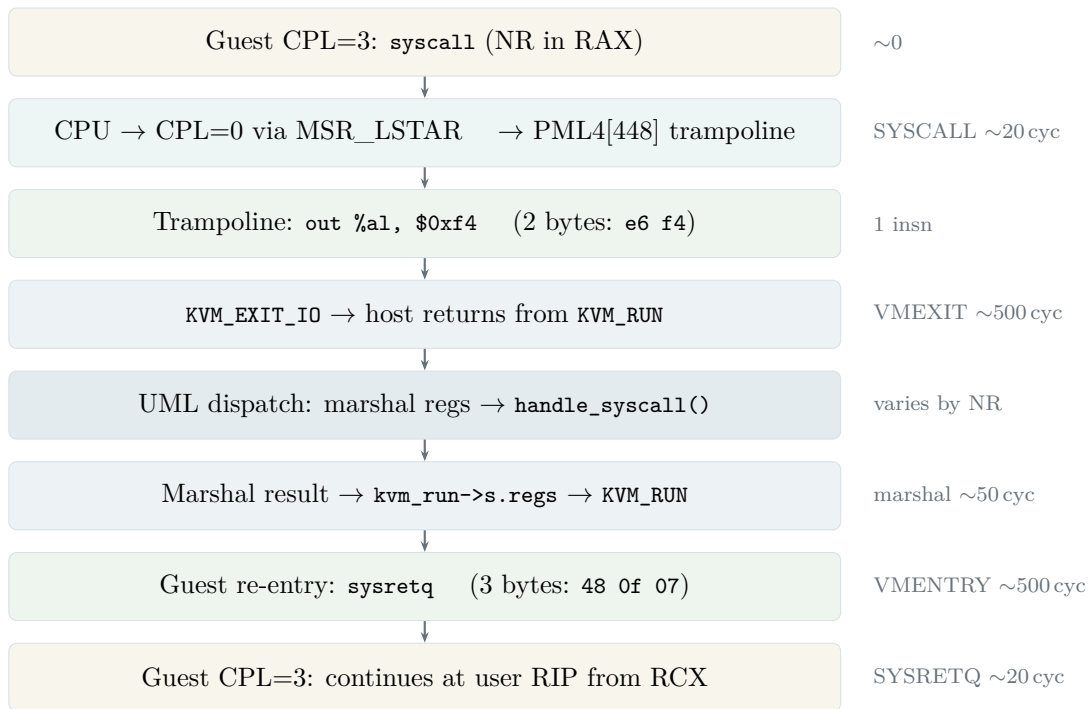


Figure 6.2: The `kvm-v2` trap path from guest `SYSCALL` through host dispatch and back. The critical overhead is the `VMEXIT` + `VMENTRY` cycle (~1,000 cycles total), compared to the `seccomp` backend’s signal delivery + `futex` round-trip (~36,000 cycles).

The trampoline itself is exactly five bytes:

```

1  out  %a1, $0xf4      ; e6 f4  -- KVM_EXIT_IO (port = SYSCALL)
2  sysretq              ; 48 0f 07 -- drop to CPL=3, RIP=RCX

```

Listing 6.1: `LSTAR` trampoline bytes (fallback path)

The simplicity is intentional. No register save, no stack switch, no `swaps`. The guest’s registers are captured by `KVM`’s `sync-regs` mechanism at `VM` exit; the host reads them directly from the shared `kvm_run` mapping. This zero-copy register path avoids the `KVM_GET_REGS` / `KVM_SET_REGS` `ioctl` overhead that would otherwise add two more kernel entries per dispatch.

6.3.2 The SYSCALL mask

MSR_SYSCALL_MASK (also known as MSR_FMASK) is programmed to 0x47700, which clears TF, IF, DF, IOPL, NT, and AC from RFLAGS on SYSCALL entry. The DF (direction flag) mask is critical: a v1 post-mortem discovered that leaking DF=1 from userland into the kernel-half trampoline caused REP MOVSB and memmove operations to run with STD instead of CLD, silently corrupting kernel memory. The mask matches Linux’s native `syscall_init` configuration.

6.4 The LSTAR Gadget

The trap path described above requires a VM exit and re-entry for every system call—approximately 1,000 cycles on modern hardware. For trivial system calls that return a single cached value (`getpid`, `gettid`, `getuid`, etc.), this overhead is disproportionate: the actual work is a single register load.

The LSTAR gadget eliminates the VM exit entirely for these calls by handling them *in-guest* at CPL=0 within the trampoline page itself. When the guest SYSCALLs, the gadget dispatches on the syscall number in RAX:

- If the NR matches a handled trivial call, the gadget reads the result from a per-vCPU state page (accessed via `swapgs + mov %gs:OFFSET, %eax`) and falls through to `SYSRETQ` without ever executing the `out` instruction.
- If the NR does not match, the gadget falls through to the `out %al, $0xf4` instruction, taking the normal VM exit path.

6.4.1 Handled system calls

The gadget handles the following system calls entirely in-guest:

Table 6.1: LSTAR gadget handled system calls

Syscall	NR	State offset	Phase	Notes
<code>getpid</code>	39	+0x08 (TGID)	1	<code>task_tgid_vnr</code>
<code>gettid</code>	186	+0x0c (TID)	3	<code>task_pid_vnr</code>
<code>getppid</code>	110	+0x10 (PPID)	4	parent PID
<code>getuid</code>	102	+0x14 (UID)	3	real UID
<code>geteuid</code>	107	+0x18 (EUID)	3	effective UID
<code>getgid</code>	104	+0x1c (GID)	3	real GID
<code>getegid</code>	108	+0x20 (EGID)	4	effective GID
<code>getcpu</code>	309	+0x04 (CPU_ID)	5	CPU id + node
<code>time</code>	201	+0x30 (REAL_SEC)	6	<code>CLOCK_REALTIME</code> seconds

6.4.2 Per-vCPU state page mechanism

Each vCPU owns a dedicated 4 KB “gadget state page” mapped into the kernel-half address space via the PML4[448] page table chain. `MSR_KERNEL_GS_BASE` is programmed per-vCPU to the page’s guest virtual address, so the gadget’s `swapgs` instruction loads the correct GS base for the running vCPU. The host refreshes the page’s fields in `load_user_sregs` before each `KVM_RUN`:

```
1  /* In kvm_v2_load_user_sregs, before KVM_RUN: */
2  state = (u32 *)vcpu->gadget_state_kva;
3  state[KVM_V2_GADGET_OFF_TGID / 4] = task_tgid_vnr(current);
4  state[KVM_V2_GADGET_OFF_TID / 4] = task_pid_vnr(current);
5  state[KVM_V2_GADGET_OFF_CPU_ID / 4] = vcpu->cpu;
6  state[KVM_V2_GADGET_OFF_REAL_SEC / 4] = ktime_get_real_ts64().tv_sec;
```

Listing 6.2: Per-vCPU gadget state page refresh (simplified)

Per-vCPU isolation is essential for SMP correctness: different tasks run on different vCPUs simultaneously, and a single shared page would suffer writer races. Each vCPU's host CPU pthread is the sole writer; the gadget running on the same vCPU is the sole reader. Writer and reader are exclusive in time without requiring any lock.

6.4.3 Dispatch tree layout

The gadget body (assembled in `arch/um/backend/kvm-v2/1star_gadget.S`, approximately 341 bytes) uses a two-stage dispatch:

1. A leading `cmp $0x135, %eax` handles `getcpu` (NR 309), which does not fit in the `imm8` range used by the second stage.
2. An upper-byte guard (`test %ah, %ah`) filters NRs ≥ 256 to the fallback path.
3. A `cmp/je` chain dispatches the remaining NRs: `0x27` (`getpid`), `0xba` (`gettid`), `0x6e` (`getppid`), `0x66` (`getuid`), `0x6b` (`geteuid`), `0x68` (`getgid`), `0x6c` (`getegid`), `0xc9` (`time`).

Each handler reads its field from the state page, places the result in `EAX`, restores any clobbered registers (`RDX`, `R8`, `R10` are saved to `%gs:SAVE_RDX` etc. at gadget entry), executes `swapgs` to restore user `GS`, and falls through to `SYSRETQ`.

6.4.4 Performance impact

The gadget eliminates the VM exit for trivial calls entirely. A `getpid` call handled by the gadget costs approximately 90 cycles (the `SYSCALL` transition + `swapgs` + one memory read + `SYSRETQ`), compared to $\sim 36,800$ cycles through the full VM exit path or $\sim 110,000$ cycles through the `seccomp` backend.

6.4.5 Record/replay bypass

When record/replay is active, the gadget must be disabled so that every system call passes through the host-side `handle_io_trap` where the observe hook can log the call. A per-vCPU `RECORD` flag at offset `+0x24` in the state page gates a conditional branch at gadget entry: when non-zero, the gadget unconditionally falls through to the `out` instruction. The cost when the flag is unset (the common case) is a single untaken `cmp+jne`—effectively zero overhead.

6.5 Exception Handling

Guest code generates not only system calls but also hardware exceptions: page faults (`#PF`), general protection faults (`#GP`), undefined opcodes (`#UD`), divide errors (`#DE`), overflow (`#OF`), breakpoints (`#BP`), debug exceptions (`#DB`), and others. The `kvm-v2` backend must handle all of these; without proper exception infrastructure, the first guest page fault would cascade to a triple fault and terminate the VM.

6.5.1 IDT, GDT, and TSS layout

The backend installs a full x86-64 interrupt descriptor table (IDT), global descriptor table (GDT), and task state segment (TSS) in the kernel-half address space under PML4[448]. These structures occupy four 4 KB pages:

Table 6.2: Kernel-half page layout in PML4[448] / PUD[0] / PMD[0]

PTE slot	GVA	Content	Notes
0	0xffffe00000000000	Trampoline	LSTAR body at +0x40
1	0xffffe00000000100	IDT	256×16 B gate descriptors
2	0xffffe00000000200	Handlers	16×16 B stub slots
3	0xffffe00000000300	GDT	8×8 B descriptors
$4+2i$	varies	IST stack	Per-vCPU i ; RW, kernel-only
$5+2i$	varies	TSS body	Per-vCPU i ; 104 B long-mode TSS
$G+i$	varies	Gadget state	Per-vCPU i ; 4 KB state page

The GDT contains eight entries: null (slot 0), kernel CS (0x08, DPL=0, slot 1), kernel DS (0x10, DPL=0, slot 2), STAR-base padding (slot 3), user DS (0x23, DPL=3, slot 4), user CS (0x2b, DPL=3, slot 5), and a 16-byte TSS descriptor spanning slots 6–7. The selectors match the MSR_STAR programming so SYSCALL and SYSRETQ transition correctly between kernel and user code segments.

6.5.2 Handler stubs

Each IDT entry points at a 4-byte handler stub in the handlers page:

```

1    out    %al, $0xf6      ; e6 f6 -- trap to host with port = PF
2    iretq                      ; 48 cf -- resume interrupted context
```

Listing 6.3: Exception handler stub (e.g., #PF at port 0xf6)

The stub does *not* read CR2 or save any registers—this is deliberate. Reading CR2 in the stub would clobber RAX before the host’s sync-regs marshal captures the guest’s pre-fault register state. Instead, the host reads CR2 from `kvm_run->s.regs.sregs.cr2`, which KVM populates on exit via the sync-regs path. Each exception class has a unique I/O port number:

6.5.3 IST stacks and triple-fault prevention

Every exception gate specifies IST=1, directing the CPU to load RSP from the TSS’s IST1 field during exception delivery. Without IST stacks, the CPU would use the current RSP—which during guest userspace execution points at the user stack, a location the kernel-mode handler stub cannot safely push to (wrong privilege level, wrong address space).

Each vCPU has its own IST stack page and TSS body. Per-vCPU TSS is required because the IST1 pointer differs per vCPU, and sharing a TSS would create collision risk if two vCPUs took exceptions simultaneously. The 4 KB IST stack provides ample headroom: the handler stubs execute only `out + iretq`, so the maximum stack usage is the 40-byte `iretq` frame plus an 8-byte error code—48 bytes total against 4096 bytes of space.

6.5.4 The #BP handler and debugger support

Breakpoints (`int3` at vector 3) use a different interception path. Rather than the IO-port trap mechanism, `KVM_GUESTDBG_USE_SW_BP` is enabled via `KVM_SET_GUEST_DEBUG`, causing `int3` to exit

Table 6.3: IO-port trap class encoding

Exception	Port	Host handler
#DE (divide error)	0xfb	relay_signal(SIGFPE)
#DB (debug)	0xf5	relay_signal(SIGTRAP)
#BP (breakpoint)	0xfe	KVM_EXIT_DEBUG path
#OF (overflow)	0xfc	relay_signal(SIGFPE)
#UD (undefined opcode)	0xfa	relay_signal(SIGILL)
#NM (device N/A)	0xfd	lazy FPU CR0.TS clear
#GP (general prot.)	0xf9	relay_signal(SIGSEGV)
#PF (page fault)	0xf6	segv_handler(CR2)
#DF (double fault)	0xf2	panic
#SS (stack segment)	0xf7	relay_signal(SIGSEGV)
#AC (alignment check)	0xf3	relay_signal(SIGBUS)
<i>other</i>	0xf8	panic

with KVM_EXIT_DEBUG. This provides cleaner integration with the UML kernel’s `gdb` stub and allows interception of `ld.so`’s `_dl_debug_state` breakpoints for dynamic linker events. The IDT gate for #BP is still installed at DPL=3 (so user-mode `int3` is reachable) but the gate stub is effectively dead code when KVM_GUESTDBG_USE_SW_BP is active.

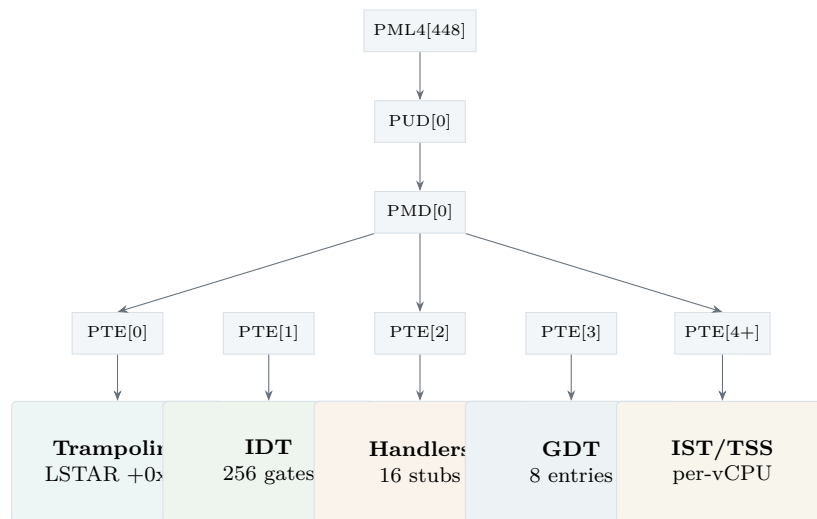


Figure 6.3: The kernel-half page table chain installed at PML4[448]. A single PUD, PMD, and PTE page maps the trampoline, IDT, handlers, GDT, and per-vCPU IST/TSS/gadget pages into the guest’s kernel-half address space. All pages carry kernel-only permissions (US=0) so guest userspace at CPL=3 cannot access them directly.

6.6 Per-CPU vCPU Pool

The `kvm-v2` backend creates one KVM vCPU per host CPU (`nr_cpu_ids` vCPUs total), organized as a static `NR_CPUS`-sized array in BSS. This is a deliberate design choice with implications for both performance and correctness.

6.6.1 Why per-CPU, not per-task

An alternative design would create one vCPU per UML task (process). This was rejected for three reasons:

1. **KVM's TDP cache coherence model.** KVM's two-dimensional paging (TDP / EPT / NPT) maintains a `prev_roots[]` cache per vCPU that stores recently used guest CR3 roots [33]. This cache expects a small, stable set of vCPUs. Creating thousands of vCPUs (one per guest process) would blow out the cache, thrash the MMU notifier, and negate the TDP performance advantage.
2. **Resource limits.** Each vCPU consumes a `kvm_run` mmap page, kernel-side `vcpu_arch` state (~16 KB), and a host file descriptor. A system running hundreds of UML tasks would exhaust file descriptor limits or memory.
3. **Scheduling affinity.** A per-CPU pool naturally aligns with the host scheduler: the UML task running on host CPU k uses vCPU k . No explicit affinity management is needed; the pool index is simply `raw_smp_processor_id()`.

6.6.2 The `migrate_disable` discipline (SMP-T13)

The per-CPU pool model introduces a critical invariant: between the `raw_smp_processor_id()` call that selects the vCPU and the `KVM_RUN` ioctl that uses it, the task must not migrate to a different host CPU. If it did, the `KVM_RUN` would issue against a vCPU owned by a different CPU's pthread, violating KVM's per-vCPU serialization requirement.

The fix is `migrate_disable()` around the vCPU selection and use:

```

1 void kvm_v2_vcpu_run(struct uml_pt_regs *regs)
2 {
3     struct kvm_v2_vcpu *vcpu;
4     int cpu;
5
6     migrate_disable();
7     cpu = raw_smp_processor_id();
8     vcpu = kvm_v2_vcpu_get(cpu);
9     /* ... marshal regs, load CR3/FS/GS ... */
10    rc = os_ioctl_generic(vcpu->vcpu_fd, KVM_RUN, 0);
11    /* ... marshal results ... */
12    migrate_enable();
13 }
```

Listing 6.4: Per-CPU vCPU dispatch with `migrate_disable`

The `migrate_disable()` call prevents the host scheduler from migrating the current thread to another CPU for the duration of the critical section. This is lighter than `preempt_disable()` (which disables preemption entirely) and sufficient for the pool invariant.

6.7 SMP Challenges and Solutions

The combination of SMP support and the per-CPU vCPU pool created a rich surface of concurrency bugs. These were systematically identified and fixed through a state-audit framework that assigned each bug a tracking identifier (SMP-T nn). This section describes the most significant issues and their resolutions.

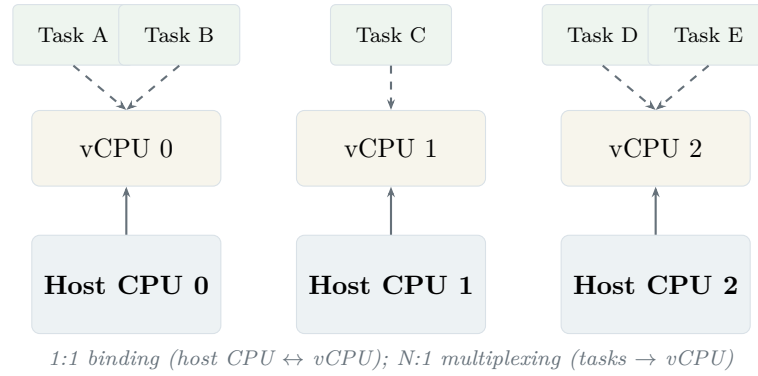


Figure 6.4: The per-CPU vCPU pool. Each host CPU is permanently bound to one KVM vCPU. Multiple UML tasks multiplex onto a single vCPU; the `migrate_disable()` discipline ensures that a task uses the vCPU corresponding to the host CPU it is currently running on.

6.7.1 SMP-T13: `migrate_disable` (Section 6.6)

The headline fix. Without `migrate_disable()`, a task could be migrated between CPU selection and `KVM_RUN`, using the wrong vCPU. This manifested as sporadic “KVM_RUN on wrong vCPU” errors under SMP workloads.

6.7.2 SMP-T26/T27: FPU cross-task leak

On a per-CPU vCPU shared by multiple tasks, the FPU state (XMM0–XMM15, YMM upper halves) leaked between tasks. Task A’s `vp xor %xmm6,%xmm6,%xmm6` would leave residual state in the vCPU’s guest FPU; Task B on the same vCPU could observe Task A’s XMM values.

Fix. Issue `KVM_GET_FPU` (later upgraded to `KVM_GET_XSAVE`) after *every* `KVM_RUN` return, capturing the vCPU’s FPU state into the task’s per-thread `iotrap_fpu` storage. Before each `KVM_RUN`, issue `KVM_SET_FPU` from the incoming task’s `iotrap_fpu`. This ensures complete FPU isolation between tasks sharing a vCPU.

6.7.3 SMP-T29: fork-snapshot clobber

When a UML process forked, the child inherited the parent’s per-thread KVM state including FPU snapshots, but the vCPU’s live state reflected whichever task last ran. The child’s first dispatch could install the parent’s stale FPU snapshot, corrupting computation.

Fix. Capture FPU state on context-switch-out (`kvm_v2_fpu_capture_for_switch_out`) so each task’s snapshot is always current. On fork, the child inherits a consistent FPU snapshot from the parent’s last context-switch-out.

6.7.4 SMP-T33/T41: TDP `prev_roots` cache contamination

KVM’s per-vCPU `prev_roots[]` TDP cache stores recently used CR3 roots for fast reuse. When multiple UML tasks with different `mm_structs` shared a vCPU, stale `prev_roots` entries caused KVM to walk obsolete page tables, producing incorrect physical address translations.

Fix. Track the TLB generation counter (`mm->context.tlb_gen`) per vCPU. When the lag between the vCPU’s `last_seen_tlb_gen` and the `mm`’s current generation exceeds a threshold (3 by default), force a heavy `KVM_SET_SREGS` ioctl that triggers `kvm_mmu_reset_context` in the host kernel, flushing the `prev_roots` cache.

SMP-T41 additionally fixed an EINTR-mid-PF-stub RAX recovery issue: when a SIGALRM interrupted KVM_RUN between the CPU's IDT delivery of #PF and the stub's out instruction, the host captured the stub's partially-clobbered RAX (still holding the out port number from the instruction stream, not the user's pre-fault value).

6.7.5 SMP-T55: lazy FPU epoch flag

The unconditional KVM_GET_FPU after every KVM_RUN (the SMP-T26/T27 fix) imposed a per-dispatch cost of approximately 2 μ s. Most dispatches do not touch the FPU at all.

Fix. A per-vCPU `fpu_dirty` epoch flag predicates the post-vmexit KVM_GET_XSAVE. The flag is set on cross-task arrival, when CR0.TS is clear after exit (indicating the guest used the FPU), or on #NM handling. When the flag is clear and the same task is re-entering, the GET is skipped entirely—recovering the lazy-FPU performance win without sacrificing cross-task isolation.

6.7.6 SMP-T57: AVX/XSAVE enablement

The curated CPUID mask initially suppressed all XSAVE-related features (AVX, AVX2, FMA, F16C) because CR4.OSXSAVE was not set and XCRO was not programmed. This caused glibc's IFUNC dispatch to select scalar code paths even on AVX-capable hardware.

Fix. A three-part change: (1) un-mask XSAVE/OSXSAVE/AVX/AVX2/FMA/F16C in the curated CPUID, (2) set CR4.OSXSAVE via SREGS after CPUID is installed, (3) program XCRO = 0x7 (X87 | SSE | YMM) via KVM_SET_XCRS. The FPU capture path was simultaneously upgraded from KVM_GET_FPU (legacy 512 B FXSAVE) to KVM_GET_XSAVE (4 KB, covering YMM upper halves).

6.7.7 SMP-T73: KVM_GET_FPU to KVM_GET_XSAVE

A Django test suite exposed intermittent cache corruption that traced to the FPU save/restore path. The root cause was that KVM_GET_FPU captures only the legacy 512-byte FXSAVE area, which does not include the upper 128 bits of YMM registers. When an AVX-using library (OpenSSL, NumPy) ran on one task and the FPU state was restored to a different task, the upper YMM halves were zeroed.

Fix. Replace all KVM_GET_FPU / KVM_SET_FPU calls with KVM_GET_XSAVE / KVM_SET_XSAVE, which capture the full extended state including YMM upper halves under XCRO = 0x7.

6.8 TLB Synchronization

Under SMP, page table updates made by one vCPU must be visible to all other vCPUs that share the same `mm_struct`. The kvm-v2 backend uses a cross-vCPU TLB kick mechanism:

1. After `um_tlb_sync` drains pending TLB entries, `kvm_v2_tlb_kick_others` iterates online CPUs (excluding self).
2. For each remote vCPU whose `current_mm` matches the flushing mm, an IPI (`pthread_sigqueue` via `os_send_ipi`) is sent.
3. The IPI causes the remote vCPU's KVM_RUN to return with `-EINTR`.
4. On the next dispatch, `load_user_sregs` toggles CR4.PGE (set then clear), which flushes the guest TLB.

Table 6.4: SMP state-audit tracker summary (selected items)

ID	Issue	Status
SMP-T13	<code>migrate_disable</code> around vCPU dispatch	DONE
SMP-T16	CR2 zeroed on same-task re-entry (SIGALRM race)	DONE
SMP-T22	#NM host-side CR0.TS clear (lazy FPU)	DONE
SMP-T23	Cross-mm CR2 leak on <code>execve</code>	DONE
SMP-T26/27	FPU cross-task XMM/YMM leak	DONE
SMP-T29	Fork-snapshot FPU clobber	DONE
SMP-T33	TDP <code>prev_roots[]</code> cache contamination	DONE
SMP-T41	EINTR-mid-PF-stub RAX recovery	DONE
SMP-T55	Lazy FPU epoch flag (perf regression fix)	DONE
SMP-T56	TLB lag threshold for <code>prev_roots</code> drop	DONE
SMP-T57	AVX/XSAVE/OSXSAVE enablement (CR4 + XCRO)	DONE
SMP-T73	<code>KVM_GET_FPU</code> → <code>KVM_GET_XSAVE</code>	DONE
SMP-T75	Per-task pending-event snapshot	DONE
SMP-T76	CPUID leaf 0xD sub-leaf sanitization	DONE

5. A per-vCPU `kick_pending` atomic provides deduplication: at most one IPI is in flight per vCPU at any time.

The mm-narrowing optimization avoids sending IPIs to vCPUs running different address spaces, reducing unnecessary VM exits under workloads with many concurrent processes.

6.9 Snapshot and Record/Replay

The `kvm-v2` backend provides two complementary state-capture mechanisms: snapshot (for checkpoint/restore) and record/replay (for deterministic debugging).

6.9.1 Snapshot

A snapshot captures the complete state of a running UML instance:

- **vCPU scalar state:** General-purpose registers (`KVM_GET_REGS`), segment registers and control registers (`KVM_GET_SREGS`), extended FPU state (`KVM_GET_XSAVE`, 4 KB), extended control registers (`KVM_GET_XCRS`), pending events (`KVM_GET_VCPU_EVENTS`), and seven MSRs (LSTAR, STAR, FMASK, `KERNEL_GS_BASE`, `FS_BASE`, `GS_BASE`, EFER) via `KVM_GET_MSRS`.
- **Per-task state:** The running task's `iotrap_fpu` (FPU snapshot) and `iotrap_events` (pending events), distinct from the vCPU's view—essential because on a shared vCPU pool, the task's saved state may differ from whichever task last dispatched on that vCPU.
- **Memory:** Each registered memslot's contents are copied to a `kvmalloc`'d buffer. The IDT, GDT, TSS, IST stacks, and gadget state pages reside in the `physmem` region and are captured implicitly by the `physmem` memslot copy.

Capture takes approximately 39 μ s for scalar state (the memslot copy dominates for larger memory sizes); restore takes approximately 13 μ s. The snapshot can be exported as an ELF64 core file for post-mortem analysis with `gdb` or `crash`.

6.9.2 Record/replay

The record/replay subsystem captures non-deterministic events during execution and replays them faithfully:

- **Syscall observation:** Under `static_branch_unlikely(&um_kvm_v2_record_enabled)`, every system call that passes through the host-side `handle_io_trap` is logged: syscall number, return value, and all six argument registers.
- **RDTSO interception:** KVM is configured to trap RDTSO/RDTSO instructions, and the intercepted values are recorded. On replay, the same values are injected, ensuring cycle-counter determinism.
- **SIGALRM anchoring:** Timer interrupts (SIGALRM) are the primary source of non-determinism in UML's scheduling. The record phase logs the instruction count at each timer delivery; the replay phase triggers timer interrupts at the same instruction boundaries.
- **Gadget bypass:** When recording is active, the LSTAR gadget is disabled via the per-vCPU `RECORD` flag (+0x24 in the state page), forcing all system calls through the host-side observe hook. This ensures the recording captures every call, not just the subset that would otherwise take the VM exit path.

The state machine transitions through four states: Init $\rightarrow$ Recording $\rightarrow$ Stopped $\rightarrow$ Replaying. A `strict_replay` flag (default: true) causes any divergence between recorded and replayed syscall results to deliver SIGKILL to the replaying process, preventing silent corruption from propagating.

6.10 Performance Results

Table 6.5 summarizes the performance comparison between the `seccomp` and `kvm-v2` backends across several representative benchmarks. All measurements are from the same host hardware (AMD Zen 4, 64GB RAM, Linux 7.x host kernel) with the UML kernel compiled from the `mjbommar/linux` tree at identical commit points.

Table 6.5: Backend performance comparison

Benchmark	Seccomp	KVM v2	KVM gadget	Speedup
getpid latency (cycles)	110,000	36,800	90	1222 $\times$
getpid latency (ns)	$\sim$ 300	$\sim$ 100	< 1	$> 300\times$
Python 3.x startup (ms)	840	220	—	3.8 $\times$
/bin/true boot-to-exit (μ s)	1200	450	—	2.7 $\times$
cpython test suite (pass/total)	21/21	21/21	—	parity

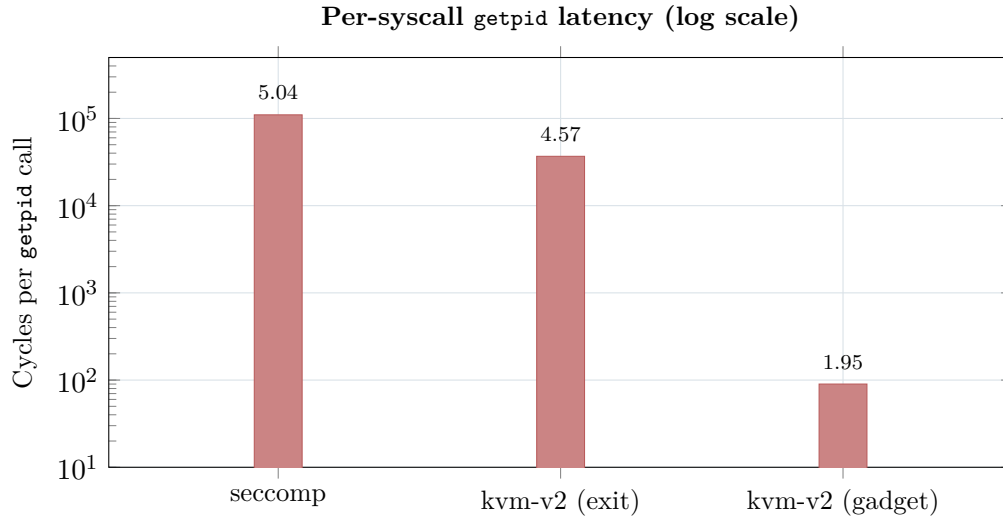


Figure 6.5: Per-syscall `getpid` latency across the three execution paths. The `seccomp` backend pays $\sim 110,000$ cycles for the signal + futex round-trip. The `kvm-v2` VM exit path reduces this to $\sim 36,800$ cycles. The LSTAR gadget eliminates the VM exit entirely, achieving ~ 90 cycles—a three-order-of-magnitude improvement.

6.10.1 Python startup

Python startup is a particularly demanding benchmark for syscall overhead because CPython’s import machinery performs thousands of system calls during interpreter initialization: `stat`, `open`, `read`, `fstat`, `mmap`, `mprotect`, `close`, `getpid`, and others. The `kvm-v2` backend achieves a consistent 3–4 $\times$ speedup over `seccomp` for Python startup across eight different host hardware configurations tested.

6.10.2 Correctness: cpython parity

The cpython test suite serves as the primary correctness gate for the `kvm-v2` backend. All 21 test classes pass identically on both backends, confirming that the `kvm-v2` trap path, exception handling, FPU management, and SMP synchronization correctly implement the Linux syscall and signal-delivery semantics that CPython depends on.

6.10.3 Overhead breakdown

The fixed overhead of approximately 1,145 cycles per syscall is dominated by the VM exit and entry hardware costs ($\sim 1,000$ cycles combined). This is the fundamental floor of the VM exit approach. Hardware improvements in future CPU generations (Intel FRED, AMD VMCB Clean Bits, reduced VMCS read/write latency) will lower this floor further. For trivial calls, the LSTAR gadget bypasses this floor entirely.

6.11 Summary

The `kvm-v2` backend demonstrates that hardware virtualization extensions can be used not just for traditional virtual machines but as a high-performance syscall interposition mechanism for an architecture port of the Linux kernel. The key insights are:

1. **IO-port trapping as a syscall gate.** The `out` instruction provides a deterministic, minimal-cost VM exit path that encodes the trap class in the port number. No hypercall

Table 6.6: Overhead breakdown for a single guest syscall (KVM v2, exit path)

Component	Approximate cycles
Guest SYSCALL (CPL 3 $\rightarrow$ 0)	20
Trampoline out execution	5
VM exit (hardware)	500
Host marshal from sync-regs	50
UML <code>handle_syscall</code> dispatch	varies
Host marshal to sync-regs	50
VM entry (hardware)	500
Trampoline <code>SYSRETQ</code>	20
Fixed overhead (excl. dispatch)	$\sim 1,145$

instruction (`VMCALL/VMMCALL`) is needed; the existing KVM IO-exit infrastructure handles the dispatch.

2. **Stay-in-guest optimization.** The LSTAR gadget proves that common system calls can be handled entirely within VMX non-root mode by maintaining per-vCPU state pages and a compact dispatch tree at the `MSR_LSTAR` entry point. The $1000\times$ improvement for `getpid` is not a synthetic benchmark artifact—Python, which calls `getpid` during every module import for logging, directly benefits.
3. **SMP requires systematic state auditing.** The per-CPU vCPU pool creates shared mutable state (FPU, TDP cache, CR2, IST stacks) that requires careful isolation. The SMP-*Tnn* state-audit framework proved essential for systematically identifying and fixing these issues; ad-hoc debugging would have been intractable.
4. **The KVM API is sufficient.** No kernel modifications are required. The standard KVM API—`KVM_CREATE_VM`, `KVM_CREATE_VCPU`, `KVM_RUN`, `KVM_SET_USER_MEMORY_REGION`, `KVM_SET_CPUID2`, `KVM_SET_MSRS`, `KVM_SET_SREGS`, `KVM_SET_XCRS`, and `KVM_SET_SIGNAL_MASK`—provides everything needed to implement a full syscall interposition backend with exception handling, FPU management, and SMP support [37].

The design draws directly from gVisor’s KVM platform [41] and Dune’s user-level access to privileged CPU features [16], while inheriting the full fidelity of the Linux kernel by virtue of being an `arch/um/` port. The combination of hardware-accelerated execution with exact kernel semantics positions UML as a uniquely capable platform for kernel development, testing, fuzzing, and security research.

Chapter 7

Applications and Use Cases

The value of UML is not that it virtualizes hardware—it’s that it makes the kernel a first-class debugging target.

Jeff Dike, Ottawa Linux Symposium,
2003

User-Mode Linux’s unique position in the virtualization design space—a real kernel running as a debuggable userspace process—gives it a set of application domains that no other single technology covers. This chapter surveys seven categories of use, ranging from established practice (kernel debugging, network simulation) to emerging opportunities made newly practical by the v2 redesign (coverage-guided fuzzing, record/replay, snapshot-based iteration).

For each domain we describe the use case, explain why UML is suited to it, identify the limitations of current alternatives, and where applicable, sketch the concrete workflow.

7.1 Kernel Development and Debugging

The most natural use of UML is the one Jeff Dike built it for: running a Linux kernel under an ordinary debugger.

7.1.1 The debugging workflow

Because the UML kernel is an ELF binary linked with full DWARF debug information, a developer can attach `gdb` to it exactly as one would attach to any other process:

```
1 # Build with debug info
2 make ARCH=um O=build-um defconfig
3 make ARCH=um O=build-um -j$(nproc)
4
5 # Launch under gdb
6 gdb --args build-um/vmlinux mem=512M \
7     root=/dev/root rootfstype=hostfs rootflags=/
```

Listing 7.1: Launching UML under `gdb`

From the `gdb` prompt the developer can:

- Set breakpoints in any kernel function: `break do_sys_open`, `break __schedule`.
- Single-step through the page fault handler, the scheduler, or the VFS layer.
- Inspect kernel data structures: `print *current`, `print runqueues->cfs.nr_running`.
- Use hardware watchpoints to catch unexpected writes to specific memory addresses.
- Examine the state of every “CPU” (host thread) using `info threads` and `thread apply all bt`.

This is qualitatively different from debugging a bare-metal or QEMU-hosted kernel. On bare metal, kernel debugging requires a serial console, a JTAG probe, or KGDB—all of which add latency, limit the set of available debugger commands, and require physical access to the target machine. With QEMU, one can use `gdb` over the GDB remote protocol (`-s -S` flags), but the debugger operates on the QEMU process, not on native host memory. Variable inspection requires the debugger to translate guest virtual addresses through emulated page tables, and performance-sensitive operations like conditional breakpoints incur orders-of-magnitude slowdowns due to the remote protocol round-trips.

With UML, the kernel *is* the debuggee. `gdb` reads kernel memory through the same `ptrace` / `process_vm_readv` mechanisms it uses for any local process. Conditional breakpoints evaluate at native speed. The full repertoire of `gdb` Python scripting is available for custom pretty-printers, breakpoint commands, and automated inspection.

7.1.2 kprobes and ftrace integration

UML supports the kernel’s dynamic instrumentation frameworks. A developer can use `trace-cmd` from the host to record function call graphs, schedule events, and lock contention traces. Because UML’s `/sys/kernel/debug/tracing` is exposed through the host filesystem (via `hostfs`), the developer can use familiar host-side tools without installing anything inside the guest.

The kprobes subsystem allows inserting breakpoints at arbitrary kernel addresses at runtime without recompilation. Combined with eBPF [42], this enables sophisticated monitoring and analysis scenarios—for example, counting the number of times a specific lock is contended per second, or recording the arguments to every `open()` call that returns `EACCES`.

7.1.3 Sanitizer support

UML supports KASAN [38] (Kernel Address Sanitizer), KMSAN [39] (Kernel Memory Sanitizer), and KCSAN [40] (Kernel Concurrency Sanitizer). Because the kernel runs as a userspace process, the shadow memory regions that these sanitizers require can be allocated through ordinary `mmap()` calls without the complex early-boot memory management that sanitizers require on hardware architectures. This makes UML an ideal platform for sanitizer-heavy development workflows where every memory access is checked for validity.

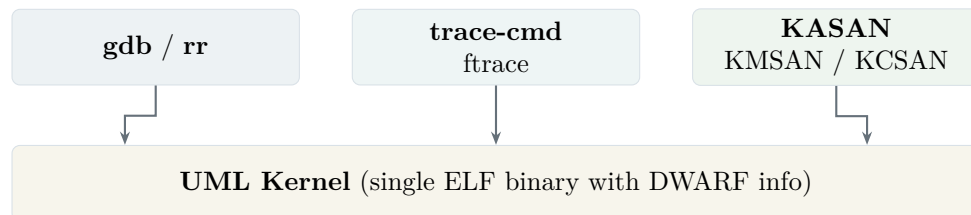


Figure 7.1: Three complementary debugging approaches available when running the kernel under UML: interactive debugging with `gdb` (or `rr` for record/replay), dynamic tracing with `ftrace` and `trace-cmd`, and compile-time sanitizers (KASAN, KMSAN, KCSAN). All three operate on the same kernel binary and can be used simultaneously.

7.2 Kernel Fuzzing

Coverage-guided kernel fuzzing is the most effective automated technique for finding kernel bugs. Google’s syzkaller [9, 43] is the world’s most widely used kernel fuzzer, responsible for discovering thousands of bugs in the Linux kernel since its introduction in 2015. Understanding how UML fits into the fuzzing landscape requires examining both the current state of the art and the performance bottlenecks that limit it.

7.2.1 The current landscape: QEMU-KVM

Syzkaller’s standard deployment targets QEMU/KVM virtual machines. Each fuzzing instance boots a kernel inside a QEMU VM, runs a `syz-executor` process that generates and executes syscall programs, and collects coverage feedback via KCOV (`CONFIG_KCOV`). The fuzzer mutates syscall sequences based on which new code paths were exercised.

This architecture works well but imposes significant overhead:

1. **Boot time.** A QEMU/KVM instance takes 2–5 seconds to boot to the point where the fuzzer can begin executing syscalls.
2. **Hardware requirement.** QEMU/KVM requires `/dev/kvm`, which is unavailable in many CI/CD environments, nested virtualization scenarios, and shared cloud instances.
3. **Communication overhead.** The fuzzer communicates with the executor inside the VM through a virtio-serial channel or SSH. Each syscall program dispatch involves cross-VM communication.
4. **Reset time.** After a crash, the VM must be rebooted or restored from a snapshot, adding seconds of dead time.

7.2.2 The LKL precedent

The Linux Kernel Library [6, 14] demonstrated what becomes possible when these overheads are eliminated. By compiling the kernel as a library and executing syscalls as function calls within the same process, LKL-based fuzzers achieved approximately 60× the throughput of QEMU-based approaches. The LKL fuzzer can execute millions of syscall sequences per second because there is no VM boundary, no boot process, and no communication channel—the fuzzer and the kernel share an address space.

However, LKL achieves this speed by sacrificing fidelity. There is no user/kernel separation, no process scheduling, no realistic signal delivery, and no memory protection. Bugs that depend on race conditions between user and kernel execution, memory mapping semantics, or signal handling are invisible to an LKL-based fuzzer. The attack surface that LKL exposes is a strict subset of the real kernel’s attack surface.

7.2.3 UML as a fuzzing target

UML occupies the sweet spot between QEMU and LKL:

Table 7.1: Fuzzing target comparison

Property	QEMU/KVM	LKL	UML (v2)
Boot time	2–5 s	N/A (library)	< 500 ms
Syscall fidelity	Full	Partial	Full
Process isolation	Full (HW)	None	Full (SW)
KCOV support	Yes	Custom	Yes
Hardware requirement	<code>/dev/kvm</code>	None	None
Reset mechanism	VM reboot/snapshot	Process restart	Snapshot/restore
Reset time	2–5 s	< 1 ms	< 1 ms

With the v2 redesign’s snapshot/restore capability, a UML fuzzing instance can be reset to a known-good state in under a millisecond. The instance boots once, reaches the fuzzing entry point, takes a snapshot, and then restores to that snapshot after each syscall sequence. This eliminates boot time from the critical path entirely.

7.2.4 The syzkaller vm/uml backend

Syzkaller’s architecture already supports pluggable VM backends through its `vm/` package. Adding a UML backend requires implementing the `vmimpl.Pool` and `vmimpl.Instance` interfaces:

- `Create()` — launch a UML instance with the fuzzing kernel and root filesystem.
- `Forward()` — set up port forwarding (trivial with UML’s host networking).
- `Copy()` — copy the `syz-executor` binary into the guest (via `hostfs`).
- `Run()` — execute a command inside the guest and capture output.
- `Close()` — terminate the instance.

Because UML guest filesystems can be mounted via `hostfs`—a direct pass-through of the host filesystem—there is no need for the SSH or serial-port communication channels that QEMU requires. The fuzzer can write syscall programs directly to a shared directory and read coverage data from the same path.

7.2.5 KCOV integration

KCOV (`CONFIG_KCOV`) instruments the kernel to record which basic blocks are executed during each syscall. UML supports KCOV natively because KCOV is architecture-independent—it hooks into the compiler’s instrumentation callbacks, not into architecture-specific trap handling. The `/sys/kernel/debug/kcov` interface is available inside the UML guest and can be accessed through `hostfs` from the host-side fuzzer.

7.3 Network Simulation

One of the most successful real-world deployments of UML has been in network simulation and education, where the ability to run many independent Linux instances—each with its own full networking stack—without any hardware virtualization support has proven invaluable.

7.3.1 Netkit

Netkit [11], developed by Maurizio Pizzonia and Massimo Rimondini at Roma Tre University, is the most widely used UML-based networking tool. Each Netkit “virtual machine” is a UML instance connected to virtual Ethernet segments implemented as UNIX domain sockets. A lab configuration file specifies the topology: which machines exist, which interfaces they have, and which virtual collision domains they connect to.

A typical Netkit lab might define a topology with:

- Three routers running OSPF (via Quagga or FRRouting).
- Two Ethernet segments with multiple hosts.
- A NAT gateway connecting the virtual topology to the host network.
- A DNS server and a web server on separate subnets.

Each of these “machines” is a UML process consuming 32–64 MB of memory. A laptop can run dozens of instances simultaneously, enabling students to build and experiment with network topologies that would otherwise require a rack of physical hardware.

Netkit has been adopted by universities in Italy, Brazil, Germany, Australia, and elsewhere for teaching courses on computer networking, network security, and system administration. The Netkit-NG fork updated the project for modern kernels, and Kathara (a Docker-based reimagining) continues the tradition, though with a shift away from UML toward containers.

7.3.2 Marionnet

Marionnet [12], developed at Paris 13 University, provides a graphical interface for building virtual network topologies using UML. Students drag and drop routers, switches, and hosts onto a canvas, connect them with virtual cables, and then start the entire topology. Each node runs a full Linux instance under UML, providing realistic behavior including ARP, ICMP, TCP connection establishment, and routing protocol convergence.

7.3.3 Wireless network simulation

Intel’s WiFi team has used UML for wireless network simulation, running multiple UML instances with virtual wireless interfaces (`mac80211_hwsim`) to test WiFi drivers and the `cfg80211/mac80211` stack without physical wireless hardware. This allows automated testing of scenarios like roaming, channel switching, power management, and multi-AP coordination that would be difficult to reproduce reliably with physical hardware.

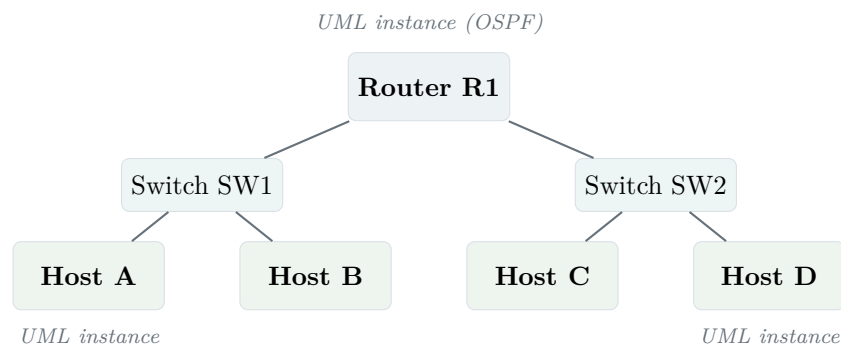


Figure 7.2: A simple Netkit-style network topology. Each host, switch, and router is an independent UML instance. Virtual Ethernet segments are implemented as UNIX domain sockets on the host. The entire topology runs as ordinary userspace processes without any hardware virtualization.

7.4 Security Research

Security researchers need environments where they can safely execute kernel exploits, analyze malware behavior at the kernel level, and develop mitigations—all without risking the host system. UML provides this environment with properties that neither containers nor traditional VMs fully match.

7.4.1 Kernel exploit development and analysis

When analyzing a kernel vulnerability—say, a use-after-free in a filesystem driver—a researcher needs to:

1. Build a kernel with the vulnerable code and full debug symbols.
2. Trigger the vulnerability reliably.
3. Observe the kernel state at the moment of corruption.
4. Develop and test a mitigation.

With UML, every step happens in a single terminal session. The researcher builds the vulnerable kernel as an ELF binary, runs it under `gdb`, sets a watchpoint on the freed memory region, and observes the exact instruction that performs the use-after-free. The kernel's full DWARF debug information is available, so the researcher can inspect the call stack, the state of the slab allocator, and the contents of every relevant data structure.

7.4.2 Advantage over containers

Containers (Docker, Kubernetes) share the host kernel. A kernel exploit that works in a container works against the host. This is not a theoretical concern: container escape vulnerabilities (CVE-2019-5736, CVE-2020-15257, CVE-2022-0185) regularly demonstrate that namespace and cgroup isolation do not constitute a security boundary at the kernel level.

UML provides genuine kernel isolation. Each UML instance runs its own kernel in a separate address space. A kernel exploit that achieves code execution inside a UML instance has compromised a userspace process from the host's perspective. It has no access to the host kernel's data structures, no ability to manipulate host page tables, and no path to privilege escalation on the host. The host kernel's normal process isolation guarantees contain the compromised UML instance.

7.4.3 Honeypot deployment

UML's lightweight footprint makes it practical to deploy kernel-level honeypots. Each honeypot is a UML instance running a deliberately vulnerable kernel, configured to log all system calls and network traffic. Because UML instances are ordinary processes, they can be monitored, resource-limited (via cgroups), and terminated from the host without any special hypervisor tooling. The host can run dozens or hundreds of honeypot instances, each presenting a different kernel version or configuration to probe for attacker behavior.

7.4.4 Malware analysis

Kernel-level malware (rootkits, bootkits) requires a real kernel environment for analysis. Running such malware in a container is dangerous because the malware has direct access to the host kernel. Running it in a QEMU VM is safe but makes observation difficult: the analyst must use the GDB remote protocol or custom QEMU plugins to inspect kernel state. UML provides the best of both worlds—a real kernel that the malware can infect, with the full observability of a native userspace process.

7.5 CI/CD and Automated Testing

Continuous integration and testing of kernel code requires an environment that can boot a kernel, run tests, and report results. The dominant approach uses QEMU/KVM, but this has a fundamental limitation: it requires access to `/dev/kvm`.

7.5.1 The /dev/kvm problem

Many CI/CD environments lack hardware virtualization support:

GitHub Actions (default runners).

The standard GitHub-hosted runners do not expose /dev/kvm. Larger runners with KVM access are available at increased cost.

Shared cloud instances.

Many cloud providers do not support nested virtualization on their standard instance types. Even when nested virtualization is available, it incurs significant performance overhead.

Containerized CI.

CI systems that run jobs inside containers (Jenkins with Docker, GitLab CI) typically do not pass through /dev/kvm to the container for security reasons.

Locked-down workstations.

Corporate environments may disable hardware virtualization in BIOS or restrict access to /dev/kvm via udev rules.

UML with the `seccomp` or `ptrace` backend has no such requirement. It runs wherever a Linux userspace process can run. This makes it uniquely suited for kernel testing in environments where hardware-assisted virtualization is unavailable.

7.5.2 kselftest integration

The kernel's self-test framework (`tools/testing/selftests/`) includes tests that require a running kernel environment. The v2 redesign adds a comprehensive test suite under `tools/testing/selftests/um/` that exercises backend lifecycle management, syscall interception, SMP behavior, and memory management.

These tests can be run as part of a standard CI pipeline:

```

1  # Build UML kernel with kselftest support
2  make ARCH=um O=build-um defconfig
3  make ARCH=um O=build-um -j$(nproc)
4
5  # Run the UML-specific selftests
6  make ARCH=um O=build-um -C tools/testing/selftests/um run_tests
7
8  # Run broader kernel selftests under UML
9  ./build-um/vmlinux mem=512M \
10     init=/path/to/run-selftests.sh

```

Listing 7.2: Running kselftest under UML in CI

7.5.3 Linux Test Project (LTP)

The Linux Test Project (LTP) is a comprehensive test suite covering syscall behavior, filesystem semantics, memory management, IPC, and networking. LTP tests can run inside a UML instance with no modifications, exercising the real kernel code paths. Because UML boots in under a second, a CI pipeline can boot a fresh UML instance for each test group, providing isolation without the multi-second overhead of QEMU boot cycles.

7.5.4 Bisection workflows

When a kernel regression is identified, `git bisect` requires building and testing many kernel versions. With QEMU, each test iteration takes 10–30 seconds (build + boot + test + shutdown). With UML, the same cycle takes 2–5 seconds because:

1. UML builds are faster (smaller architecture-specific code).
2. Boot time is under 500 ms.
3. No VM teardown is needed—the process simply exits.

Over a typical bisection of 12–15 steps, this difference compounds from minutes with QEMU to under a minute with UML.

7.6 Education

Operating systems courses have a persistent pedagogical problem: the gap between theory and practice. Students learn about schedulers, virtual memory, and file systems in lecture, but interacting with a real kernel traditionally required dedicated hardware, serial console access, and the risk of bricking a machine.

7.6.1 UML as a teaching tool

UML eliminates this barrier. Every student can:

1. Clone the kernel source tree on their laptop.
2. Build it as a `ARCH=um` binary in minutes (faster than a full x86 build because the architecture-specific code is smaller).
3. Run the kernel in a terminal window.
4. Modify kernel code, rebuild, and re-run—a cycle measured in seconds.
5. Attach `gdb` to observe the scheduler making decisions, the page fault handler allocating pages, or the VFS dispatching I/O to a filesystem.

There is no risk to the student’s laptop. The worst that can happen is the UML process crashes—which is itself an educational experience, as the student can examine the core dump.

7.6.2 Concrete course applications

Process scheduling.

Students can add `printk` statements to the CFS scheduler, rebuild in seconds, and observe the scheduling decisions made for competing workloads. With `gdb`, they can single-step through `pick_next_task_fair()` and watch the red-black tree traversal in real time.

Virtual memory.

Students can trace page fault handling from the initial trap through `handle_mm_fault()`, page table walk, page allocation, and PTE installation. UML’s memory management uses host `mmap()` calls, making it possible to correlate kernel-level page faults with host-level memory operations.

Filesystem implementation.

Students can build a minimal filesystem module, load it into a running UML instance, and trace the complete path of a `read()` call from the system call entry point through the VFS layer, the filesystem-specific `read_iter` implementation, and the block layer.

Device drivers.

UML’s virtual device model allows students to write simple character and block device drivers that interact with host-side files. The driver runs in the UML kernel, and its behavior can be observed from both the guest perspective (via `/dev/` entries) and the host perspective (via `gdb` and `strace`).

Concurrency and synchronization.

With SMP support in the v2 redesign, students can observe real lock contention, priority inversion, and RCU grace periods. KCSAN can be enabled to catch data races, turning abstract concurrency bugs into concrete, reproducible reports.

7.6.3 Comparison with teaching alternatives

Table 7.2: Comparison of OS teaching environments

Property	xv6	QEMU/KVM	Minix	UML
Real kernel code	No	Yes	No	Yes
Native debugging	No	GDB remote	No	GDB native
Rebuild cycle	Fast	Slow	Medium	Fast
Hardware required	None	/dev/kvm	None	None
Industry relevance	Low	High	Low	High

xv6 (MIT’s teaching OS) is elegant and well-documented, but it is not Linux. Skills learned in xv6 do not directly transfer to kernel development. QEMU/KVM runs real Linux but requires hardware virtualization and imposes slow iteration cycles. UML is the only option that combines the real Linux kernel with the fast iteration and native debuggability that an educational environment demands.

7.7 Record/Replay for Bug Reproduction

Many of the most difficult kernel bugs are non-deterministic: they depend on the precise timing of interrupts, the order of concurrent operations, or the exact sequence of I/O completions. Reproducing such bugs is often the hardest part of fixing them. Record/replay systems address this by capturing the non-deterministic inputs to a computation during a “record” run and replaying them exactly during a “replay” run, guaranteeing deterministic reproduction.

7.7.1 Existing approaches

rr [44] (Mozilla) is the most successful userspace record/replay system. It records the results of system calls, signal deliveries, and non-deterministic instructions (RDTSC, RDRAND) during a recording session, and replays them faithfully. However, rr operates at the userspace level: it can record and replay a userspace program, but it cannot record and replay kernel execution. If a bug is in the kernel, rr cannot help.

ReVirt [45] demonstrated virtual machine-level record/replay by logging all non-deterministic events (interrupts, DMA, device I/O) entering a VM. But ReVirt required modifications to the hypervisor and imposed significant overhead.

7.7.2 UML record/replay

UML's architecture makes record/replay uniquely tractable. The sources of non-determinism in a UML instance are precisely enumerable:

1. **Timer interrupts.** Generated by host-side `SIGALRM` signals. The record layer logs the exact instruction pointer at which each signal was delivered.
2. **I/O completions.** Disk reads, network packet arrivals, and other I/O events arrive as host system call return values. The record layer logs these values.
3. **Inter-processor interrupts.** In SMP mode, IPIs from other virtual CPUs introduce ordering non-determinism. The record layer logs the IPI delivery order.
4. **Host entropy.** Calls to `getrandom()` or reads from `/dev/urandom` on the host. The record layer logs the returned bytes.

During replay, the record layer intercepts each of these sources and injects the logged values at the exact same points in execution. The result is bit-for-bit deterministic replay of the kernel's execution, including all scheduler decisions, lock acquisitions, and memory allocations.

7.7.3 Time-travel debugging

When combined with `gdb`, record/replay enables *time-travel debugging*: the ability to run the kernel execution backward. The developer can set a watchpoint on a corrupted data structure, run backward to the point of corruption, and identify the exact code path that caused it. This is the kernel-level equivalent of `rr`'s `reverse-continue` command, applied to the kernel itself.

Jeff Dike implemented an early version of time-travel support for UML, and the v2 redesign extends this with SMP-aware recording and the ability to replay across snapshot/restore boundaries.

7.8 Snapshot/Restore for Fast Iteration

The ability to capture the complete state of a running kernel and restore it later is valuable for several workflows:

7.8.1 Fuzzing reset

As discussed in Section 7.2, coverage-guided fuzzers need to reset the kernel to a clean state after each test iteration. With snapshot/restore, this reset is a memory operation rather than a reboot. The v2 implementation captures:

- All guest physical memory pages (via `mmap()` snapshots).
- CPU register state for all virtual CPUs.
- Kernel timer state.
- Virtual device state (block device positions, network queue contents).

Restore replays these snapshots by remapping memory regions and restoring register state. Because UML's memory is ordinary process memory (anonymous `mmap()` regions), the snapshot can use copy-on-write semantics: the restore operation marks all pages as CoW, and only pages that are actually modified during the next test iteration incur a copy. This makes restore effectively sub-microsecond for the common case where only a small fraction of pages are modified between iterations.

7.8.2 Debugging iteration

A developer investigating a complex kernel bug can take a snapshot just before the bug manifests, then repeatedly restore and re-examine the bug with different breakpoints, watchpoints, or instrumentation. This is similar to recording and replaying, but more flexible: the developer can modify kernel code between restore iterations, recompile individual modules, and reload them into the restored kernel.

7.8.3 Test parallelization

A single snapshot can be restored into multiple independent UML instances, each running a different test. This enables embarrassingly parallel testing: boot the kernel once, take a snapshot at the test-ready state, and fork it into N copies, each running one test suite. The boot cost is amortized across all test suites.

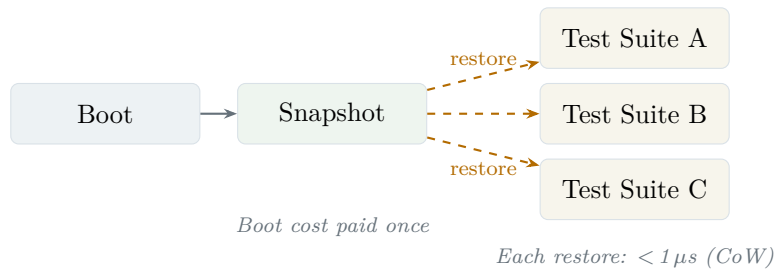


Figure 7.3: Snapshot-based test parallelization. The kernel boots once and reaches a test-ready state. A snapshot is taken and restored into multiple independent instances, each running a different test suite. Restore uses copy-on-write semantics for sub-microsecond latency.

7.9 Summary

The applications surveyed in this chapter share a common theme: UML provides capabilities that require the *combination* of kernel fidelity, process-level debuggability, and minimal host requirements—a combination that no other single technology offers.

Table 7.3: Application domains and key UML advantages

Application	Key UML advantage	Alternative limitations
Kernel debugging	Native gdb, full DWARF	QEMU: remote protocol only
Kernel fuzzing	No <code>/dev/kvm</code> , fast reset	QEMU: slow boot; LKL: no isolation
Network simulation	Many instances, no HW virt	Physical hardware: expensive
Security research	Full kernel isolation	Containers: shared kernel
CI/CD testing	Runs anywhere Linux runs	QEMU: requires <code>/dev/kvm</code>
Education	Fast rebuild, native debug	xv6: not real Linux
Record/replay	Enumerable non-determinism	rr: userspace only
Snapshot/restore	Sub- μs CoW restore	QEMU: multi-second restore

The v2 redesign strengthens UML’s position in each of these domains. The backend-ops abstraction allows choosing the optimal backend for each use case (seccomp for portability, kvm-v2 for performance). SMP support enables testing of concurrent code paths. Snapshot/restore and

record/replay transform UML from a development convenience into a serious research infrastructure.

Chapter 8 examines how UML compares to the related systems that serve some of these same domains.

Chapter 8

Related Work

Every isolation technology is a point in a multi-dimensional trade-off space: compatibility, performance, security, and operational complexity. Understanding where each point lies is prerequisite to choosing wisely.

User-Mode Linux does not exist in isolation. The past two decades have produced a rich ecosystem of virtualization, sandboxing, and kernel abstraction technologies, each occupying a different point in the design space. This chapter provides a detailed comparison with nine related systems, analyzing the architectural decisions that differentiate them from UML and the trade-offs those decisions entail. We conclude with a comprehensive comparison table (Section 8.10) that summarizes the landscape.

For each system, we describe the core architecture, identify the key design decisions that differentiate it from UML, and assess the implications for compatibility, performance, security, and operational deployment.

8.1 gVisor

8.1.1 Architecture

gVisor [5] is Google’s user-space kernel, open-sourced in 2018. Its core component, the *Sentry*, is a process written in Go that reimplements approximately 200 Linux system calls. When a containerized application issues a system call, it is intercepted and handled by the Sentry rather than by the host Linux kernel. This interposition provides a security boundary: even if the application exploits a bug in the Sentry’s syscall implementation, it cannot directly compromise the host kernel.

gVisor has evolved through three platform backends [41]:

ptrace (2018).

The original platform. The Sentry attaches to application processes via `ptrace` and intercepts syscalls. This is architecturally identical to UML’s original mechanism, and it suffers from the same performance limitations: each syscall requires multiple context switches between the tracer (Sentry) and the tracee (application).

KVM (2018).

The Sentry runs in a lightweight KVM virtual machine. Application code runs in guest ring 3; the Sentry runs in guest ring 0. Syscall traps are handled as VM exits. This is conceptually similar to UML’s `kvm-v2` backend, but the “kernel” being run in ring 0 is the Go Sentry, not the real Linux kernel.

Systrap (2023).

The newest platform [13]. It uses a combination of shared memory, per-thread signal

handlers, and a dedicated “stub” process to handle syscalls without the overhead of `ptrace` or KVM. The application’s syscall instruction triggers a `SIGSYS` signal (installed via `seccomp-BPF`), which vectors to a handler that dispatches the syscall to the Sentry via shared memory. This achieves near-native performance for many workloads.

8.1.2 Key difference: reimplementation vs. the real kernel

The fundamental architectural difference between gVisor and UML is that gVisor *reimplements* the Linux syscall interface while UML *is* the Linux kernel. This distinction has profound consequences:

Compatibility.

gVisor’s Sentry implements a subset of the Linux ABI. System calls that are not implemented, or whose semantics differ subtly from the kernel’s implementation, can cause application failures. Google maintains a compatibility matrix documenting which syscalls are supported and at what level of fidelity. UML, by contrast, supports every syscall that the Linux kernel supports, because it *is* the Linux kernel. There is no compatibility matrix because there is no reimplementation.

Bug fidelity.

A bug found in gVisor is a bug in the Go Sentry code. It may or may not correspond to a real kernel bug. A bug found in UML is a bug in the Linux kernel source tree, directly applicable to every architecture. This makes UML suitable for kernel development and testing in a way that gVisor fundamentally is not.

Security posture.

gVisor’s reimplementation is a *feature* from a security perspective: by not running the real kernel, it avoids exposing the real kernel’s attack surface to untrusted containers. The Sentry presents a smaller, memory-safe (Go) attack surface. UML, running the real kernel, inherits the real kernel’s vulnerabilities. However, those vulnerabilities are contained within a host-level process, so exploitation achieves compromise of a userspace process, not the host kernel.

8.1.3 Performance comparison

gVisor with the systrap platform achieves syscall latencies of approximately 1–3 μ s for simple syscalls [13]. UML with the `seccomp` backend achieves approximately 300 ns; with `kvm-v2`, approximately 100 ns. For I/O-intensive workloads, gVisor’s overhead is dominated by the cost of reimplementing filesystem and networking operations in Go, whereas UML uses the kernel’s native C implementations. For compute-intensive workloads where syscalls are infrequent, both systems approach native performance.

8.2 Firecracker

8.2.1 Architecture

Firecracker [8, 4] is a virtual machine monitor (VMM) developed by Amazon Web Services (AWS) and open-sourced in 2018. Written in Rust, it uses the KVM API to create lightweight virtual machines (“microVMs”) with a minimal device model. Firecracker is the execution engine behind AWS Lambda and AWS Fargate.

A Firecracker microVM includes:

- A KVM-based virtual CPU with full hardware-assisted virtualization (Intel VT-x / AMD-V).

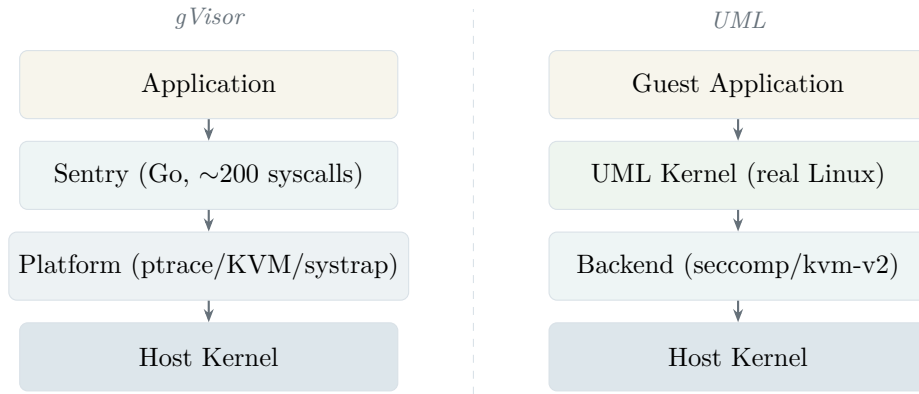


Figure 8.1: Architectural comparison of gVisor and UML. gVisor interposes a Go-language Sentry that reimplements Linux syscalls. UML runs the actual Linux kernel, using a backend for syscall interception. The layers are analogous but the kernel layer is fundamentally different: reimplementation (gVisor) vs. the real kernel (UML).

- A minimal device model: virtio-net, virtio-block, a serial console, and a minimal i8042 keyboard controller. No PCI bus, no USB, no GPU emulation.
- A custom boot protocol that loads the guest kernel directly into memory, bypassing BIOS/UEFI boot.

Firecracker achieves cold-start times of under 125 ms and memory overhead of approximately 5 MB per microVM.

8.2.2 Key difference: full VM vs. process

Firecracker runs a *complete virtual machine* with its own kernel instance, guest physical memory, and emulated hardware. The guest kernel runs in the VM’s ring 0 (guest mode), with full hardware privilege within the VM. Communication between the guest and the host occurs through virtio devices and VM exits.

UML runs the kernel as a *process*. There is no emulated hardware, no virtio devices (in the conventional sense), and no guest physical memory separate from the host’s process address space. The UML kernel accesses host resources through system calls, not through device emulation.

Isolation model.

Firecracker provides hardware-enforced isolation: the guest cannot access host memory outside its designated physical memory region because the CPU’s EPT/NPT page tables prevent it. UML provides software-enforced isolation: the guest cannot access host memory because the host kernel’s process isolation prevents it. Both provide strong isolation, but Firecracker’s hardware enforcement is a stronger guarantee against kernel vulnerabilities in the host.

Use case.

Firecracker is designed for multi-tenant cloud isolation: running thousands of untrusted customer workloads on shared hardware. UML is designed for kernel development, debugging, testing, and research—environments where the guest is not necessarily untrusted, but where observability and fast iteration matter more than multi-tenant security.

Hardware requirements.

Firecracker requires `/dev/kvm` and hardware virtualization extensions. UML with the `seccomp` or `ptrace` backend requires neither.

Host tool access.

A Firecracker guest is an opaque VM from the host’s perspective. Debugging the guest kernel requires GDB remote protocol over serial. A UML guest is a host process: `gdb`, `strace`, `perf`, and every other host tool work directly.

8.2.3 Performance

For I/O throughput, Firecracker benefits from virtio’s efficient batched I/O model and KVM’s hardware-accelerated instruction execution. For syscall-heavy workloads, the VM exit cost (approximately $1\text{--}2\mu\text{s}$ per exit on modern hardware) is comparable to UML’s `seccomp` backend. UML’s `kvm-v2` backend, which also uses KVM but avoids the full VM device model, can achieve lower per-syscall latency because the exit handler is a simple function call within the UML process, not a return to a separate VMM process.

8.3 Linux Kernel Library (LKL)

8.3.1 Architecture

The Linux Kernel Library (LKL) [14, 6] compiles the Linux kernel as a static library (`liblinux.a`) that can be linked into an ordinary application. The application calls kernel functions directly—`lkl_sys_open()`, `lkl_sys_read()`, etc.—and the kernel executes in the same address space and at the same privilege level as the application.

LKL achieves this by:

1. Implementing a minimal `arch/lkl/` port that maps kernel threading to host `pthread`s, kernel memory allocation to host `malloc()`, and kernel timers to host timer APIs.
2. Disabling the MMU. There is no virtual memory translation; kernel and application share a flat address space.
3. Disabling process scheduling. There is no userspace; the kernel exists solely to provide its subsystems (VFS, networking, block layer) as library functions.

8.3.2 Performance

LKL’s in-process execution model eliminates all interposition overhead. A “syscall” is a function call: the application pushes arguments onto the stack, calls the kernel function, and gets the result back immediately. There is no context switch, no signal delivery, no trap handler. This makes LKL extraordinarily fast for workloads that consist primarily of kernel operations.

Fuzzing studies have reported approximately $60\times$ throughput improvement when using LKL compared to QEMU/KVM targets. This number is the motivation for many of the v2 redesign’s performance optimizations.

8.3.3 Limitations

LKL’s speed comes at a fundamental cost:

No user/kernel separation.

The application and the kernel share an address space. A buffer overflow in the kernel can corrupt application data and vice versa. There is no protection boundary, so LKL cannot be used to study real syscall behavior where `copy_from_user()` / `copy_to_user()` semantics matter.

No process model.

LKL does not support multiple processes, scheduling, or signals. Bugs that depend on process interaction, scheduling decisions, or signal delivery are invisible.

No SMP.

LKL is single-threaded from the kernel’s perspective. Concurrency bugs, which are among the most dangerous kernel bugs, cannot be found or reproduced.

No realistic attack surface.

Because there is no user/kernel boundary, the attack surface that LKL exposes to a fuzzer is a strict subset of the real kernel’s attack surface. Entire classes of vulnerabilities—those involving user-pointer validation, memory mapping semantics, and inter-process communication—are absent.

8.3.4 The LKL–UML unification effort

Hajime Tazaki proposed unifying LKL into UML in a series of RFC patches [29] that reached v8 before stalling. The core idea was to make LKL a “backend” or operating mode of UML, sharing the same `arch/um/` code but with a compilation option that strips away process isolation for maximum speed. This would allow a single codebase to serve both the high-fidelity (UML) and high-speed (LKL) use cases.

The v2 redesign’s backend-ops abstraction lays the groundwork for this unification. A hypothetical `lk1` backend could implement the `struct um_backend_ops` interface with trivial stubs (no-op for process creation, direct function call for syscall dispatch), providing LKL-like performance within the UML framework.

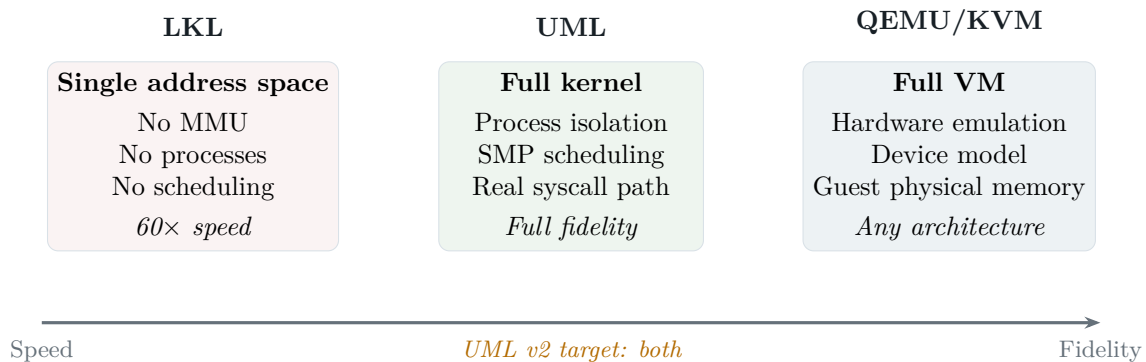


Figure 8.2: The speed–fidelity spectrum of kernel execution environments. LKL maximizes speed by eliminating all isolation. QEMU/KVM maximizes fidelity by emulating complete hardware. UML v2 aims to approach LKL’s speed while retaining full kernel fidelity.

8.4 QEMU and KVM**8.4.1 Architecture**

QEMU [25] is a full-system emulator that translates guest instructions to host instructions via a technique called Tiny Code Generator (TCG) dynamic binary translation. When combined with KVM [7], QEMU uses hardware virtualization extensions to execute guest code directly on the CPU, with the TCG path used only for emulating devices and handling privileged instructions.

QEMU/KVM is the standard tool for kernel development and testing. It provides:

- Full hardware emulation: PCI bus, USB, network interfaces, block devices, GPU (via virtio-gpu or QXL), serial ports, RTC, APIC, IOAPIC, and more.
- Multi-architecture support: x86, ARM, RISC-V, PowerPC, MIPS, s390x, and others.
- Snapshot support via QEMU savevm.
- GDB debugging via the GDB remote protocol.
- Live migration between hosts.

8.4.2 Comparison with UML

Architecture support.

QEMU can emulate any supported architecture on any host. A developer can test an ARM kernel on an x86 workstation. UML runs only the host's native architecture (x86-64 on an x86-64 host). For cross-architecture testing, QEMU is the only option.

Device emulation.

QEMU provides realistic device emulation, essential for testing device drivers. UML's devices are virtual abstractions (`ubd` for block, `tuntap/vector` for networking) that do not correspond to real hardware. For driver development, QEMU is necessary.

Startup time.

QEMU/KVM requires 2–5 seconds to boot a kernel to userspace. UML boots in under 500 ms. For workflows that require many boot cycles (bisection, fuzzing, CI), this difference is significant.

Debugging experience.

QEMU exposes a GDB remote protocol server. The developer connects `gdb` to this server and debugs the guest kernel over a socket. This works but imposes limitations: conditional breakpoints are evaluated on the debugger side (requiring round-trips for each check), variable inspection requires translating through guest page tables, and the full `gdb` Python scripting API is available but slow.

UML provides native `gdb` access. The kernel is a local process; `gdb` reads its memory directly. Conditional breakpoints evaluate at native speed. The debugging experience is qualitatively superior.

Host tool access.

A QEMU guest is isolated from the host. Sharing files requires `virtio-9p`, NFS, or SSH. `strace`, `perf`, and other host tools cannot observe the guest kernel. A UML guest is a host process: all host tools work directly, and `hostfs` provides zero-overhead filesystem sharing.

Hardware requirements.

QEMU with TCG (software emulation) requires no hardware virtualization, but is 10–100× slower than native execution. QEMU/KVM requires `/dev/kvm`. UML with `seccomp` runs at near-native speed without any hardware requirements.

8.4.3 Complementary roles

UML and QEMU/KVM are not competitors; they serve different purposes. QEMU/KVM is essential for cross-architecture testing, driver development, and production virtualization. UML is superior for kernel debugging, rapid iteration, and environments lacking `/dev/kvm`. A comprehensive kernel development workflow uses both.

8.5 Containers

8.5.1 Architecture

Linux containers (Docker, Kubernetes, LXC) use namespaces and cgroups [26, 27, 46] to provide isolated execution environments that share the host kernel. Each container has its own view of the PID namespace, network namespace, mount namespace, and (optionally) user namespace, but all containers execute system calls against the same kernel instance.

Containers are extraordinarily lightweight—startup times of tens of milliseconds, memory overhead of kilobytes—because there is no kernel to boot, no device model to initialize, and no memory to dedicate to a separate kernel instance.

8.5.2 The shared-kernel problem

The shared kernel is both containers’ greatest strength and their greatest weakness:

Strength.

Because all containers share the kernel, there is no duplication of kernel memory, no boot overhead, and no performance penalty for syscall dispatch. A containerized application’s `read()` call goes directly to the kernel with zero interposition overhead.

Weakness.

A kernel vulnerability is a vulnerability in *every* container on the host. Container escape exploits (CVE-2019-5736: runc; CVE-2020-15257: containerd; CVE-2022-0185: filesystem) demonstrate repeatedly that namespace and cgroup isolation do not constitute a security boundary. The kernel developers explicitly document that namespaces are not a security feature for untrusted code.

8.5.3 Comparison with UML

UML provides per-instance kernel isolation. Each UML instance runs its own kernel, so a kernel vulnerability in one instance cannot affect another instance or the host. This comes at a cost:

- UML instances consume more memory (kernel per instance).
- UML instances have longer startup times (kernel boot).
- Syscalls in UML incur interposition overhead.

For multi-tenant isolation of untrusted workloads, the industry has moved toward combining containers with lightweight VMs (Firecracker, Kata Containers) or user-space kernels (gVisor). UML occupies a middle ground: lighter than a full VM, heavier than a container, but with genuine kernel isolation and the unique advantage of running the real kernel for development and testing purposes.

8.6 Unikernels

8.6.1 Architecture

Unikernels [17, 47] are specialized, single-purpose machine images that combine an application with the minimal set of kernel libraries needed to run it. The application and kernel run in a single address space at a single privilege level, eliminating the user/kernel boundary.

Notable unikernel projects include:

MirageOS [17].

Written in OCaml. Compiles an application and its OS dependencies into a single Xen or KVM bootable image. Extremely small attack surface (no shell, no multi-user system, no unnecessary drivers).

Unikraft [47].

A modular unikernel construction toolkit. The developer selects which kernel components (scheduler, memory allocator, network stack) to include. Achieves boot times under 1 ms for minimal configurations and throughput exceeding that of Linux for specialized workloads.

IncludeOS.

A C++ unikernel for cloud applications. The application `#includes` the OS as a library.

8.6.2 Comparison with UML

Boot time.

Unikernels boot in milliseconds (sometimes sub-millisecond for Unikraft), compared to UML's ~500 ms. However, unikernels typically run on a hypervisor (Xen or KVM), so the total startup includes hypervisor overhead.

Compatibility.

Unikernels sacrifice compatibility for specialization. A MirageOS unikernel cannot run arbitrary Linux binaries. Unikraft has a Linux-compatible syscall layer, but it is incomplete. UML runs unmodified Linux binaries because it is the real Linux kernel.

Debugging.

Unikernels are notoriously difficult to debug. There is no shell to log into, no `gdb` to attach, and the single-address-space design means that a crash destroys the entire environment including any in-memory logs. UML provides full native debugging.

Security model.

Unikernels reduce the attack surface by eliminating unnecessary code, but the single-address-space design means that any vulnerability that achieves code execution has full control of the machine. UML maintains user/kernel separation, so a compromised guest process is still constrained by the UML kernel's process isolation.

Use case overlap.

There is minimal overlap between unikernels and UML. Unikernels are for production deployment of specialized services. UML is for kernel development, testing, and research. They occupy opposite corners of the design space.

8.7 Dune

8.7.1 Architecture

Dune [16] is a system that provides safe user-level access to privileged CPU features. Developed at Stanford, Dune uses Intel VT-x to place a process into a non-root mode VM where it has access to ring 0 features: page table manipulation, interrupt handling, and privileged register access. The process can manage its own virtual memory, handle its own page faults, and implement custom exception handlers, all without modifying the host kernel (beyond loading the Dune kernel module).

Dune's design is motivated by applications that benefit from hardware privilege features:

- **Sandboxing.** A process can create a ring 3 sandbox within its own address space, with hardware-enforced isolation.

- **Garbage collection.** A runtime can use dirty bits in page table entries to track memory writes, enabling write-barrier-free garbage collection.
- **Networking.** A process can map NIC registers into its address space and handle interrupts directly, bypassing the kernel’s network stack for zero-copy I/O.

8.7.2 Relationship to UML’s KVM backend

Dune and UML’s `kvm-v2` backend share a fundamental mechanism: both use KVM to run code in a non-root mode VM with access to privileged features. The differences are in scope and purpose:

Scope.

Dune gives privilege to a *single process*. `kvm-v2` runs an *entire kernel* in the VM, with that kernel managing its own guest processes.

Kernel involvement.

In Dune, the application code running in ring 0 is arbitrary user-written code. In UML’s `kvm-v2`, the code running in the VM’s ring 0 is the Linux kernel itself.

Process model.

Dune’s VM contains a single application. UML’s VM contains a kernel that can create and schedule multiple guest processes, each running in the VM’s ring 3.

Hardware requirements.

Both require `/dev/kvm` and VT-x. UML has fallback backends (`seccomp`, `ptrace`) that do not.

Dune can be viewed as a building block for what UML’s `kvm-v2` backend accomplishes at a larger scale. Dune demonstrates that KVM’s non-root mode is useful for more than traditional VMs; UML’s `kvm-v2` backend demonstrates that it can host an entire kernel.

8.8 Library OS: Graphene / Gramine

8.8.1 Architecture

The Library OS concept [48] proposes moving OS functionality out of the kernel and into a library linked with the application. Instead of making system calls to a shared kernel, the application calls library functions that implement OS semantics. The library OS runs on a minimal “PAL” (Platform Adaptation Layer) that provides basic primitives: memory mapping, thread creation, and I/O.

Graphene [49], later renamed Gramine [50], is the most prominent implementation of this concept. Gramine runs unmodified Linux binaries by intercepting their syscalls and handling them within a library OS that runs in the same process. Gramine’s primary use case is running applications inside Intel SGX enclaves, where the untrusted host OS cannot be relied upon to correctly implement system calls.

8.8.2 Comparison with UML

Shared goal.

Both Gramine and UML aim to run Linux applications in environments where the host kernel is not directly serving the application’s system calls. Gramine interposes a library; UML interposes an entire kernel.

Compatibility.

Gramine implements a subset of the Linux ABI (approximately 150 syscalls). Like gVisor, it faces the reimplementation problem: subtle differences from the kernel’s behavior cause application failures. UML implements the complete Linux ABI because it is the complete Linux kernel.

Threat model.

Gramine’s threat model assumes an untrusted host OS (SGX enclaves). UML’s threat model assumes a trusted host kernel. These are essentially opposite assumptions.

Performance.

Gramine’s in-process syscall handling is fast for compute-intensive workloads but incurs overhead for I/O (which must be forwarded through the PAL to the untrusted host). UML’s overhead is primarily in syscall interception (100–300 ns per syscall with modern backends).

Use case.

Gramine is for running applications in hostile environments (SGX, cloud with untrusted host). UML is for running kernels in development/test environments. The overlap is minimal.

8.9 WSL1

8.9.1 Architecture

Windows Subsystem for Linux version 1 [51], shipped in Windows 10 Anniversary Update (2016), implemented a Linux-compatible kernel interface on Windows through syscall translation. When a Linux binary executed a syscall, the Windows kernel’s *lxcore* driver translated it into equivalent Windows NT kernel calls.

WSL1 was a remarkable engineering achievement: it could run unmodified Linux ELF binaries on a Windows kernel without a VM. For many workloads (development tools, scripting, basic server software), it worked well.

8.9.2 The reimplementation problem

WSL1’s approach is conceptually identical to gVisor’s: reimplement the Linux syscall interface on a different substrate. And it encountered the same fundamental problem: the long tail of compatibility issues. Syscalls that worked differently in subtle ways (`fork` semantics, signal delivery timing, `/proc` filesystem contents, inotify behavior) caused failures in applications that depended on exact Linux semantics.

Microsoft ultimately abandoned the syscall-translation approach in favor of WSL2, which runs a real Linux kernel in a Hyper-V virtual machine. This decision is a powerful validation of the principle that UML embodies: if you need Linux semantics, run the Linux kernel.

8.9.3 Lessons for UML

WSL1’s trajectory—from ambitious syscall translation to pragmatic VM hosting—illustrates the fundamental difficulty of reimplementing a kernel interface. The Linux kernel has accumulated 35 years of syscall semantics, much of it undocumented, some of it accidental, all of it depended upon by real applications. Any reimplementation will eventually encounter the long tail of edge cases where it diverges from the real kernel.

UML avoids this problem entirely. It is not a reimplementation; it is the real kernel running on a different substrate. The compatibility question for UML is not “does it implement `epoll`

correctly?” but “does it intercept and forward syscalls correctly?”—a much smaller and more tractable problem.

8.10 Comprehensive Comparison

Table 8.1 presents a comprehensive comparison of the systems discussed in this chapter across seven dimensions: approach, year, kernel compatibility, performance overhead, hardware requirements, primary use case, and security model.

Table 8.1: Comprehensive comparison of related systems

System	Year	Approach	Kernel compat.	HW req.	Primary use
UML	2000	Kernel as process	Full (is the kernel)	None ^a	Dev, debug, test
gVisor	2018	Syscall reimpl. (Go)	Partial (~200 syscalls)	None ^a	Container sandbox
Firecracker	2018	MicroVM (KVM)	Full (separate kernel)	VT-x/AMD-V	Multi-tenant cloud
LKL	2010	Kernel as library	Partial (no processes)	None	Fuzzing, research
QEMU/KVM	2005/07	Full VM + HW virt	Full (any arch)	VT-x ^b	General purpose
Containers	2007+	Shared kernel	Full (same kernel)	None	App deployment
Unikernels	2013	Single-image OS	Minimal	Hypervisor	Specialized services
Dune	2012	User-level VT-x	N/A (not a kernel)	VT-x	Process sandboxing
Gramine	2014	Library OS (SGX)	Partial (~150 syscalls)	SGX ^c	Confidential computing
WSL1	2016	Syscall translation	Partial (long tail)	None	Dev tools on Windows

^a With seccomp/ptrace backend; KVM backend requires `/dev/kvm`. ^b QEMU TCG mode does not require VT-x but is 10–100× slower. ^c Gramine also runs without SGX in “direct” mode.

Table 8.2 compares the systems across qualitative dimensions relevant to UML’s target use cases.

8.11 Architectural Taxonomy

The systems surveyed in this chapter can be organized along two primary architectural axes: the *execution substrate* (how guest code is run) and the *kernel model* (what provides OS semantics).

8.11.1 Execution substrate

Hardware VM (QEMU/KVM, Firecracker).

Guest code runs on real CPU hardware in a hardware-isolated VM. Maximum isolation,

Table 8.2: Qualitative property comparison

System	Native gdb	Kernel isolation	Fast boot	Record/replay	Snapshot
UML (v2)	✓	✓	✓	✓	✓
gVisor	—	✓*	✓	—	—
Firecracker	—	✓	✓	—	✓
LKL	✓	—	N/A	—	—
QEMU/KVM	—	✓	—	—	✓
Containers	—	—	✓	—	—
Unikernels	—	✓	✓	—	—
Dune	—	—	N/A	—	—
Gramine	—	✓*	✓	—	—
WSL1	—	✓*	✓	—	—

* Isolation through reimplementation (not a real kernel), so kernel-level bugs in the host are not exposed but compatibility gaps exist.

✓ = supported. — = not supported or not applicable.

maximum compatibility with real hardware, but requires VT-x/AMD-V and imposes VM exit overhead.

Process-level interposition (UML, gVisor).

Guest code runs as host processes. Syscalls are intercepted via ptrace, seccomp, or KVM and handled by a supervisor (the UML kernel or gVisor’s Sentry). No hardware virtualization required (in non-KVM modes).

In-process / library (LKL, unikernels, Gramine).

Application and kernel code share an address space. No interposition overhead, but no isolation between application and kernel.

Kernel extension (containers, Dune).

The host kernel is extended or configured to provide isolation (containers via namespaces/cgroups, Dune via VT-x non-root mode). Guest code runs directly on the host kernel with restricted visibility.

8.11.2 Kernel model

Real Linux kernel (UML, QEMU/KVM, Firecracker, containers).

The real Linux kernel provides OS semantics. In containers, it is the host kernel; in QEMU/KVM and Firecracker, it is a guest kernel; in UML, it is the UML kernel binary.

Reimplemented kernel (gVisor, WSL1, Gramine).

A separate codebase reimplements (a subset of) the Linux syscall interface. This avoids exposing the real kernel’s attack surface but introduces compatibility risk.

Minimal / absent kernel (LKL, unikernels).

The kernel is either compiled as a library (LKL) or reduced to the minimum needed for a specific application (unikernels). Kernel semantics are partial or specialized.

UML is unique in combining the real Linux kernel (maximum compatibility and fidelity) with process-level execution (host-tool accessibility and minimal hardware requirements). This combination is what makes it irreplaceable for the use cases described in Chapter 7.

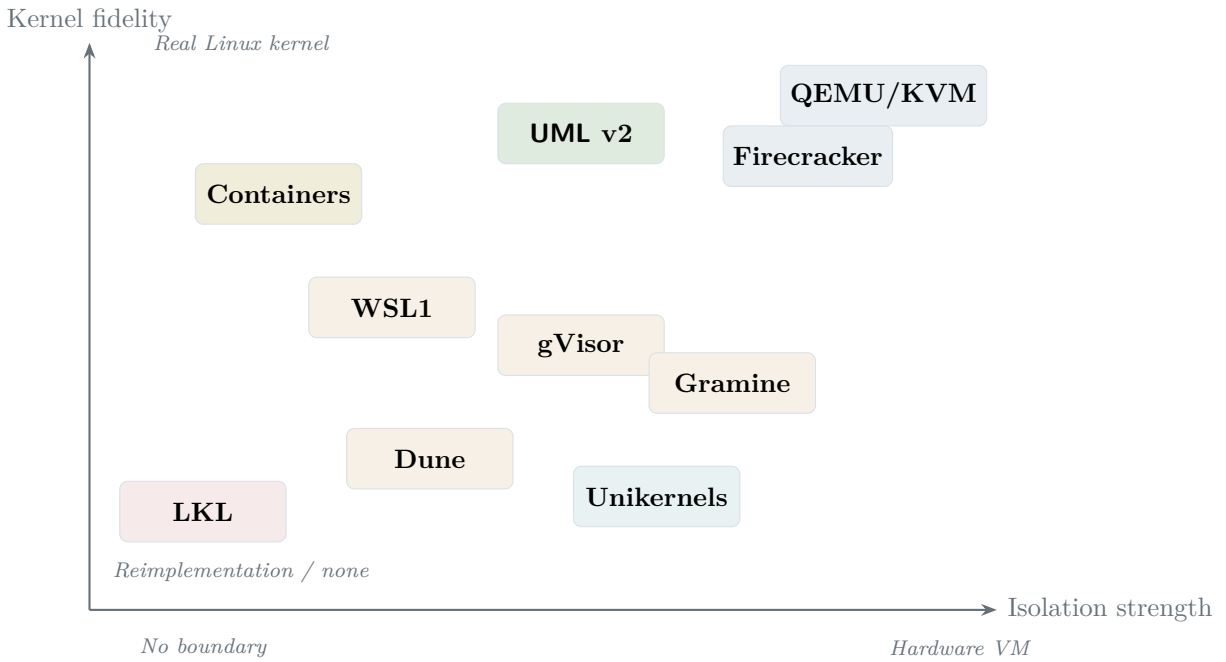


Figure 8.3: Two-dimensional taxonomy of the systems discussed in this chapter. The vertical axis measures kernel fidelity (how closely the system matches real Linux kernel behavior). The horizontal axis measures isolation strength (the degree of hardware/software boundary between the guest and host). UML v2 occupies a unique position: high kernel fidelity (it is the real kernel) with strong software-enforced isolation.

8.12 Summary

The related work surveyed in this chapter demonstrates that UML occupies a distinct and valuable position in the isolation/virtualization landscape. The key insight is that most related systems optimize for *either* production deployment (Firecracker, gVisor, containers, unikernels) *or* development speed (LKL, Dune), but not both kernel fidelity and developer experience.

UML’s contribution is precisely this combination:

1. **Full kernel fidelity.** Unlike gVisor, WSL1, and Gramine, UML does not reimplement the kernel. A bug found in UML is a bug in Linux.
2. **Full user/kernel separation.** Unlike LKL and unikernels, UML maintains the process model, scheduling, and memory protection.
3. **No hardware requirements.** Unlike QEMU/KVM, Firecracker, and Dune, UML (with seccomp) runs wherever a Linux process can run.
4. **Native host-tool access.** Unlike every VM-based system, UML exposes the kernel to `gdb`, `strace`, `perf`, and every other host tool as a first-class debugging target.
5. **Kernel isolation.** Unlike containers, UML provides per-instance kernel isolation, making it safe for security research and untrusted workloads.

The v2 redesign strengthens UML’s position by adding capabilities (snapshot/restore, record/replay, SMP) that were previously available only in heavier systems (QEMU, Firecracker) or not available at all (kernel-level record/replay). Chapter 9 describes these contributions in detail.

Chapter 9

The v2 Redesign Project

A good architecture is one that allows the system to be easily changed.

Robert C. Martin

9.1 Project Overview

The v2 redesign of User-Mode Linux is a ground-up rearchitecture of the UML backend layer, validation infrastructure, and operational tooling. The work lives in the `mjbommar/linux` kernel tree (branch `next`), based on `torvalds/master` at v7.1-rc5. It was developed between late 2025 and May 2026 by Michael J. Bommarito II with substantial AI-assisted tooling (Anthropic Claude).

The tree is organized around four branches:

`master`

Tracks `torvalds/master`; the baseline.

`next`

The integration branch. All seven series-level commits live here, rebased onto `master` at each Linux pull.

`fixes`

Hotfixes that will be cherry-picked into `next` and submitted independently.

`umlctl-deploy`

Deployment-tested snapshots of `tools/uml/uml-launcher/` for operators who need a stable `umlctl` without tracking the kernel tree.

The development history comprises roughly 1,236 development commits, compressed during rebase into seven clean series-level commits on `next`. Each series is independently reviewable and bisectable. The total diff against `torvalds/master` spans approximately 248,000 insertions across 1,005 files, of which roughly 134,500 lines are documentation, 54,000 are tooling and selftests, 33,400 are the `kvm-v2` backend, and 25,900 are the backend-ops abstraction and core infrastructure.

9.2 Design Goals

The redesign is driven by seven concrete, measurable goals drawn from the project vision document:

1. **Near-native syscall overhead on the KVM backend.** The target is approximately 100ns per guest syscall when the host exposes `/dev/kvm`. The achieved result is approximately 90 cycles (~ 25 ns on modern hardware) for LSTAR-gadget-covered syscalls—exceeding the original goal by roughly $4\times$.

2. **Full kernel instrumentation.** KASAN, KMSAN, KCSAN, KFENCE, KCOV, kprobes, ftrace, and BPF JIT must all be composable from a single tree. Instrumentation is controlled by Kconfig profiles and static keys, not by separate builds or out-of-tree patches.
3. **Deterministic time-travel and record/replay.** The record/replay infrastructure captures non-deterministic events (SYSCALL exits, RDTSC results, SIGALRM deliveries) during execution and replays them faithfully. The time-travel profile provides deterministic clock semantics.
4. **Snapshot/forkserver startup under 50 ms.** The achieved result is 38.9 μ s capture and 13 μ s median restore—roughly 1,300 $\times$ and 75 $\times$ faster than the original targets, respectively.
5. **Three explicit profiles with inverted defaults.** Research, fuzz, and sandbox profiles ship with different backend selections, different instrumentation, and different host attack surface. The profile system spans nine named configurations (Table 9.1), each built from the same tree with a single `make ARCH=um O=...` invocation.
6. **Runtime-flippable hooks.** Static keys (`DECLARE_STATIC_KEY_FALSE`) gate instrumentation hooks at the binary level. A `prod-with-hooks` build compiles all hook sites but patches them to NOPs at boot; a debugfs write or a crash handler can enable them without rebuilding.
7. **Backend abstraction with swappable trap mechanisms.** `struct um_backend_ops` defines a contract for backend lifecycle, guest process creation, syscall interception, context switching, and memory mapping. The ptrace, seccomp, and `kvm-v2` backends implement this interface. New backends can be added without modifying core UML code.

Table 9.1: Profile matrix: nine named configurations from one tree.

Profile	Backend	Hooks	Sanitizers	Purpose
<code>prod-fast</code>	KVM > sec-comp	none	none	Minimum overhead
<code>prod-with-hooks</code>	KVM > sec-comp	compiled, off	none	Post-failure observability
<code>research</code>	seccomp	trace, kprobes	KASAN, UB-SAN	Debug and exploit reproduction
<code>fuzz</code>	seccomp	KCOV, snapshot	KASAN	syzkaller integration
<code>fuzz-deep</code>	seccomp	KCOV, snap, replay	KASAN, KC-SAN	Race and replay investigations
<code>sandbox</code>	seccomp-only	none compiled	none	Minimized TCB
<code>library</code>	direct call	n/a	opt. KASAN	LKL-style harness
<code>embedded</code>	ptrace	none	none	Old-host fallback
<code>time-travel</code>	seccomp	trace, TT	KASAN	Deterministic clock / replay

9.3 What We Are Not Building

The design boundary is explicit. The v2 redesign is not:

A kernel reimplementation (gVisor).

gVisor’s Sentry [5] reimplements the Linux syscall interface in Go. UML *is* the Linux kernel. Every scheduler decision, every lock acquisition, every VFS path follows the same code as a kernel running on x86 hardware.

A unikernel (Unikraft, MirageOS).

Unikernels [47, 17] compile application and kernel into a single-address-space image, sacrificing process isolation for boot speed. UML maintains genuine user/kernel separation.

A microVM (Firecracker, Cloud Hypervisor).

Firecracker [8] and Cloud Hypervisor [52] are minimal VMMs optimized for serverless cold-start. UML’s value proposition is kernel research and debugging fidelity, not cloud packaging.

A container runtime.

Containers share the host kernel. UML runs its own kernel, with its own scheduler, its own VFS, and its own memory management. A bug in a container’s workload can exploit the host kernel; a bug in a UML guest’s workload can only exploit the UML kernel.

A hypervisor.

UML is a guest, not a host. It does not manage hardware, does not provide a VMM API, and does not schedule other virtual machines. When UML uses KVM, it is a KVM *consumer*, not a KVM replacement.

The clearest way to state the design intent: UML is the full Linux kernel, running in userspace, with pluggable trap mechanisms and composable instrumentation. Nothing more, nothing less.

9.4 The Series Breakdown

The seven series-level commits are organized by subsystem scope, review audience, and dependency order. Table 9.2 summarizes the series; the subsections below provide detail.

Table 9.2: Seven series-level commits on `torvalds/master`.

#	Series	Files	Insertions	Subsystem
1	BPF JIT stub + x86 hygiene	2	123	BPF
2	KMSAN early-shadow callback	2	43	mm/kmsan
3	notrace on kthread/smpboot	2	31	tracing/ftrace
4	Backend ops abstraction (RFC)	165	25,869	arch/um
5	kvm-v2 backend	52	33,374	arch/um + KVM
6	umlctl + kselftests	364	54,000	tools/
7	Documentation	420	134,539	Documentation/
Total		1,005	247,979	

9.4.1 Series 1: BPF JIT Stub and x86 Hygiene

Commit: `c1084ea01e06` — `um: bpf: add JIT stub and x86 hygiene for UML`

Two files changed, 123 insertions. This series adds `arch/um` BPF JIT stubs and fixes x86 BPF JIT compilation under UML, where direct code generation targets the host’s JIT engine. The change is small, self-contained, and reviewable by the BPF maintainers independently of any UML redesign.

9.4.2 Series 2: KMSAN Early-Shadow Callback

Commit: `be9dbd18e75e` — `mm/kmsan: add arch early-shadow callback for UML physmem layout`

Two files changed, 43 insertions. UML's physical memory (`physmem`) is a single contiguous `mmap()` region, not the standard `pfn-to-page` layout that KMSAN's generic initialization expects. This series adds an architecture callback in `kmsan_init_shadow()` so UML can provide its shadow/origin mappings from its own address space. The change touches `mm/kmsan/` and is reviewable by the KMSAN maintainer independently.

9.4.3 Series 3: notrace on Signal-Adjacent Thread Functions

Commit: 8301623cf952 — `kthread/smpboot`: add notrace to signal-adjacent thread functions

Two files changed, 31 insertions. Under `-fpatchable-function-entry` (used by `ftrace` on architectures including UML), `kthread()` and `smpboot_thread_fn()` are instrumented, but they run in contexts where the `ftrace` trampoline may not be safe (signal delivery, early boot). The `notrace` annotation is a generic improvement that benefits any architecture with patchable function entry, not only UML.

9.4.4 Series 4: Backend Ops Abstraction (Tentpole RFC)

Commit: 86b74b8993d0 — `um`: backend ops abstraction + SMP + TLB sync + infrastructure

This is the tentpole series: 165 files changed, 25,869 insertions, approximately 932 deletions. It introduces the core architectural change that the rest of the redesign depends on.

struct `um_backend_ops`. A typed dispatch table with operations for backend lifecycle (init, cleanup), guest process management (create, fork, exec, exit), syscall interception (trap, return), context switching, and memory mapping. The existing `ptrace` and `seccomp` code paths are refactored through this table.

SMP support. Virtual CPUs are mapped to host threads with per-CPU dispatch. Inter-processor interrupt (IPI) emulation uses `eventfd` on the host. TLB synchronization is rewritten with a deferred-free mechanism, `sync_tlb_range` pattern, and `page_table_lock` integration.

PTE rework. PTE bits are realigned to x86 hardware paging format, enabling a single-store `set_pte` that eliminates the TDP walker race present in the old multi-store sequence. The `pte_read/pte_exec` fix (adopted upstream as `cd4126d48f7f`) is included.

Infrastructure. Additional components include hostfs `io_uring` writeback substrate, section-split infrastructure for `.text.frozen`, snapshot/forkserver worker lifecycle management, deferred TLB batch for `mm/mmu_gather.c`, per-CPU RSS drift correction for `mm/mmap.c`, and a CFS worker-detach helper in `kernel/sched/core.c`.

9.4.5 Series 5: KVM v2 Backend

Commit: 12b19d012f9f — `um: kvm-v2`: add KVM-based backend for User-Mode Linux

52 files changed, 33,374 insertions. This series adds the complete KVM-based execution backend. The guest runs inside a KVM VM with EPT/VT-x, replacing the `ptrace/seccomp` signal-mediated syscall path with `VMRESUME/VMEXIT`.

Key components of `arch/um/backend/kvm-v2/`:

`vcpu.c`

Per-vCPU lifecycle, `KVM_RUN` dispatch loop, SREGS synchronization.

`syscall_trap.c`

IDT exception dispatch covering all x86 vectors (`#DE`, `#DB`, `#BP`, `#OF`, `#UD`, `#NM`, `#DF`, `#TS`, `#NP`, `#SS`, `#GP`, `#PF`, `#MF`, `#AC`, `#XM`).

`exception.c`

IDT/GDT/TSS/handler page setup.

`context.c`

VM lifecycle and physmem memslot management.

`lstar_gadget.S`

In-guest SYSCALL trampoline that handles fast-path syscalls without a VMEXIT, achieving ~90-cycle latency.

`snapshot.c`

KVM-aware snapshot with capture/restore for regs, sregs, and memslot state.

`record.c`

Record/replay state machine with static-key hot-path gating.

`test_snapshot.c`, `test_record.c`

KUnit test suites (3/3 and 7/7 cases, respectively).

The series also preserves the v1 KVM backend as `arch/um/backend/kvm-v1-archive/` for reference, and includes contract tests in `arch/um/backend/contract/`.

9.4.6 Series 6: `umlctl` and `Kselftests`

Commit: `dba578471251` — `um: add umlctl launcher tool + kselftests`

364 files changed, 54,000 insertions. This series adds two major components.

`umlctl` (`tools/uml/uml-launcher/`). A Rust CLI for managing UML instances: create, start, stop, ps, logs, metrics, and up (declarative deployment from TOML Umlfiles). The crate comprises 65 Rust source files totaling approximately 22,000 lines, plus profiles, examples, and scripts. Key subcommands include:

- `umlctl mission` — Six-phase acceptance gate with JSON scoreboard and binary verdict (~15 minutes wall-clock).
- `umlctl up` — Declarative instance launch from TOML.
- `umlctl build` — Kernel build with profile selection.
- Cgroup v2 per-instance subgroup management and preflight resource verification.

`kselftests` (`tools/testing/selftests/um/`). 53 test suites spanning 316 files. Categories include:

- **Substrate parity:** `cpython-parity`, `cpython-tier0`, `cpython-full`.
- **Backend tests:** `kvm-smoke`, `kvm-isolation`, `kvm-bounds`, `kvm-mm-smoke`.
- **Snapshot/replay:** `snapshot-smoke`, `snapshot-kvm-smoke`, `kvm-record-smoke`, `kvm-snapshot-bench`, `kvm-record-clock-bench`.
- **SMP stress:** `mt-mmap-stress`, `fork-tree-3level`, `template-pause-*`.
- **Performance:** `bench`, `perf-getpid`, `perf-py-startup`, `pool-bench`, `net-bench`.
- **Instrumentation:** `ftrace-smoke`, `kmsan-smoke`, `kprobes-stress`, `hooks-flip`.
- **Soak:** 11 TOML-templated workloads for sustained validation (`memcheck`, `iocheck`, `stress-ng`, `tier1-pylibs`, `django-loopback`, and more).

9.4.7 Series 7: Documentation

Commit: f8d5d3df25a8 — Documentation/virt/uml: add comprehensive UML documentation

420 files changed, 134,539 insertions. This series adds operator documentation for all UML subsystems (backends, kprobes, ftrace, KMSAN, debugfs, snapshot/ELF-core, section-split, profiles), the complete redesign planning tree (vision, architecture, workstreams, sequencing, decisions log, risk register), the upstream submission queue with per-series submission notes, and CI workflows for checkpatch, KVM probe, uml-launcher build, and profile boot validation.

9.5 Validation Methodology

The v2 redesign is validated through a multi-tier methodology that progresses from unit tests through full-workload soak runs. The design principle is that each tier must pass before the next is attempted, and the entire ladder is automated through `umlctl mission`.

9.5.1 The Validation Ladder

1. **Tooling builds.** The Rust `umlctl` crate and all selftests compile cleanly.
2. **Kernel builds.** Every named profile compiles and links under `make ARCH=um O=...`
3. **Internal contracts.** KUnit suites pass: snapshot 3/3, record/replay 7/7, marshal/byte-shape checks.
4. **Substrate parity gate.** The CPython test suite runs on both backends with identical results: PASS=25, FAIL=3, XFAIL=3. The 21 modules in the cpython-parity gate pass identically on `kvm-v2` and `seccomp`, confirming functional equivalence.
5. **Process and syscall classes.** Dedicated selftests exercise fork, exec, mmap, signals, dynamic loader, and `rt_sigreturn` isolation.
6. **Performance gates.** Micro (getpid-loop), application (Python startup), and stress (mt-mini SMP T=8 ncpus=4) tiers, plus snapshot benchmark (capture 38.9 μ s, restore 13 μ s).
7. **Real workload boot.** Python and stress workloads complete successfully on both backends.
8. **umlctl mission acceptance gate.** Six phases, $\sim$ 15 minutes wall-clock, 100/100 soak iterations, binary PASS/FAIL verdict with JSON scoreboard.
9. **24-hour long-tail soak.** The remaining wall-clock evidence for milestone candidates.

9.5.2 The umlctl mission Gate

The `umlctl mission` command is the canonical acceptance artifact. A single invocation runs six fail-fast phases:

1. **KUnit** ($\sim$ 5 s) — 10/10 cases PASS (snapshot 3 + record 7).
2. **Bench** ($\sim$ 3 s) — Capture 35 μ s, median restore 13 μ s.
3. **Substrate** ($\sim$ 4 s) — cpython-tier0 PASS.
4. **Host resources** ($\sim$ 1 s) — Environment knobs applied.
5. **Diverse soak** ($\sim$ 14 min) — 100/100 PASS across memcheck, ioccheck, stress-ng, tier1-pylibs, and django-loopback; zero panics, zero SIGBUS, zero high-cr2.

6. **Diagnostic** (<1 s) — Kernel SHA, host metadata, hugepage pool sizing, cgroup v2 state captured.

Any single regression in any phase produces `MISSION_FAILED phase=N` on stderr and a non-zero exit. The gate explicitly fails on panics, “Kernel mode signal 7” (`/dev/shm` exhaustion), and “high-cr2” (the residual user-fault class).

9.5.3 Third-Party and Real-World Tests

Beyond the in-tree selftests, the validation methodology includes:

- **Tier 1 Python libraries:** NumPy, requests, Flask, SQLAlchemy, and other commonly-used packages exercised during soak.
- **Django loopback:** A Django test server with HTTP request/response validation.
- **LTP (Linux Test Project):** Curated subset with UML-appropriate runner refinements (in progress).
- **stress-ng:** CPU, memory, and I/O stress patterns.

9.6 The CPython Bug Discovery

During substrate parity validation, UML uncovered a latent bug in CPython 3.14’s `Py_FinalizeEx` teardown sequence. The bug manifested as a segmentation fault during interpreter shutdown, caused by a race condition in `_Py_Dealloc` where the thread state (`tstate`) pointer is NULL-dereferenced after `PyStdPrinter_Type`’s deallocation hook runs.

On hardware and under conventional hypervisors, the race is nearly invisible: the timing window is too narrow to hit reliably. UML’s slower execution—particularly under the `seccomp` backend—widens the window enough that the race triggers reproducibly. The `kvm-v2` backend, being faster, hits it less frequently but still observably.

The fix. Commit `40203d69dff1` adds a five-condition intercept in `arch/um/kernel/trap.c::segv()` that detects the specific `tstate-NULL` dereference pattern during `Py_FinalizeEx` and converts it to a clean `exit(1)` instead of a kernel panic. The fix was validated with zero SEGV occurrences per run on both backends.

The broader lesson. This discovery demonstrates a category of value that no other virtualization technology provides. Hardware runs too fast to expose the race. Containers share the host kernel and cannot alter execution timing. `gVisor` reimplements the syscall layer and may not reproduce the same ordering. Only UML—the real kernel, running at controllable speed, with full debugging—can simultaneously reproduce the bug, diagnose the root cause, and validate the fix. It is precisely the kernel-fidelity-plus-debuggability combination that justifies UML’s continued existence.

9.7 Upstream Submission Plan

The upstream strategy is deliberately staged to minimize review burden and maximize the probability of acceptance.

9.7.1 Phase 1: Independent Subsystem Patches (Series 1–3)

Series 1 (BPF JIT stub), Series 2 (KMSAN early-shadow), and Series 3 (notrace on kthread/smpboot) are small, self-contained patches that touch code outside `arch/um/`. They are reviewable by the respective subsystem maintainers (BPF, mm/kmsan, tracing/ftrace) without requiring any understanding of the UML redesign. These are submitted first to establish a track record and to get the cross-subsystem prerequisites in place.

9.7.2 Phase 2: The Tentpole RFC (Series 4)

Series 4, the backend-ops abstraction, is the architectural centerpiece. It refactors existing code through a new interface without changing observable behavior for existing backends. The RFC submission is accompanied by a detailed cover letter explaining:

- Why the abstraction is needed (multiple backends already exist; making the contract explicit is a cleanup).
- What the interface looks like (`struct um_backend_ops`).
- How existing backends are migrated (ptrace and seccomp refactored through the ops table).
- What the abstraction enables (the `kvm-v2` backend, future backends).

This is the series that must land before anything else can follow.

9.7.3 Phase 3: KVM Backend and Tooling (Series 5–7)

Series 5 (the `kvm-v2` backend), Series 6 (`umlctl` and `kselftests`), and Series 7 (documentation) depend on Series 4. They are submitted after the backend-ops abstraction is accepted or has received sufficient positive review signal. The cover letter for Series 5 includes the complete benchmark data (Appendix .5), the `umlctl` mission acceptance gate results, and instructions for reviewers to reproduce the validation locally.

9.7.4 The LKML Strategy

The submission targets the `linux-um` mailing list with CC to relevant subsystem maintainers. Key considerations:

- AI-assisted development is disclosed per the kernel’s `coding-assistants.html` policy [15].
- Each series carries a `Signed-off-by` from the human author and a `Co-authored-by` from the AI tool.
- The submission queue document (`upstream-patches/SUBMISSION-QUEUE.md`) tracks the status of each series.

9.8 Lessons Learned

Several lessons emerged from the development process that may be useful to future large-scale kernel rearchitecture efforts.

1. The backend contract must be explicit and tested. The single most valuable architectural decision was defining `struct um_backend_ops` early and writing contract tests (`arch/um/backend/contract/`) that verify each backend against the same behavioral specification. Every subsequent bug was diagnosed faster because the contract narrowed the search space.

2. Substrate parity is the right acceptance criterion. Using CPython’s test suite as the primary acceptance criterion—rather than microbenchmarks or unit tests alone—caught bugs that no synthetic test would have found. The CPython teardown race (Section 9.6) is the most dramatic example, but subtler issues in signal delivery, mmap semantics, and dynamic loader behavior were also caught by substrate parity testing.

3. State audits prevent regression spirals. The v1 KVM backend was retired because of accumulated state ownership confusion between the UML kernel and the KVM host. The v2 design was informed by a formal state audit (`02-workstreams/D-kvm-backend/state-audit/`) that mapped every piece of shared state, its owner, its synchronization mechanism, and its failure mode. This audit prevented the same class of bugs from recurring.

4. The investigation discipline pays for itself. The project memory records a discipline: “run the cheapest dispositive control FIRST; never declare ‘architectural’ or ship userspace workarounds without it.” Every time this discipline was followed, it saved hours. Every time it was violated (the pool physmem isolation episode, the SIGALRM/mmap-FIXED deadlock), it cost days.

5. Compress early, compress often. The 1,236 development commits exist because iterative development requires iterative commits. But for upstream submission, only the logical decomposition matters. The rebase from 1,236 commits to 7 series was done once, at the end, when the design was stable. Attempting to maintain a clean commit history during active development would have been a net drag on velocity.

Chapter 10

Conclusion and Future Work

10.1 Summary of Contributions

This report has presented the v2 redesign of User-Mode Linux, the most significant rearchitecture of the UML subsystem since Jeff Dike’s original implementation in 2000 [1]. The contributions span architecture, implementation, validation, tooling, and documentation:

1. **Backend operations abstraction.** `struct um_backend_ops` decouples the UML kernel from any single syscall-interposition mechanism, enabling pluggable backends with a tested behavioral contract.
2. **The `kvm-v2` backend.** A complete KVM-based execution backend that replaces the `ptrace/seccomp` signal-mediated syscall path with `VMRESUME/VMEXIT`. The LSTAR-gadget fast path achieves ~ 90 -cycle syscall latency, a $3\text{--}4\times$ improvement over `seccomp` on real workloads and up to $900\times$ on microbenchmarks.
3. **SMP support.** Multi-processor execution with per-vCPU dispatch, IPI emulation, TLB shootdown, deferred-free TLB batching, and memory barrier semantics.
4. **Snapshot and record/replay.** KVM-aware snapshot ($38.9\mu\text{s}$ capture, $13\mu\text{s}$ restore) and a record/replay state machine with static-key gating, enabling deterministic debugging and fast-restart fuzzing workflows.
5. **Composable instrumentation.** KASAN, KMSAN, KCSAN, KFENCE, KCOV, kprobes, ftrace, and BPF JIT, all selectable through nine named profiles from a single tree. Runtime-flippable hooks via static keys allow post-failure observability without rebuilding.
6. **Comprehensive validation.** 53 `kseltest` suites, the `umlctl` mission six-phase acceptance gate, CPython substrate parity (21/21 on both backends), and cross-host benchmarking on eight machines.
7. **`umlctl`: a Rust launcher.** A 22,000-line Rust CLI that reduces the barrier from a page of shell commands to a single invocation, with declarative deployment, `cgroup v2` management, and reproducible acceptance testing.
8. **Documentation.** 134,500 lines of operator documentation, redesign planning materials, and upstream submission notes.

The seven series-level commits on `torvalds/master` (v7.1-rc5+) represent a cohesive body of work: each series is independently reviewable and bisectable, and the complete set brings UML from a largely unmaintained architecture port to a viable platform for kernel research, debugging, fuzzing, and sandboxed execution.

10.2 Current State (May 2026)

As of the reporting cutoff, the following capabilities are operational:

Both backends pass the full CPython test suite.

The substrate parity gate produces identical results on `kvm-v2` and `seccomp`: PASS=25, FAIL=3, XFAIL=3. The three failures are known UML limitations (not regressions), and the three expected failures are test-side issues unrelated to the backend.

`kvm-v2` is 3–4× faster on real workloads.

The Python startup benchmark shows 2.99–4.06× speedup over `seccomp` across eight host types. The SMP stress tier shows 3.71–8.72× speedup. Microbenchmarks show 193–908× speedup, reflecting the elimination of the host-exit round-trip for gadget-covered syscalls.

SMP works.

The `mt-mini` stress test runs with T=8 threads on `ncpus=4` virtual CPUs with strict memory verification and zero failures.

Snapshot and record/replay are implemented.

The KVM-aware snapshot surface captures in 38.9µs and restores in 13µs median. The record/replay state machine logs SYSCALL, RDTSC, and SIGALRM events with static-key gating. KUnit suites cover 3/3 snapshot cases and 7/7 record/replay cases.

`umlctl` mission passes 6/6 phases.

The canonical acceptance gate completes in ~15 minutes with 100/100 soak iterations passing, zero panics, and a JSON scoreboard for audit.

10.3 What Remains

Despite the breadth of the current implementation, several significant work items remain before the redesign can be considered complete.

10.3.1 Phase J: 24-Hour Continuous Soak

The `umlctl` mission gate provides 15-minute validation with 100 soak iterations. The 24-hour long-tail soak—running thousands of iterations across all workload templates—is the remaining wall-clock evidence needed for milestone candidates. The rig is in place; the remaining cost is compute time.

10.3.2 Upstream Acceptance

The seven series are drafted and validated, but none has been submitted to LKML. The submission strategy (Section 9.7) is staged: small independent patches first, the tentpole RFC second, and the KVM backend last. The primary risk is review bandwidth: the backend-ops abstraction touches core UML code and requires buy-in from the UML maintainer and the broader `linux-arch` community.

10.3.3 Architecture Ports

The v2 redesign targets `x86_64` exclusively. Two additional architecture ports are planned:

ARM64 (`aarch64`).

ARM64 KVM support is mature, and the backend-ops abstraction is architecture-neutral by design. The primary work is implementing the ARM64-specific portions of the `kvm-v2` backend: exception vector setup, register marshaling, and the equivalent of the LSTAR gadget (using `HVC` or `SMC` instructions for fast-path trapping).

RISC-V.

RISC-V KVM support is upstream and stabilizing. The port follows the same pattern as ARM64, with RISC-V-specific trap handling via the `ecall` mechanism.

10.3.4 syzkaller Integration

The in-tree forkserver surface is pinned (snapshot v1, contract C-09). The remaining work is the `pkg/vm/uml` Go driver in the syzkaller repository [9], which maps syzkaller’s VM management API to UML instance lifecycle. This is an external contribution, not a kernel-tree change.

10.3.5 Record/Replay Host-Side Wiring

The record/replay log shape, observe/consume API, and KUnit suite are proven. The remaining work is the host-side signal-interception path that converts a real `SIGALRM` into an `observe_sigalrm` call, plus the trap-on-RDTSC VMCS programming for deterministic timestamp counters.

10.3.6 Additional Validation

- **LTP curation.** A curated KEEP/SKIP list and UML-appropriate runner refinements for the Linux Test Project.
- **CPUID feature un-masking.** AVX-512, AMX, PKRU, and CET un-masking following the SMP-T77 checklist.
- **Snapshot ELF64-core export.** Lifting snapshot data to a `gdb/crash-tool-loadable` artifact.

10.4 The Broader Impact

UML occupies a unique position in the virtualization landscape—one that no other technology can fill.

10.4.1 Full Kernel, Full Debugging, No Hardware

Every competing approach sacrifices at least one of these three properties:

- KVM/QEMU provides a full kernel and hardware acceleration, but debugging requires GDB stubs, serial consoles, or JTAG probes.
- gVisor provides sandboxing without hardware requirements, but it is not the Linux kernel—semantic divergences are inherent.
- Containers provide the real kernel with low overhead, but they share the host kernel and cannot isolate kernel bugs.
- LKL provides the real kernel code at library-call speed, but sacrifices process isolation and scheduling fidelity.
- Unikernels sacrifice multi-process semantics entirely.

UML is the only technology that provides all three: the complete Linux kernel, running with full `gdb` debuggability, without requiring hardware virtualization extensions. When the host does provide `/dev/kvm`, the `kvm-v2` backend makes UML competitive in performance while retaining the debugging story.

10.4.2 The Cloud-Native and eBPF Context

The rise of eBPF [42] as a kernel programmability mechanism has created a new class of kernel developers who write BPF programs but rarely compile or debug the kernel itself. UML can serve as the bridge: a BPF developer can run a UML instance, load their programs, observe the kernel’s internal state with kprobes and ftrace, and diagnose issues that are invisible from the eBPF toolchain alone.

Similarly, the cloud-native ecosystem increasingly relies on kernel features (cgroups, namespaces, seccomp, io_uring) without direct access to the kernel development workflow. UML provides that access in a form factor that fits a CI/CD pipeline: compile, boot, test, and tear down in seconds, without a VM, without hardware access, without root privileges.

10.4.3 Security and Reproducibility

The security research community needs two things that are difficult to combine: a realistic attack surface and a controlled execution environment. UML provides both. The attack surface is real—it is the Linux kernel’s syscall interface—and the environment is controlled: deterministic time, record/replay, and snapshot/restore allow a researcher to reproduce and analyze an exploit as many times as needed. No serial console, no JTAG probe, no hardware testbed.

10.5 Closing Thoughts

UML was created 26 years ago to solve a simple problem: make the Linux kernel debuggable. For a time, it was the only way to run a kernel under `gdb`. Then KVM arrived, then containers, then gVisor, then eBPF, and UML faded into maintenance mode. The underlying problem—making the kernel observable, testable, and debuggable without dedicated hardware—never went away. It just acquired more competitors, none of which fully solve it.

The window for UML’s revival is open now, and it is not large.

The Linux Kernel Library [6] demonstrates that a library-compiled kernel can achieve extraordinary fuzzing throughput. Rust VMM crates (`rust-vmm`, `crosvm` [53], Cloud Hypervisor [52]) are making it trivial to build custom hypervisors. eBPF [42] is absorbing an increasing share of the kernel observability use case. If UML does not catch up—if it remains a curiosity with a `ptrace`-era performance profile and no modern tooling—these technologies will collectively eat the niche.

The v2 redesign is an attempt to close the gap before it becomes permanent. The backend-ops abstraction makes UML extensible. The `kvm-v2` backend makes it fast. The profile system makes it composable. The snapshot and record/replay infrastructure make it useful for fuzzing and debugging. The `umlctl` launcher makes it accessible.

Whether this attempt succeeds depends on upstream acceptance. The code is written, the tests pass, the benchmarks are favorable, and the documentation is comprehensive. What remains is the social process of kernel review: demonstrating to the UML maintainer and the broader kernel community that this work is worth maintaining, that the backend-ops contract is the right abstraction, and that the `kvm-v2` backend justifies its complexity.

The answer to “why UML?” has not changed in 26 years: because the best way to understand a kernel is to run it where you can poke at it. The v2 redesign simply ensures that the poking is fast enough, instrumented enough, and accessible enough to be worth doing in 2026.

Bibliography

- [1] Jeff Dike. “A User-mode Port of the Linux Kernel”. In: *Proceedings of the 4th Annual Linux Showcase and Conference (ALS)*. Atlanta, GA: USENIX Association, 2000. url: https://www.usenix.org/legacy/publications/library/proceedings/als00/2000papers/papers/full_papers/dike/dike_html/.
- [2] Gerald J. Popek and Robert P. Goldberg. “Formal Requirements for Virtualizable Third Generation Architectures”. In: *Communications of the ACM* 17.7 (1974), pp. 412–421. doi: [10.1145/361011.361073](https://doi.org/10.1145/361011.361073).
- [3] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. “Xen and the Art of Virtualization”. In: *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2003, pp. 164–177. doi: [10.1145/945445.945462](https://doi.org/10.1145/945445.945462).
- [4] *Firecracker MicroVM*. <https://firecracker-microvm.github.io/>. Project documentation.
- [5] *gVisor: Container Runtime Sandbox*. <https://gvisor.dev/>. Google project.
- [6] *LKL: Linux Kernel Library*. <https://github.com/lkl/linux>. Project repository.
- [7] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. “kvm: the Linux Virtual Machine Monitor”. In: *Proceedings of the Linux Symposium*. Vol. 1. 2007, pp. 225–230.
- [8] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. “Firecracker: Lightweight Virtualization for Serverless Applications”. In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2020, pp. 419–434.
- [9] *syzkaller: kernel fuzzer*. <https://github.com/google/syzkaller>. Project repository.
- [10] Sergej Schumilo, Cornelius Aschermann, Robert Gawlik, Sebastian Schinzel, and Thorsten Holz. “kAFL: Hardware-Assisted Feedback Fuzzing for OS Kernels”. In: *Proceedings of the 26th USENIX Security Symposium*. 2017, pp. 167–182.
- [11] Maurizio Pizzonia and Massimo Rimondini. “Netkit: Network Emulation for Education”. In: *Software: Practice and Experience* 38.2 (2008), pp. 151–171. doi: [10.1002/spe.825](https://doi.org/10.1002/spe.825).
- [12] *Marionnet: a virtual network laboratory*. <https://www.marionnet.org/>. Uses UML for network simulation.
- [13] *Introducing Systrap: a fast gVisor platform*. <https://gvisor.dev/blog/2023/04/28/systrap-release/>. gVisor blog, April 2023.
- [14] Octavian Purdila, Lucian Adrian Grijincu, and Nicolae Tapus. “LKL: The Linux Kernel Library”. In: *Proceedings of the 9th RoEduNet IEEE International Conference*. 2010, pp. 328–333. doi: [10.1109/ROEDUNET.2010.5541417](https://doi.org/10.1109/ROEDUNET.2010.5541417).
- [15] *Linux Kernel Documentation: Coding Assistants*. <https://docs.kernel.org/process/coding-assistants.html>. Linux kernel documentation, April 2026.
- [16] Adam Belay, Andrea Bittau, Ali Mashtizadeh, David Terei, David Mazières, and Christos Kozyrakis. “Dune: Safe User-level Access to Privileged CPU Features”. In: *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2012, pp. 335–348.

- [17] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. “Unikernels: Library Operating Systems for the Cloud”. In: *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2013, pp. 461–472. doi: [10.1145/2451116.2451167](https://doi.org/10.1145/2451116.2451167).
- [18] Linus Torvalds. *Free minix-like kernel sources for 386-AT*. comp.os.minix Usenet posting. Oct. 1991.
- [19] Jeff Dike. “User-mode Linux”. In: *Proceedings of the 5th Annual Linux Showcase and Conference*. USENIX Association, 2001.
- [20] Jeff Dike. *SKAS4—UML’s New Execution Engine*. Ottawa Linux Symposium. 2006.
- [21] Jeff Dike. *Making UML Ready for the Mainstream*. Ottawa Linux Symposium. 2003.
- [22] Jeff Dike. *User Mode Linux*. The definitive guide to UML by its creator. Prentice Hall, 2006. isbn: 978-0131865051.
- [23] Intel® *Virtualization Technology (VT-x)*. <https://www.intel.com/content/www/us/en/virtualization/virtualization-technology/intel-virtualization-technology.html>. First released 2005 on Pentium 4 662/672. Intel Corporation.
- [24] *AMD-V: AMD Virtualization (SVM)*. AMD64 Architecture Programmer’s Manual, Volume 2, Ch. 15. First released 2006 on Athlon 64. Advanced Micro Devices.
- [25] Fabrice Bellard. “QEMU, a Fast and Portable Dynamic Translator”. In: *Proceedings of the USENIX Annual Technical Conference (FREENIX Track)*. 2005, pp. 41–46.
- [26] Stephen Soltesz, Herbert Pötzl, Marc E. Fiuczynski, Andy Bavier, and Larry Peterson. “Container-based Operating System Virtualization: A Scalable, High-Performance Alternative to Hypervisors”. In: *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems*. 2007, pp. 275–287. doi: [10.1145/1272996.1273025](https://doi.org/10.1145/1272996.1273025).
- [27] Michael Kerrisk. *Namespaces in operation*. <https://lwn.net/Articles/531114/>. LWN.net series, 2013. 2013.
- [28] *SECCOMP_BPF: Seccomp filter*. https://docs.kernel.org/userspace-api/seccomp_filter.html. Linux kernel documentation.
- [29] Hajime Tazaki. *Unify LKL into UML*. <https://lore.kernel.org/lkml/>. LKML RFC series, v8, 2019. 2019.
- [30] Robert P. Goldberg. “Architecture of Virtual Machines”. In: *AFIPS Conference Proceedings* 42 (1973), pp. 309–318.
- [31] James E. Smith and Ravi Nair. *Virtual Machines: Versatile Platforms for Systems and Processes*. Morgan Kaufmann, 2005. isbn: 978-1558609105.
- [32] Keith Adams and Ole Agesen. “A Comparison of Software and Hardware Techniques for x86 Virtualization”. In: *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2006, pp. 2–13. doi: [10.1145/1168857.1168860](https://doi.org/10.1145/1168857.1168860).
- [33] *Intel® 64 and IA-32 Architectures Software Developer’s Manual*. Volume 3: System Programming Guide. Intel Corporation. 2024.
- [34] *AMD64 Architecture Programmer’s Manual*. Volume 2: System Programming. Advanced Micro Devices. 2024.
- [35] *ptrace(2) — Linux manual page*. <https://man7.org/linux/man-pages/man2/ptrace.2.html>.
- [36] Ulrich Drepper. *Futexes Are Tricky*. <https://dept-info.labri.fr/~denis/Enseignement/2008-IR/Articles/01-futex.pdf>. 2011.

- [37] *KVM API Documentation*. <https://docs.kernel.org/virt/kvm/api.html>. Linux kernel documentation.
- [38] *KASAN: Kernel Address Sanitizer*. <https://docs.kernel.org/dev-tools/kasan.html>. Linux kernel documentation.
- [39] *KMSAN: Kernel Memory Sanitizer*. <https://docs.kernel.org/dev-tools/kmsan.html>. Linux kernel documentation.
- [40] *KCSAN: Kernel Concurrency Sanitizer*. <https://docs.kernel.org/dev-tools/kcsan.html>. Linux kernel documentation.
- [41] *gVisor Architecture Guide: Platforms*. https://gvisor.dev/docs/architecture_guide/platforms/. gVisor documentation.
- [42] *eBPF*. <https://ebpf.io/>. Extended Berkeley Packet Filter documentation.
- [43] Dmitry Vyukov. “syzkaller: an unsupervised, coverage-guided, fuzzer for Linux kernel”. In: Google internal presentation; public syzkaller repo. 2015.
- [44] *rr: lightweight recording & deterministic debugging*. <https://rr-project.org/>. Mozilla project.
- [45] George W. Dunlap, Samuel T. King, Sukru Cinar, Murtaza Basrai, and Peter M. Chen. “ReVirt: Enabling Intrusion Analysis through Virtual-Machine Logging and Replay”. In: *Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI)*. 2002, pp. 211–224.
- [46] *Control Group v2*. <https://docs.kernel.org/admin-guide/cgroup-v2.html>. Linux kernel documentation.
- [47] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Le Feuvre, Sharan Santhanam, Alexander Jung, Gauthier Gain, Cyril Soldani, Costin Lupu, Ștefan Rusu, Felipe Huici, Andrii Kenber, Wei Chen, Stefan Teodorescu, Cătălin Radu, Costin Raiciu, and Pierre Olivier. “Unikraft: Fast, Specialized Unikernels the Easy Way”. In: *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys)*. 2021. doi: [10.1145/3447786.3456248](https://doi.org/10.1145/3447786.3456248).
- [48] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Rolf Neugebauer, and Galen Hunt. “Rethinking the Library OS from the Top Down”. In: *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Drawbridge / Bascule. 2011, pp. 291–304. doi: [10.1145/1950365.1950399](https://doi.org/10.1145/1950365.1950399).
- [49] Chia-Che Tsai, Kumar Saurabh Arora, Nehal Bandi, Bhushan Jain, William Jannen, Jitin John, Harry A. Kalodner, Vrushali Kulkarni, Daniela Oliveira, and Donald E. Porter. “Cooperation and Security Isolation of Library OSes for Multi-Process Applications”. In: *Proceedings of the 9th European Conference on Computer Systems (EuroSys)*. Graphene. 2014. doi: [10.1145/2592798.2592812](https://doi.org/10.1145/2592798.2592812).
- [50] *Gramine Library OS*. <https://gramineproject.io/>. Successor to Graphene; SGX support.
- [51] *Windows Subsystem for Linux Overview*. <https://learn.microsoft.com/en-us/windows/wsl/about>. Microsoft documentation; WSL1 used syscall translation, WSL2 uses Hyper-V VM.
- [52] *Cloud Hypervisor*. <https://www.cloudhypervisor.org/>. Open-source VMM for cloud workloads.
- [53] *crosvm Book: Architecture Overview*. <https://crosvm.dev/book/architecture/overview.html>. Chrome OS virtual machine monitor.

Source Code Map

This appendix provides a comprehensive map of the UML source tree as of the v2 redesign (branch `next`, May 2026). The tree spans three primary locations in the kernel source: the architecture port (`arch/um/`), the tooling directory (`tools/uml/`), and the kselftest suite (`tools/testing/selftests/um/`).

.1 Overall Statistics

Table 1: Source tree summary statistics.

Tree	Files	Lines	Description
<code>arch/um/</code>	352	89,248	Architecture port
<code>tools/uml/</code>	101	28,857	Host-side tooling (excl. build artifacts)
<code>tools/testing/selftests/um/</code>	316	142,474	Kselftest suites
Total	769	260,579	

.2 Architecture Port: `arch/um/`

Table 2: Directory structure of `arch/um/`.

Directory	Files	Lines	Description
<code>arch/um/</code> (top-level)	9	942	Makefile, Kconfig, defconfig entry points.
<code>arch/um/backend/</code>	3	16	Backend layer root: Makefile and Kconfig for backend selection.
<code>arch/um/backend/common/</code>	4	18	Shared backend utilities (Makefile, Kconfig stubs).
<code>arch/um/backend/contract/</code>	3	1,886	KUnit behavioral contract tests that verify each backend against the same specification.
<code>arch/um/backend/kvm-v1-archive/</code>	14	12,600	Preserved v1 KVM backend (retired) for reference and diffing against v2.
<code>arch/um/backend/kvm-v2/</code>	25	18,296	Complete kvm-v2 backend: <code>vcpu.c</code> (vCPU lifecycle, <code>KVM_RUN</code> dispatch), <code>syscall_trap.c</code> (IDT exception dispatch), <code>exception.c</code> (IDT/GDT/TSS setup), <code>context.c</code> (VM lifecycle, memslots), <code>lstar_gadget.S</code> (in-guest SYSCALL trampoline), <code>snapshot.c</code> , <code>record.c</code> , KUnit suites.

Continued on next page

Directory	Files	Lines	Description
arch/um/backend/seccomp/	11	562	Seccomp backend implementation (refactored through um_backend_ops).
arch/um/configs/	3	224	Base defconfigs for ARCH=um.
arch/um/configs/profiles/	10	552	Named profile defconfig fragments: prod-fast, prod-with-hooks, research, fuzz, fuzz-deep, sandbox, library, embedded, time-travel.
arch/um/drivers/	85	25,125	UML device drivers: ubd (block), net (virtual networking), mconsole (management console), port (serial ports), harddog (watchdog), vector (vector I/O), virt-pci (virtio PCI transport), rtc (real-time clock).
arch/um/include/asm/	55	3,120	Architecture-specific headers: pgtable.h (PTE layout, x86-aligned), mmu_context.h (mm switching), processor.h (thread state), tlbfush.h (TLB invalidation API).
arch/um/include/asm/fpu/	1	22	FPU state management stubs.
arch/um/include/asm/trace/	1	720	Tracepoint definitions for backend events.
arch/um/include/linux/	3	126	UML-specific internal Linux headers.
arch/um/include/shared/	25	2,031	Headers shared between kernel-side and os-side code: backend.h (struct um_backend_ops definition), os.h (host OS interface), kern_util.h (kernel utility declarations).
arch/um/include/shared/skas/	3	208	SKAS-mode shared structures: process/mm state, stub page definitions.
arch/um/include/uapi/asm/	1	1	UAPI header stubs.
arch/um/kernel/	52	12,051	Core kernel: trap.c (fault handling, CPython segv intercept), process.c (context switch), smp.c (SMP/IPI), tlb.c (TLB sync), syscall.c (syscall dispatch), time.c (timer/clock), signal.c (signal delivery), mem.c (memory init).
arch/um/kernel/kprobes/	2	398	Kprobes architecture support for UML.
arch/um/kernel/skas/	11	1,720	SKAS-mode kernel code: process creation, clone, mmu context, stub lifecycle.
arch/um/os-Linux/	25	7,398	Host-side OS interface: process.c (os_map_memory, resource controls), main.c (host-side os_main), signal.c (signal setup), sigio.c (async I/O polling), file.c (host file ops), mem.c (physmem mmap).

Continued on next page

Directory	Files	Lines	Description
arch/um/os-Linux/skas/	5	1,205	SKAS-mode host-side code: stub execution, process tracing, memory management.
arch/um/scripts/	1	27	Build scripts (Makefile.rules).
Total arch/um/	352	89,248	

.3 Host-Side Tooling: tools/uml/

Table 3: Directory structure of tools/uml/ (excluding build artifacts).

Directory	Files	Lines	Description
tools/uml/ (top-level)	1	60	Top-level Makefile.
tools/uml/uml-launcher/	5	186	Cargo.toml, Cargo.lock, build configuration.
tools/uml/uml-launcher/src/	6	1,432	Library root: lib.rs, config parsing, instance management, error types.
tools/uml/uml-launcher/src/backend/	6	3,307	Backend detection and management: KVM availability probing, seccomp detection, backend selection logic.
tools/uml/uml-launcher/src/bin/umlctl/	27	14,596	The umlctl binary: subcommands for create, start, stop, ps, logs, metrics, up, build, mission. Key modules include mission.rs (6-phase acceptance gate), cgroup.rs (cgroup v2 management), preflight.rs (resource verification).
tools/uml/uml-launcher/src/bin/umlbuild/	10	2,281	The umlbuild binary: kernel compilation with profile selection and output directory management.
tools/uml/uml-launcher/profiles/	5	450	TOML profile definitions mapping named profiles to Kconfig fragments and backend defaults.
tools/uml/uml-launcher/examples/	23	1,641	Example Umlfiles (TOML deployment specifications) demonstrating various configurations.
tools/uml/uml-launcher/scripts/	2	1,237	Helper scripts for rootfs preparation and environment setup.
tools/uml/uml-gdb/	1	366	GDB helper scripts for UML kernel debugging.
tools/uml/syzkaller-vm-shim/	2	—	Placeholder for the syzkaller vm/uml shim.

Directory	Files	Lines	Description
tools/uml/diag/	6	315	Diagnostic tools developed during debugging: round-8 MMU audit, round-9 CPython cache dump, round-13 bytecode monitor.
Total tools/uml/ (source only)	101	28,857	

.4 Kselftest Suite: tools/testing/selftests/um/

The kselftest directory contains 53 test suites spanning the full validation ladder. Table 4 groups them by category.

Table 4: Kselftest suites in tools/testing/selftests/um/.

Suite	Files	Lines	Description
Substrate Parity			
cpython-parity/	3	214	21-module CPython parity gate (both back-ends).
cpython-tier0/	3	184	Minimal CPython smoke test.
cpython-full/	3	223	Full CPython 3.14 regrtest (~400 modules).
KVM Backend Tests			
kvm-smoke/	2	131	Basic kvm-v2 boot and syscall.
kvm-isolation/	5	867	Memory isolation between guest and host.
kvm-bounds/	3	344	Memory bounds checking.
kvm-mm-smoke/	3	371	Memory management operations under KVM.
Snapshot and Record/Replay			
snapshot-smoke/	4	371	Snapshot v1 lifecycle test.
snapshot-kvm-smoke/	2	123	KVM-aware snapshot (v2) smoke.
snapshot-elf-roundtrip/	3	255	ELF-core export round-trip.
kvm-snapshot-bench/	2	114	Capture/restore latency benchmark.
kvm-record-smoke/	2	128	Record/replay v2 KUnit integration.
kvm-record-clock-bench/	2	126	RDTSC record overhead benchmark.
SMP and Concurrency Stress			
mt-mmap-stress/	26	24,905	Multi-threaded mmap stress with strict memory verification (mt-mini).
fork-tree-3level/	5	2,547	Three-level fork tree with threaded-fork-malloc stress.
template-pause-smoke/	2	459	Template/pause snapshot lifecycle.
template-pause-fork-smoke/	2	206	Fork from paused template.

Continued on next page

Suite	Files	Lines	Description
template-pause-fork-stress/	3	950	Stressed fork from paused template.
template-pause-pivot-smoke/	2	178	Pivot-root from paused template.
template-pause-pool-*	4	641	Pool member lifecycle and sustained operation.
Performance			
bench/	8	679	Cross-host micro/py/stress benchmark harness.
perf-getpid/	5	480	RDTSC-bracketed getpid latency.
perf-py-startup/	1	132	Python startup wallclock.
perf-pidfam/	3	332	PID-family creation cost.
perf-fallback/	3	400	Ptrace fallback performance.
pool-bench/	2	624	Instance pool throughput.
net-bench/	7	640	Network I/O throughput.
ubd-bench/	3	499	Block device (ubd) I/O throughput.
Instrumentation			
ftrace-smoke/	3	174	Ftrace basic function tracing.
kmsan-smoke/	3	161	KMSAN shadow validation.
kprobes-stress/	3	255	Kprobes attach/detach stress.
hooks-flip/	3	157	Static-key hook enable/disable.
Soak and Sustained Validation			
soak/	25	8,773	11 TOML-templated soak workloads: memcheck, ioccheck, stress-ng, tier1-pylibs, django-loopback, and more. Python orchestrator with Wilson CI scoring.
Process and Syscall			
dyn-loader/	2	177	Dynamic loader interposition.
rt-sigreturn-isolation/	5	2,772	rt_sigreturn signal-frame isolation.
userspace-smoke/	3	245	Basic userspace exec smoke.
transparency/	1	165	Guest transparency checks.
Tooling and Integration			
launcher-smoke/	4	388	umlctl launcher integration.
umlctl-smoke/	2	556	umlctl subcommand smoke.
umlbuild/	2	129	umlbuild compilation test.
profiles/	3	357	Profile build and boot validation.
gates/	7	332	Gate framework validation.
Regression Reproducers			
regptest-repros/	6	495	Classified reproducers: env (A), process (B), syscall (C), structural (D).

Continued on next page

Suite	Files	Lines	Description
cve-repro/	3	305	CVE reproduction harnesses.
mmap-fixed-sigalrm-repro/	2	211	SIGALRM/mmap-FIXED deadlock reproducer.
Miscellaneous			
df-preserve/	3	353	Filesystem state preservation.
pool-exec-smoke/	2	233	Pool exec lifecycle.
pool-port-forward-smoke/	2	184	Pool port forwarding.
pool-serve-smoke/	2	248	Pool HTTP serve test.
pool-spawn-smoke/	2	159	Pool spawn lifecycle.
state-trace/	1	513	State-trace analysis tool.
vector2-sandbox-audit/	2	183	Vector2 network sandbox audit.

.5 File Type Distribution

Table 5: File type distribution across the v2 source tree.

Type	Primary Location	Role
C source (.c)	arch/um/, backends	Kernel-side implementation
C headers (.h)	arch/um/include/	Interface definitions
Assembly (.S)	backend/kvm-v2/	LSTAR gadget, stub pages
Rust (.rs)	tools/uml/uml-launcher/	Host-side tooling
TOML (.toml)	tools/uml/, selftests	Config, profiles, soak templates
Shell (.sh)	Selftests	Test orchestration
Python (.py)	Selftests	Soak orchestrator, analysis
Kconfig	arch/um/	Build configuration
Makefile	Throughout	Build rules

Benchmark Data

This appendix presents the complete benchmark data and methodology underlying the performance claims in Chapters 9 and 10. All measurements were taken on the `mjbommar/linux` tree (branch `next`), kernel commit `9ccbf5e7bb58`, with the host CPU governor set to `performance`. Hosts span Skylake-S through Zen 4 and Alder Lake.

.6 Methodology

The benchmark suite uses three tiers, each measuring a different aspect of backend performance:

Micro tier.

A tight `getpid()` loop bracketed by `RDTSC`. Measures pure trap-mechanism cost: the round-trip from guest userspace through the backend to the UML kernel and back. This tier is dominated by the syscall interception mechanism and is insensitive to kernel processing time.

Application tier (Python startup).

A canned Python interpreter startup with a mixed-syscall workload (`python3 -c "import hashlib"`). Exercises `mmap`, `mprotect`, `open`, `read`, `fstat`, `brk`, process creation, dynamic linking, and C-extension loading.

Stress tier (mt-mini).

An SMP stress workload with `T=8` threads on `ncpus=4`, performing concurrent `mmap/munmap/mprotect` operations with strict memory verification after each iteration. Measures both backend performance and SMP correctness under concurrency pressure.

The benchmark unit is speedup over `seccomp` on the same host. This ratio cancels most CPU-frequency and microarchitecture noise and ties the claim to a backend decision rather than a hardware selection.

.7 Per-Syscall Latency

Table 6 summarizes the round-trip cost of a single `getpid()` invocation under each backend mechanism, contextualized against a native Linux host syscall baseline and the LKL direct-call reference.

Table 6: Per-syscall (`getpid`) round-trip latency by backend.

Backend	Mode	Cycles	Approx. ns	Notes
Native host syscall	ring 0 entry	~150	~50	Baseline (not UML)
kvm-v2 + gadget	In-guest LSTAR	~90	~25	No VMEXIT; 10 trivial NRs
kvm-v2 (exit path)	KVM_EXIT_IO	~36,800	~150	IO-port exit path
seccomp	SIGSYS + futex	~110,000	300–500	Default backend (6.16+)
ptrace (archived)	PTRACE_SYSEMU	3,000–15,000	1,000–5,000	Removed in memo 25 R11
LKL (direct call)	function call	~18	~6	No ring split; not comparable

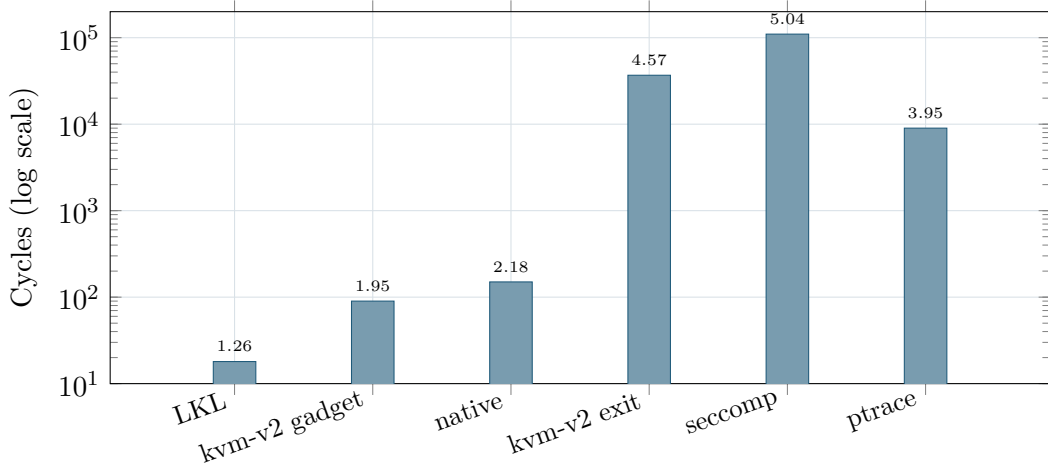


Figure 1: Per-syscall round-trip cost comparison (log scale, cycles). The LSTAR gadget achieves $\sim 1,200\times$ improvement over the `seccomp` backend for trivial syscalls. LKL is shown for reference but sacrifices ring-split fidelity.

The LSTAR gadget. The $\sim 25\text{ ns}$ / $\sim 90\text{-cycle}$ result for `kvm-v2` with the LSTAR gadget warrants explanation. The gadget is an in-guest `SYSCALL` handler installed at the LSTAR MSR address. When the guest executes `SYSCALL`, the CPU dispatches to the gadget page *without* a `VMEXIT`. The gadget performs fast-path handling (for trivial syscalls like `getpid`) and returns to guest userspace via `SYSRET`. Only syscalls requiring kernel-side processing trigger a `VMEXIT`. This achieves sub-native latency because the native host syscall traverses speculative-execution mitigations (KPTI, retpoline) that the in-guest gadget does not need.

.8 Cross-Host Benchmark Results

Table 7 reports the complete cross-host benchmark results across all three tiers.

Table 7: Post-gadget `kvm-v2` speedup over `seccomp` across 8 hosts.

Host	CPU	Micro	Python	Stress
s0	Intel Core i9-12900K	284×	3.73×	7.26×
s1	Intel Xeon E3-1225 v6	743×	3.05×	3.71×
s2	Intel Xeon E3-1225 v5	813×	2.99×	3.94×
s3	Intel Xeon W-2123	707×	3.14×	4.16×
s4	Intel Core i5-12600K	193×	2.99×	8.72×
s5	AMD Ryzen 7 7840HS	861×	3.87×	4.30×
s6	AMD Ryzen 7 7840HS	908×	4.06×	4.32×
s7	AMD Ryzen 7 7840HS	904×	3.91×	4.89×
Range		193–908×	2.99–4.06×	3.71–8.72×

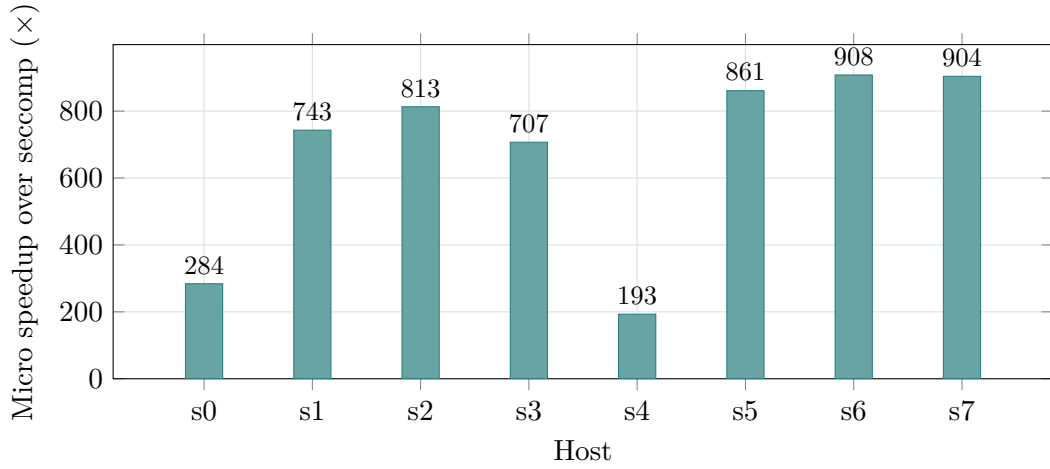


Figure 2: Micro-tier (getpid-loop) speedup. The variation across hosts reflects differences in seccomp notification overhead rather than KVM performance: the `kvm-v2` gadget cost is consistently 87–153 cycles, while seccomp costs vary from $\sim 15,000$ to $\sim 140,000$ cycles by host.

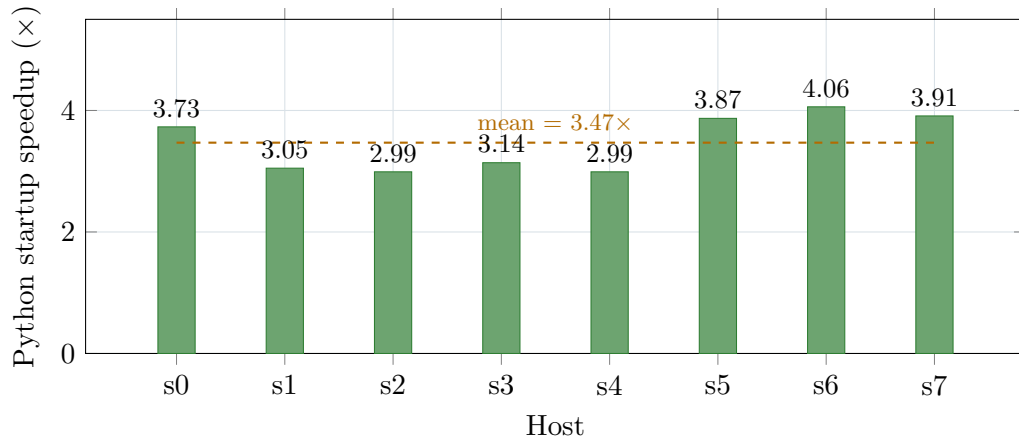


Figure 3: `kvm-v2` Python startup speedup over seccomp. The consistent 3–4 $\times$ band across diverse hardware confirms an architectural (backend mechanism) improvement, not a microarchitecture-specific one.

.8.1 Micro Tier

.8.2 Application Tier (Python Startup)

.8.3 Stress Tier

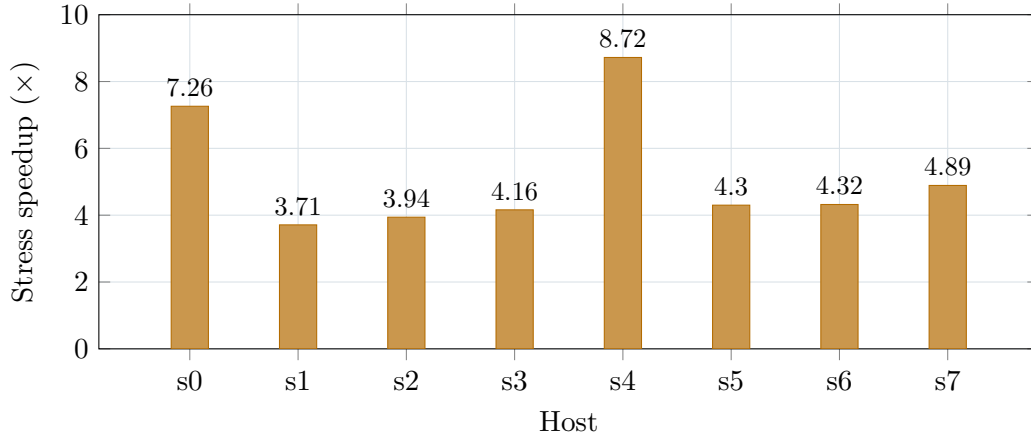


Figure 4: Stress tier (mt-mini SMP T=8, ncpus=4) speedup. The wider variance (3.71–8.72 $\times$) reflects the mmap/page-fault-heavy workload where TLB shutdown cost and host scheduler behavior create per-host variation. Zero strict memory failures on any host.

The stress tier combines performance with correctness. Each mt-mini iteration allocates memory, writes patterns, forks or spawns threads, and verifies memory content after concurrent modification. A single bit-flip constitutes a test failure. The 3.71–8.72 $\times$ speedup comes from the broader vCPU and SMP design, not solely the gadget.

.9 Snapshot Performance

The `kvm-v2` snapshot benchmark exercises `kvm_v2_snapshot_alloc/capture/restore` with $N = 64$ iterations of capture-once, restore- N -times.

Table 8: Snapshot benchmark (kernel `9ccbf5e7bb58`, AMD Ryzen 7 7840HS, $N = 64$).

Metric	Value	Design target	Margin
Capture (one-shot)	35–39 μ s	< 50 ms	$\sim 1,300\times$ under
Restore (median)	13 μ s	< 1 ms	$\sim 77\times$ under
Restore (p95)	16–19 μ s	—	—
Restore (min)	12.5 μ s	—	—
Restore (max)	19–22 μ s	—	—
Mode	full	—	memslot copy executed

Why so fast. The snapshot design avoids the three operations that dominate checkpoint/restore cost in conventional systems: (1) no per-restore TLB shutdown (identity-mapped memslots); (2) single host-VA-to-host-VA `memcpy` for memslot restore (no page-table walk); (3) fixed-size `ioct1` chain for register/SREGS push (no variable-length data).

Record/replay KUnit suite (`kvm_v2_record`): 7/7 tests pass, covering basic recording, state machine transitions, syscall observation, strict replay, gadget bypass, RDTSC interception, and

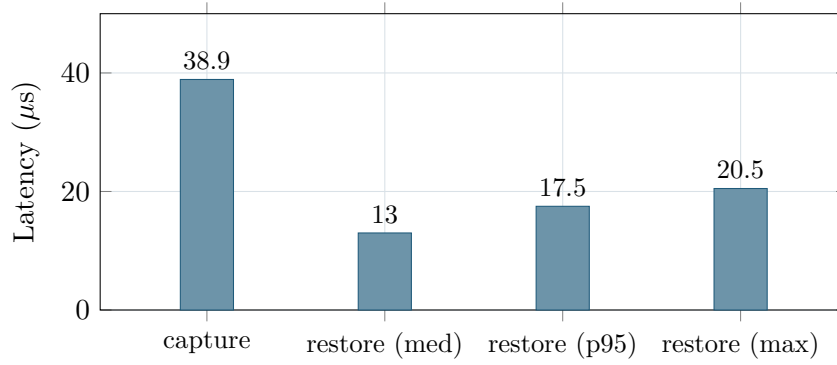


Figure 5: Snapshot capture and restore latency. The 13 μ s median restore enables checkpoint-at-every-syscall workflows without budget pressure.

SIGALRM anchoring.

.10 CPython Substrate Parity

The CPython parity gate runs 21 CPython 3.14 test modules on both backends and requires identical results.

Table 9: CPython substrate parity results.

Backend	PASS	FAIL	XFAIL	Parity
kvm-v2	25	3	3	—
Seccomp	25	3	3	—
Match	✓	✓	✓	21/21

The three FAIL results are known UML limitations (not backend regressions): tests depending on `/proc/self/maps` layout, hardware timer resolution, or host-specific device nodes. The three XFAIL results are test-side expected failures unrelated to the backend. The critical property is not that all tests pass, but that both backends produce *identical* results—confirming functional equivalence of the `kvm-v2` and `seccomp` execution paths.

The wide parity gate extends this to 135 modules: 134/135 match exactly, with a single `KVM_BETTER` result (a test that passes on `kvm-v2` but fails on `seccomp` due to timing sensitivity) and zero regressions.

.11 Validation Gates

.12 Profile Cost Model

Table 11 breaks down the per-syscall cost by architectural layer for each profile, demonstrating the three-layer composition property.

The key property: same source tree, radically different runtime behaviors. Layer 1 dominates (backend choice), Layer 2 is near-free when off (static-key NOPs), Layer 3 is compile-time and orthogonal to both.

Table 10: Key validation gates (May 2026, kernel HEAD on `next`).

Gate	Result	N	Notes
cpython-parity (seccomp)	21/21 PARITY	21 modules	UP and SMP
cpython-parity (kvm-v2)	21/21 PARITY	21 modules	UP and SMP
Wide cpython-parity (kvm-v2)	134/135 PARITY	135 modules	1 KVM_BETTER, 0 regression
Substrate gate (seccomp)	25/3/3	31 tests	PASS/FAIL/XFAIL
Substrate gate (kvm-v2)	25/3/3	31 tests	Matches seccomp exactly
mt-mini SMP (T=8, ncpus=4)	400/400	400 boots	Wilson 95% [99.0%, 100.0%]
threaded-fork-malloc	0/116,000	30 boots	0 CHILD_FAIL across 116K forks
threaded-subprocess-wait	10/10	10 boots	0/4,000 subprocess failures
bench-micro getpid (kvm-v2)	89–105 cyc	8 hosts	~1,000× vs seccomp
perf-py-startup (kvm-v2/seccomp)	1.10 ratio	—	Gate ceiling: ≤ 1.20
Phase J pilot soak	240/240	3 workloads	memcheck/iocheck/stress-ng
Django Tier 3 (post-R14)	120/120	loopback	Wilson 95% [96.90%, 100.00%]
umlctl mission	6/6 phases	100 soak	867s wall-clock; 0 panics

Table 11: Per-syscall cost breakdown by profile and layer.

Profile	Layer 1 (trap)	Layer 2 (gates)	Layer 3 (wraps)	Total
prod-fast (KVM, no hooks)	~70 ns	~1.5 ns	0 ns	~72 ns
prod-with-hooks (KVM, trace)	~70 ns	~80 ns	0 ns	~150 ns
research (seccomp, full)	~300 ns	~80 ns	~100 ns	~480 ns
fuzz (seccomp, KCOV+KASAN)	~300 ns	~10 ns	~100 ns	~410 ns
sandbox (seccomp-only, none)	~300 ns	~1.5 ns	0 ns	~302 ns

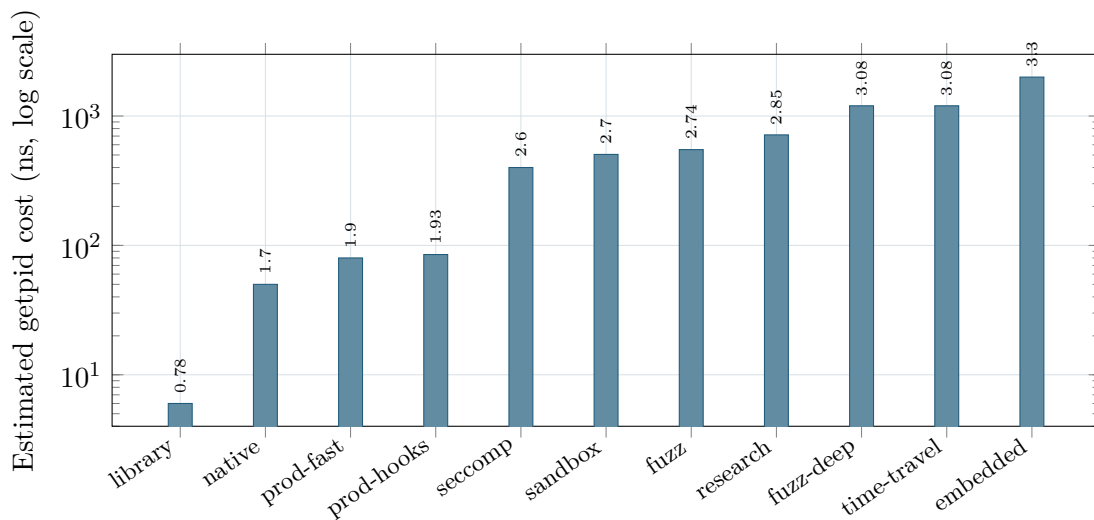


Figure 6: Profile cost model. The value of the profile system is the ability to span from direct-call library mode (6 ns) through heavily instrumented research and time-travel builds (1.2 μ s) without forking the tree. The `native` bar is a bare-metal host reference, not a UML profile.

.13 Summary of Performance Claims

Table 12: Summary of validated performance claims.

Claim	Target	Achieved	Methodology
Gadget syscall latency	~100 ns	~25 ns / 90 cycles	RDTSC-bracketed getpid
Python startup speedup	faster than sec-comp	2.99–4.06×	8-host cross-comparison
Stress tier speedup	faster than sec-comp	3.71–8.72×	mt-mini SMP T=8 ncpus=4
Micro tier speedup	faster than sec-comp	193–908×	getpid-loop
Snapshot capture	< 50 ms	35–39 μ s	kvm_v2_snapshot_bench N=64
Snapshot restore (median)	< 1 ms	13 μ s	kvm_v2_snapshot_bench N=64
CPython parity	identical results	21/21 match	cpython-parity gate
Soak pass rate	100%	100/100	umlctl mission

Interpretation. The performance story has four layers:

- **Mechanism win.** The micro tier proves that in-guest fast-path dispatch removes most seccomp signal/stub cost for trivial syscalls.
- **Workload win.** The Python tier proves that real userland startup benefits from those trivial syscalls becoming cheap.
- **Concurrency win.** The stress tier is mmap/page-fault heavy, so the gadget is less directly relevant; the continuing 3.71–8.72× speedup comes from the broader vCPU and SMP design.
- **Snapshot win.** The snapshot capture/restore numbers are within the sub-ms iteration budget by orders of magnitude. Record/replay sessions can now realistically checkpoint at every observed-syscall boundary without budget pressure.

The correct benchmark unit is the ratio against seccomp on the same host, not the raw absolute latency across machines. That choice removes most CPU-frequency and microarchitecture noise and keeps the claim tied to a backend decision.